

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN 2
RANKING TRONG HỌC MÁY

Mã số sinh viên	Họ và tên
23120245	NGUYỄN QUANG DUY
23120258	LƯU TRỌNG HIẾU
23120260	VĂN ĐÌNH HIẾU

Giảng viên hướng dẫn TS Bùi Tiến Lên
Th.S Lê Nhật Nam

Môn học Nhập môn Học máy
Thành phố Hồ Chí Minh – 2026

Tóm tắt

Báo cáo này trình bày chủ đề Ranking trong học máy, tập trung vào các khái niệm nền tảng như hàm điểm, quan hệ ưu tiên, ranking loss và các hướng tiếp cận pointwise, pairwise, listwise. Phần thực nghiệm lựa chọn pairwise ranking làm trọng tâm, trong đó mô hình scoring tuyến tính được cài đặt từ đầu bằng NumPy và huấn luyện bằng pairwise hinge loss trên dữ liệu tổng hợp. Các kết quả thực nghiệm được đánh giá thông qua pairwise accuracy, ranking loss và Kendall tau distance nhằm minh họa mối liên hệ giữa lý thuyết và thực hành.

Danh mục từ viết tắt

Từ viết tắt	Ý nghĩa
LTR	Learning to Rank
ML	Machine Learning
SVM	Support Vector Machine
ROC	Receiver Operating Characteristic
AUC	Area Under the ROC Curve
TPR	True Positive Rate
FPR	False Positive Rate
NDCG	Normalized Discounted Cumulative Gain
DCG	Discounted Cumulative Gain
IDCG	Ideal Discounted Cumulative Gain
MAP	Mean Average Precision
AP	Average Precision
MRR	Mean Reciprocal Rank
RR	Reciprocal Rank

Mục lục

Tóm tắt	i
Danh mục từ viết tắt	ii
Danh mục hình vẽ	ix
Danh mục bảng	x
1 Giới thiệu	1
2 Kiến thức chuẩn bị	2
2.1 Bài toán xếp hạng	2
2.2 Hàm điểm và ranking dựa trên điểm số	2
2.3 Quan hệ ưu tiên theo từng cặp	3
2.4 Lỗi xếp hạng theo từng cặp	3
2.5 Margin trong ranking	4
2.6 Score-based ranking và preference-based ranking	4
2.7 Bipartite ranking và AUC	4
2.8 Tóm tắt	5
3 Score-based ranking và chặn tổng quát hoá	5
3.1 Thiết lập score-based ranking	6
3.2 Lỗi tổng quát và lỗi thực nghiệm	7
3.3 Margin loss cho ranking	7
3.4 Rademacher complexity	8
3.5 Định lý chặn margin cho ranking	9
3.6 Chứng minh Định lý 3.1	10
3.7 [MỞ RỘNG] Giải thích bước Lipschitz của Φ_ρ	10
3.8 Hệ quả cho lớp hàm kernel	11
3.9 [MỞ RỘNG] Phân tích độ phức tạp của score-based ranking	11
3.10 Ví dụ minh hoạ về margin	13
3.11 Tóm tắt	14
4 Ranking với Support Vector Machines	14
4.1 Từ chặn margin đến bài toán tối ưu	15
4.2 Bài toán tối ưu primal của SVM-ranking	15
4.3 Liên hệ với pairwise hinge loss	16
4.4 Ranking như một bài toán SVM trên vector hiệu	16
4.5 Hình học của SVM-ranking	17
4.6 Dạng dual và kernel cho ranking	18

4.7	Chứng minh liên hệ giữa SVM-ranking và SVM chuẩn	19
4.8	Ví dụ minh hoạ	20
4.9	[MỞ RỘNG] Phân tích độ phức tạp của SVM-ranking	20
4.10	[MỞ RỘNG] So sánh với SVM phân loại nhị phân	21
4.11	Hạn chế của SVM-ranking	21
4.12	Tóm tắt	22
5	RankBoost	22
5.1	Thiết lập của RankBoost	23
5.2	Ý tưởng lặp của RankBoost	23
5.3	Phân phối trọng số trên các cặp	24
5.4	Các đại lượng ϵ_t^+ , ϵ_t^- , ϵ_t^0	24
5.5	Trọng số của base ranker	25
5.6	Cập nhật phân phối trọng số	26
5.7	Thuật toán RankBoost	27
5.8	Biểu diễn phân phối sau nhiều vòng	27
5.9	Chặn lỗi thực nghiệm của RankBoost	28
5.10	Chứng minh Định lý 5.1	29
5.11	[MỞ RỘNG] Vì sao α_t tối thiểu hoá Z_t	30
5.12	RankBoost và coordinate descent	30
5.13	Ví dụ minh hoạ	31
5.14	[MỞ RỘNG] Phân tích độ phức tạp của RankBoost	31
5.15	Liên hệ với AdaBoost	32
5.16	Tóm tắt	33
6	Bipartite ranking, ROC và AUC	33
6.1	Thiết lập bipartite ranking	33
6.2	Lỗi tổng quát trong bipartite ranking	34
6.3	Lỗi thực nghiệm	34
6.4	So sánh binary classification và bipartite ranking	35
6.5	ROC curve	35
6.6	AUC	36
6.7	Quan hệ giữa AUC và ranking loss	36
6.8	Cách tính AUC trên mẫu hữu hạn	37
6.9	Ví dụ minh hoạ cách tính AUC	37
6.10	[MỞ RỘNG] Phân tích độ phức tạp tính AUC	38
6.11	Liên hệ với các thuật toán ranking	38
6.12	Tóm tắt	38
7	Preference-based ranking	39

7.1	Thiết lập preference-based ranking	39
7.2	Hai giai đoạn của preference-based ranking	40
7.3	Ranking và loss giữa hai thứ tự	40
7.4	Regret trong preference-based ranking	41
7.5	Thuật toán sort-by-degree	41
7.6	Thuật toán sort-by-degree	42
7.7	Hạn chế của sort-by-degree	42
7.8	Randomized QuickSort cho preference-based ranking	43
7.9	Thuật toán randomized QuickSort	44
7.10	So sánh sort-by-degree và randomized QuickSort	44
7.11	[MỞ RỘNG] Phân tích độ phức tạp	44
7.11.1	Độ phức tạp của sort-by-degree	44
7.11.2	Độ phức tạp của randomized QuickSort	45
7.12	Mở rộng sang weighted loss	45
7.13	Ví dụ minh hoạ	45
7.14	Tóm tắt	46
8	Thảo luận về các tiêu chí đánh giá ranking	46
8.1	Pairwise misranking và ranking loss	47
8.2	Margin-based ranking criteria	47
8.3	Kendall tau distance	48
8.4	AUC trong bipartite ranking	48
8.5	Precision@k, MAP và MRR	49
8.6	DCG và NDCG	50
8.7	So sánh các tiêu chí đánh giá	51
8.8	Liên hệ với thực nghiệm trong báo cáo	52
9	Thực nghiệm	52
9.1	Thiết lập thực nghiệm	53
9.2	Cài đặt thuật toán pairwise ranking từ đầu	53
9.3	Các độ đo đánh giá	56
9.3.1	Pairwise accuracy	56
9.3.2	Ranking loss	56
9.3.3	Kendall tau distance	56
9.3.4	Top-k overlap	57
9.4	Kết quả thực nghiệm chính	57
9.5	Minh hoạ quá trình hội tụ	57
9.6	So sánh ranking thật và ranking dự đoán	58
9.7	Ảnh hưởng của nhiễu đến chất lượng ranking	59

9.8	Ảnh hưởng của kích thước mẫu	60
9.9	Khả năng tái tạo thực nghiệm	61
9.10	Nhận xét chung	61
10	Tổng hợp và thảo luận các bài báo liên quan	61
10.1	Bài báo 1: Which Tricks are Important for Learning to Rank?	62
10.1.1	Bài toán đặt ra	62
10.1.2	Đóng góp chính	62
10.1.3	Liên hệ với lý thuyết trong chương	63
10.1.4	Hạn chế và nhận xét	63
10.2	Bài báo 2: Decision-Focused Learning: Through the Lens of Learning to Rank	64
10.2.1	Bài toán đặt ra	64
10.2.2	Đóng góp chính	64
10.2.3	Liên hệ với lý thuyết trong chương	65
10.2.4	Hạn chế và nhận xét	65
10.3	Bài báo 3: Linear Label Ranking with Bounded Noise	66
10.3.1	Bài toán đặt ra	66
10.3.2	Đóng góp chính	66
10.3.3	Liên hệ với lý thuyết trong chương	67
10.3.4	Hạn chế và nhận xét	67
10.4	Nhận xét tổng quan về xu hướng phát triển	67
10.4.1	Từ pairwise ranking sang tối ưu metric trực tiếp	67
10.4.2	Sử dụng smoothing và surrogate loss để xử lý tính không khả vi	68
10.4.3	Mở rộng learning-to-rank ra ngoài truy xuất thông tin	68
10.4.4	Tăng cường phân tích lý thuyết cho ranking có nhiễu	68
10.5	Hạn chế chung của các phương pháp	68
10.6	Câu hỏi mở và ý tưởng cải tiến	69
10.7	Kết luận	69
11	Kết luận	70
A	Phụ lục: Cấu trúc mã nguồn và hướng dẫn tái tạo thực nghiệm	72
A.1	Cấu trúc thư mục	72
A.2	Môi trường thực nghiệm	73
A.3	Cách chạy thực nghiệm chính	73
A.4	Cách chạy toàn bộ thực nghiệm	73
A.5	Các file kết quả	74
A.6	Khả năng tái tạo	74
A.7	Ghi chú về kiểm thử	75

Tài liệu tham khảo

76

Danh sách hình vẽ

1	Score-based ranking tạo thứ tự bằng cách sắp xếp các đối tượng theo điểm số $h(x)$	3
2	Quy trình tổng quát của score-based ranking: học hàm điểm từ các cặp ưu tiên, sau đó dùng điểm số để sinh ranking toàn cục.	6
3	Hàm mất mát margin Φ_ρ . Khi $u \leq 0$, cặp bị xếp sai hoặc hoà nên bị phạt tối đa. Khi $0 < u < \rho$, cặp đã đúng chiều nhưng margin chưa đủ lớn nên vẫn bị phạt tuyến tính. Khi $u \geq \rho$, cặp được xem là đúng với độ tin cậy đủ lớn và không bị phạt.	8
4	Ba thành phần trong chặn tổng quát hoá: lỗi trên tập huấn luyện, độ phức tạp của lớp giả thuyết và sai số do lấy mẫu hữu hạn.	9
5	Số cặp ưu tiên tăng theo bậc hai khi số đối tượng n tăng. Với $n = 100$, nếu xét toàn bộ các cặp không lặp thì có $\binom{100}{2} = 4950$ cặp.	12
6	Minh hoạ margin giữa các tài liệu trên trục điểm số. Cặp (d_2, d_1) có margin lớn hơn, trong khi cặp (d_3, d_2) có margin nhỏ nên kém chắc chắn hơn dù vẫn được xếp đúng.	13
7	Ý tưởng chính của SVM-ranking: biến mỗi cặp ưu tiên thành một mẫu phân loại trên vector hiệu.	15
8	Hai thành phần trong hàm mục tiêu của SVM-ranking: chuẩn của w kiểm soát độ phức tạp, còn các biến slack đo mức vi phạm margin.	16
9	Hình học của SVM-ranking trong không gian vector hiệu $z = \Psi(x, x')$. Đường $w^\top z = 0$ đóng vai trò siêu phẳng phân tách, còn hai đường biên margin tương ứng với $w^\top z = 1$ và $w^\top z = -1$. Các điểm có nhãn $y = +1$ và $y = -1$ biểu diễn các cặp ưu tiên sau khi được biến đổi thành vector hiệu.	18
10	Kernel trick trong SVM-ranking: từ kernel giữa các đối tượng ban đầu, ta xây dựng kernel giữa các cặp thông qua vector hiệu.	19
11	Vòng lặp chính của RankBoost. Ở mỗi vòng, thuật toán chọn một base ranker tốt theo phân phối hiện tại, gán trọng số cho nó, rồi cập nhật phân phối để tập trung hơn vào các cặp khó.	23
12	Trực giác cập nhật trọng số trong RankBoost: cặp xếp sai được tăng trọng số để thuật toán chú ý hơn ở các vòng tiếp theo.	24
13	Phân phối trọng số D_t được chia thành ba nhóm: tổng trọng số của cặp xếp đúng, cặp xếp sai và cặp hoà.	25
14	Trọng số α_t tăng khi base ranker xếp đúng nhiều hơn xếp sai theo phân phối hiện tại.	25

15	Ảnh hưởng của α lên hệ số nhân trọng số trong RankBoost. Khi α tăng, cặp xếp đúng được giảm trọng số theo $e^{-\alpha}$, cặp xếp sai được tăng trọng số theo e^{α} , còn cặp hoà giữ nguyên hệ số 1.	26
16	Nếu mỗi vòng tìm được base ranker có edge dương, chặn trên của lỗi thực nghiệm giảm theo hàm mũ theo số vòng boosting.	28
17	Góc nhìn coordinate descent: RankBoost chọn một hướng h_t rồi tối ưu hệ số α_t theo hướng đó.	31
18	Ví dụ phân loại ba cặp thành nhóm đúng, sai hoặc hoà theo base ranker h_1	31
19	AdaBoost và RankBoost có cùng tinh thần boosting, nhưng khác nhau ở đối tượng được gán trọng số.	32
20	Hai giai đoạn của preference-based ranking.	40
21	Đường cong pairwise hinge loss trong quá trình huấn luyện. Loss giảm nhanh ở các epoch đầu và tiếp tục giảm chậm dần về sau, cho thấy quá trình tối ưu đang hội tụ.	58
22	So sánh thứ tự ranking thật và ranking dự đoán. Hai đường càng gần nhau thì mô hình càng bảo toàn tốt quan hệ thứ tự giữa các mẫu.	58
23	Ảnh hưởng của độ lệch chuẩn nhiễu đến chất lượng ranking. Khi nhiễu tăng, pairwise accuracy giảm, trong khi ranking loss và Kendall tau distance tăng.	59
24	Ảnh hưởng của kích thước mẫu đến chất lượng ranking. Khi số mẫu tăng, pairwise accuracy nhìn chung tăng và ranking loss giảm, sau đó ổn định ở mức tốt.	60

Danh sách bảng

1	Tóm tắt độ phức tạp của score-based pairwise ranking với mô hình tuyến tính.	13
2	SVM-ranking có cùng cấu trúc với SVM phân loại, nhưng dữ liệu đầu vào là các vector hiệu sinh từ từng cặp đối tượng.	17
3	Tóm tắt chi phí chính của SVM-ranking khi làm việc trên dữ liệu pairwise.	21
4	Tóm tắt độ phức tạp của RankBoost trong thiết lập pairwise tổng quát.	32
5	So sánh một số tiêu chí đánh giá ranking	51
6	Kết quả thực nghiệm chính của mô hình pairwise ranking	57
7	Ảnh hưởng của nhiễu đến chất lượng ranking	59
8	Ảnh hưởng của kích thước mẫu đến chất lượng ranking	60

1 Giới thiệu

Trong nhiều bài toán học máy hiện đại, mục tiêu của mô hình không chỉ là dự đoán một nhãn rời rạc hoặc một giá trị số, mà còn là sắp xếp một tập các đối tượng theo một thứ tự mong muốn. Bài toán này được gọi là *ranking* hoặc *learning to rank*. Ranking xuất hiện tự nhiên trong nhiều ứng dụng thực tế như xếp hạng kết quả tìm kiếm, gợi ý sản phẩm, sắp xếp tài liệu theo mức độ liên quan, xếp hạng ứng viên, hoặc đánh giá mức độ ưu tiên của các hành động trong một hệ thống ra quyết định.

Khác với bài toán phân loại, trong đó mỗi mẫu thường được gán vào một lớp cụ thể, ranking quan tâm đến quan hệ thứ tự giữa các đối tượng. Ví dụ, trong hệ thống tìm kiếm, điều quan trọng không chỉ là xác định một tài liệu có liên quan hay không, mà còn là tài liệu nào nên được đặt ở vị trí cao hơn trong danh sách kết quả. Do đó, ranking thường được mô hình hoá thông qua các hàm điểm, quan hệ ưu tiên giữa các cặp đối tượng, hoặc các độ đo đánh giá chất lượng của toàn bộ danh sách xếp hạng.

Trong báo cáo này, nhóm tập trung tìm hiểu chủ đề Ranking dựa trên nội dung trong sách *Foundations of Machine Learning*. Mục tiêu chính là trình bày lại các khái niệm nền tảng của ranking bằng tiếng Việt, giải thích ý nghĩa trực giác của các định nghĩa và công thức, đồng thời minh hoạ lý thuyết bằng thực nghiệm lập trình. Báo cáo không chỉ xem ranking như một bài toán ứng dụng, mà còn phân tích các cách tiếp cận lý thuyết phổ biến như *pointwise ranking*, *pairwise ranking* và *listwise ranking*.

Trong ba hướng tiếp cận trên, phần thực nghiệm của báo cáo chọn *pairwise ranking* làm trọng tâm. Lý do là pairwise ranking có mối liên hệ trực tiếp với quan hệ ưu tiên giữa hai đối tượng, ranking loss, và reduction từ ranking sang classification. Bên cạnh đó, hướng tiếp cận này có thể được cài đặt từ đầu bằng các phép toán tuyến tính cơ bản, phù hợp với mục tiêu minh hoạ lý thuyết bằng Python và NumPy. Cụ thể, nhóm xây dựng một mô hình scoring tuyến tính, sinh dữ liệu tổng hợp, tạo các cặp preference, huấn luyện mô hình bằng pairwise hinge loss, và đánh giá kết quả thông qua các độ đo như pairwise accuracy, ranking loss và Kendall tau distance.

Báo cáo được tổ chức như sau. Phần đầu trình bày các khái niệm chuẩn bị cần thiết cho bài toán ranking, bao gồm scoring function, preference relation và ranking loss. Tiếp theo, báo cáo giới thiệu ba hướng tiếp cận chính trong learning to rank: pointwise, pairwise và listwise. Sau đó, phần lý thuyết chính tập trung vào pairwise ranking, cách xây dựng dữ liệu dạng cặp, hàm mất mát pairwise hinge loss và mối liên hệ với bài toán classification. Phần thực nghiệm mô tả quá trình cài đặt mô hình ranking tuyến tính từ đầu, thiết kế dữ liệu tổng hợp, các độ đo đánh giá và phân tích kết quả. Cuối cùng, báo cáo thảo luận một số hướng nghiên cứu hiện đại liên quan đến ranking, đồng thời tổng kết những kết quả đạt được và các hạn chế còn tồn tại.

2 Kiến thức chuẩn bị

Chương Ranking nghiên cứu các bài toán học máy trong đó đầu ra của mô hình là một thứ tự xếp hạng giữa các đối tượng. Khác với phân loại, ranking không chỉ hỏi một mẫu thuộc lớp nào, mà quan tâm đối tượng nào nên đứng trước đối tượng nào. Bài toán này xuất hiện trong công cụ tìm kiếm, hệ thống gợi ý, truy xuất thông tin, xếp hạng sản phẩm, xếp hạng phim và phát hiện gian lận.

2.1 Bài toán xếp hạng

Gọi $\mathcal{X}$ là không gian đầu vào. Với một tập hữu hạn gồm n đối tượng:

$$X = \{x_1, x_2, \dots, x_n\} \subseteq \mathcal{X},$$

mục tiêu của ranking là tìm một thứ tự sao cho các đối tượng quan trọng hơn hoặc liên quan hơn được đặt ở vị trí cao hơn. Nếu x_i nên được xếp trước x_j , ta viết:

$$x_i \succ x_j.$$

Một ranking đầy đủ có thể được biểu diễn bởi một hoán vị π của tập chỉ số $\{1, 2, \dots, n\}$:

$$x_{\pi(1)} \succ x_{\pi(2)} \succ \dots \succ x_{\pi(n)}.$$

2.2 Hàm điểm và ranking dựa trên điểm số

Một cách phổ biến để tạo ranking là học một hàm điểm:

$$h : \mathcal{X} \rightarrow \mathbb{R}.$$

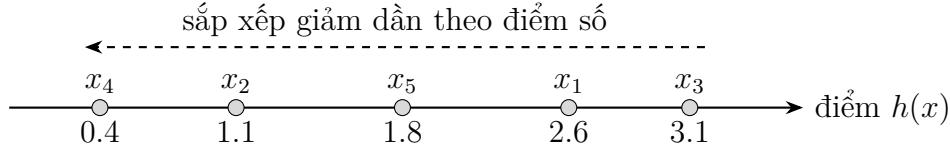
Hàm h gán cho mỗi đối tượng một điểm số thực. Nếu $h(x_i) > h(x_j)$, mô hình xếp x_i cao hơn x_j . Do đó, ranking được tạo bằng cách sắp xếp các đối tượng theo điểm số giảm dần:

$$h(x_{\pi(1)}) \geq h(x_{\pi(2)}) \geq \dots \geq h(x_{\pi(n)}).$$

Trong nhiều mô hình, mỗi đối tượng được biểu diễn bởi vector đặc trưng $x_i \in \mathbb{R}^d$. Một lớp hàm điểm đơn giản và quan trọng là hàm tuyến tính:

$$h_w(x) = w^\top x,$$

trong đó $w \in \mathbb{R}^d$ là vector tham số cần học.



Hình 1: Score-based ranking tạo thứ tự bằng cách sắp xếp các đối tượng theo điểm số $h(x)$.

2.3 Quan hệ ưu tiên theo từng cặp

Dữ liệu huấn luyện trong ranking thường được cho dưới dạng các cặp đối tượng:

$$(x_i, x'_i, y_i) \in \mathcal{X} \times \mathcal{X} \times \{-1, 0, +1\}.$$

Nhãn y_i biểu diễn quan hệ ưu tiên giữa x_i và x'_i :

$$y_i = \begin{cases} +1, & \text{nếu } x'_i \text{ được ưu tiên hơn } x_i, \\ -1, & \text{nếu } x_i \text{ được ưu tiên hơn } x'_i, \\ 0, & \text{nếu hai đối tượng ngang nhau hoặc không có thông tin.} \end{cases}$$

Với quy ước này, một cặp được xếp đúng khi:

$$y_i(h(x'_i) - h(x_i)) > 0. \quad (1)$$

Biểu thức trên kiểm tra xem dấu của hiệu điểm giữa hai đối tượng có phù hợp với nhãn ưu tiên hay không.

2.4 Lỗi xếp hạng theo từng cặp

Chất lượng của hàm điểm h có thể được đo bằng xác suất xếp sai một cặp đối tượng. Gọi D là phân phối trên $\mathcal{X} \times \mathcal{X}$ và f là hàm ưu tiên mục tiêu. Lỗi tổng quát được định nghĩa là:

$$R(h) = \Pr_{(x, x') \sim D} [f(x, x') \neq 0 \wedge f(x, x')(h(x') - h(x)) \leq 0]. \quad (2)$$

Trong thực tế, ta chỉ có mẫu huấn luyện:

$$S = ((x_1, x'_1, y_1), \dots, (x_m, x'_m, y_m)).$$

Lỗi thực nghiệm trên S là:

$$\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1} [y_i \neq 0 \wedge y_i(h(x'_i) - h(x_i)) \leq 0]. \quad (3)$$

Đây là tỉ lệ các cặp huấn luyện bị mô hình xếp sai hoặc không phân biệt được đúng chiều.

2.5 Margin trong ranking

Tương tự classification, ranking cũng có thể được phân tích thông qua margin. Với một cặp (x_i, x'_i, y_i) , margin theo cặp là:

$$M_i(h) = y_i(h(x'_i) - h(x_i)). \quad (4)$$

Nếu $M_i(h) > 0$, cặp được xếp đúng. Nếu $M_i(h) \leq 0$, cặp bị xếp sai hoặc hoà điểm. Với một ngưỡng $\rho > 0$, lỗi margin thực nghiệm được viết là:

$$\hat{R}_\rho(h) = \frac{1}{m} \sum_{i=1}^m \Phi_\rho(y_i(h(x'_i) - h(x_i))), \quad (5)$$

trong đó Φ_ρ là hàm mất mát margin. Trực giác là mô hình không chỉ cần xếp đúng, mà còn cần tạo ra khoảng cách điểm đủ lớn giữa hai đối tượng.

2.6 Score-based ranking và preference-based ranking

Có hai thiết lập chính trong ranking. Trong *score-based ranking*, mô hình học một hàm điểm:

$$h : \mathcal{X} \rightarrow \mathbb{R},$$

rồi sinh ranking bằng cách sắp xếp các đối tượng theo điểm số.

Trong *preference-based ranking*, mô hình học trực tiếp một hàm ưu tiên:

$$h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1].$$

Giá trị $h(u, v)$ càng lớn thì mô hình càng tin rằng u nên được xếp trước v . Có thể tóm tắt sự khác nhau như sau:

$$\text{score-based: } x \mapsto h(x), \quad \text{preference-based: } (u, v) \mapsto h(u, v).$$

2.7 Bipartite ranking và AUC

Một trường hợp quan trọng của ranking là *bipartite ranking*, trong đó không gian đầu vào được chia thành hai lớp:

$$\mathcal{X} = \mathcal{X}_+ \cup \mathcal{X}_-, \quad \mathcal{X}_+ \cap \mathcal{X}_- = \emptyset.$$

Mục tiêu là học một hàm điểm h sao cho các điểm positive được xếp cao hơn các điểm negative:

$$h(x^+) > h(x^-), \quad x^+ \in \mathcal{X}_+, \quad x^- \in \mathcal{X}_-.$$

Nếu có m positive points và n negative points, lỗi thực nghiệm trong bipartite ranking là:

$$\hat{R}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1} [h(x_i^+) < h(x_j^-)]. \quad (6)$$

AUC có thể được hiểu là tỉ lệ các cặp positive-negative được xếp đúng:

$$\text{AUC}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1} [h(x_i^+) \geq h(x_j^-)]. \quad (7)$$

Do đó, nếu bỏ qua trường hợp hoà điểm hoặc xử lý hoà điểm theo một quy ước cố định, ta có:

$$\text{AUC}(h) = 1 - \hat{R}(h). \quad (8)$$

2.8 Tóm tắt

Phần này đã giới thiệu các khái niệm nền tảng của bài toán ranking. Ranking có thể được sinh ra từ một hàm điểm hoặc từ quan hệ ưu tiên theo từng cặp. Trong score-based ranking, mô hình học $h : \mathcal{X} \rightarrow \mathbb{R}$; trong preference-based ranking, mô hình học trực tiếp $h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$. Lỗi ranking thường được đo bằng tỉ lệ cặp bị xếp sai. Riêng trong bipartite ranking, AUC đo tỉ lệ các cặp positive-negative được xếp đúng.

3 Score-based ranking và chặn tổng quát hoá

Trong phần trước, ta đã giới thiệu các khái niệm cơ bản của bài toán ranking như hàm điểm, quan hệ ưu tiên theo từng cặp, lỗi xếp hạng và margin. Phần này tập trung vào thiết lập *score-based ranking*. Trong thiết lập này, mô hình học một hàm điểm

$$h : \mathcal{X} \rightarrow \mathbb{R},$$

sau đó sinh ranking bằng cách sắp xếp các đối tượng theo điểm số giảm dần.

Mục tiêu chính của phần này là trình bày chặn tổng quát hoá dựa trên margin cho bài toán ranking. Kết quả này cho thấy nếu một hàm điểm có lỗi margin nhỏ trên tập huấn luyện và lớp giả thuyết không quá phức tạp, thì lỗi xếp hạng trên dữ liệu chưa thấy cũng có thể được kiểm soát. Nói cách khác, score-based ranking không chỉ yêu cầu mô hình xếp đúng các cặp đối tượng, mà còn cần tạo ra khoảng cách điểm đủ lớn để thứ tự dự đoán ổn định hơn.

3.1 Thiết lập score-based ranking

Gọi $\mathcal{X}$ là không gian đầu vào. Trong score-based ranking, ta xét một lớp giả thuyết gồm các hàm điểm:

$$\mathcal{H} \subseteq \{h \mid h : \mathcal{X} \rightarrow \mathbb{R}\}.$$

Với $h \in \mathcal{H}$, giá trị $h(x)$ là điểm số của đối tượng x . Nếu

$$h(x') > h(x),$$

thì mô hình xếp x' cao hơn x . Vì ranking được sinh ra bằng cách sắp xếp điểm số, mọi thứ tự do hàm điểm tạo ra đều có tính bắc cầu. Đây là ưu điểm về mặt đơn giản hoá mô hình, nhưng cũng là hạn chế nếu quan hệ ưu tiên thật có thể không bắc cầu.

Dữ liệu huấn luyện gồm m cặp đối tượng có nhãn:

$$S = ((x_1, x'_1, y_1), \dots, (x_m, x'_m, y_m)), \quad y_i \in \{-1, 0, +1\}.$$

Các cặp (x_i, x'_i) được lấy từ một phân phối chưa biết D trên $\mathcal{X} \times \mathcal{X}$. Nhãn được xác định bởi hàm ưu tiên mục tiêu

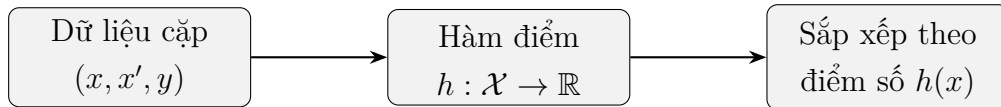
$$f : \mathcal{X} \times \mathcal{X} \rightarrow \{-1, 0, +1\}.$$

Theo quy ước, $f(x, x') = +1$ nghĩa là x' được ưu tiên hơn x , $f(x, x') = -1$ nghĩa là x được ưu tiên hơn x' , và $f(x, x') = 0$ nghĩa là hai đối tượng ngang nhau hoặc không có đủ thông tin.

Với một cặp có nhãn khác 0, hàm điểm h xếp đúng nếu

$$f(x, x')(h(x') - h(x)) > 0. \quad (9)$$

Nếu biểu thức trên nhỏ hơn hoặc bằng 0, mô hình xếp sai hoặc không phân biệt được đúng thứ tự giữa hai đối tượng.



Hình 2: Quy trình tổng quát của score-based ranking: học hàm điểm từ các cặp ưu tiên, sau đó dùng điểm số để sinh ranking toàn cục.

3.2 Lỗi tổng quát và lỗi thực nghiệm

Lỗi tổng quát của hàm điểm h là xác suất mô hình xếp sai một cặp được lấy ngẫu nhiên từ phân phối D :

$$R(h) = \Pr_{(x, x') \sim D} [f(x, x') \neq 0 \wedge f(x, x')(h(x') - h(x)) \leq 0]. \quad (10)$$

Đại lượng $R(h)$ là lỗi thật mà ta quan tâm, vì nó đo chất lượng ranking trên dữ liệu chưa thấy. Tuy nhiên, do phân phối D và hàm mục tiêu f không được biết đầy đủ, ta dùng lỗi thực nghiệm trên mẫu huấn luyện:

$$\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1} [y_i \neq 0 \wedge y_i(h(x'_i) - h(x_i)) \leq 0]. \quad (11)$$

Để đơn giản hoá phân tích lý thuyết, phần này thường giả sử nhãn chỉ thuộc $\{-1, +1\}$, tức là mỗi cặp đều có một quan hệ ưu tiên rõ ràng. Khi đó, điều kiện $y_i \neq 0$ có thể được bỏ đi trong công thức lỗi thực nghiệm.

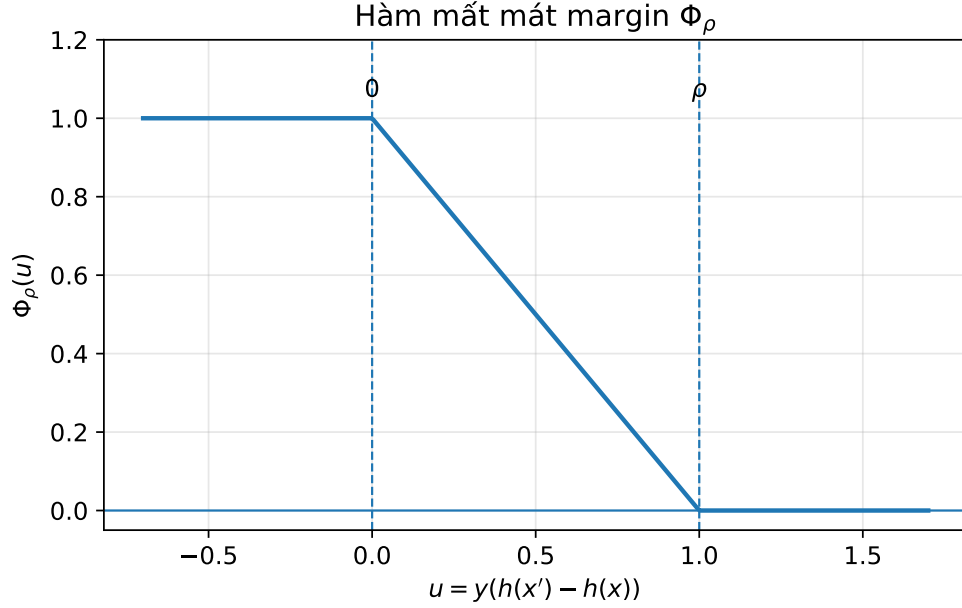
3.3 Margin loss cho ranking

Trong ranking, việc một cặp được xếp đúng chưa hẳn đã đủ. Ta còn muốn mô hình xếp đúng với một khoảng cách điểm đủ lớn. Với một cặp huấn luyện (x_i, x'_i, y_i) , margin theo cặp được định nghĩa là:

$$M_i(h) = y_i(h(x'_i) - h(x_i)). \quad (12)$$

Nếu $M_i(h) > 0$, cặp thứ i được xếp đúng. Nếu $M_i(h) \leq 0$, cặp đó bị xếp sai. Với một tham số $\rho > 0$, ta định nghĩa hàm mất mát margin:

$$\Phi_\rho(u) = \begin{cases} 1, & u \leq 0, \\ 1 - \frac{u}{\rho}, & 0 < u < \rho, \\ 0, & u \geq \rho. \end{cases} \quad (13)$$



Hình 3: Hàm mất mát margin Φ_ρ . Khi $u \leq 0$, cặp bị xếp sai hoặc hoà nên bị phạt tối đa. Khi $0 < u < \rho$, cặp đã đúng chiều nhưng margin chưa đủ lớn nên vẫn bị phạt tuyến tính. Khi $u \geq \rho$, cặp được xem là đúng với độ tin cậy đủ lớn và không bị phạt.

Hình 3 cho thấy vai trò của tham số ρ . Tham số này đặt ra một ngưỡng an toàn cho hiệu điểm giữa hai đối tượng. Nếu margin chỉ vừa lớn hơn 0, mô hình tuy xếp đúng nhưng vẫn thiếu chắc chắn; nếu margin vượt qua ρ , mô hình được xem là đã phân biệt cặp đó đủ rõ.

Lỗi margin thực nghiệm được định nghĩa bởi:

$$\hat{R}_\rho(h) = \frac{1}{m} \sum_{i=1}^m \Phi_\rho(y_i(h(x'_i) - h(x_i))). \quad (14)$$

Vì

$$\mathbf{1}[u \leq 0] \leq \Phi_\rho(u),$$

nên lỗi thực nghiệm dạng chỉ thị luôn được chặn trên bởi lỗi margin:

$$\hat{R}(h) \leq \hat{R}_\rho(h).$$

Điều này cho phép ta phân tích ranking thông qua một hàm mất mát mềm hơn và thuận tiện hơn cho việc xây dựng chặn tổng quát hoá.

3.4 Rademacher complexity

Để đo độ phức tạp của lớp giả thuyết $\mathcal{H}$, ta sử dụng Rademacher complexity. Trực giác, Rademacher complexity đo khả năng một lớp hàm có thể khớp với nhiễu ngẫu nhiên.

Một lớp hàm có độ phức tạp cao thường dễ overfit hơn.

Với một mẫu $S = (z_1, \dots, z_m)$, empirical Rademacher complexity của một lớp hàm $\mathcal{G}$ được định nghĩa là:

$$\hat{\mathfrak{R}}_S(\mathcal{G}) = \mathbb{E}_\sigma \left[\sup_{g \in \mathcal{G}} \frac{1}{m} \sum_{i=1}^m \sigma_i g(z_i) \right], \quad (15)$$

trong đó $\sigma_1, \dots, \sigma_m$ là các biến Rademacher độc lập, nhận giá trị $+1$ hoặc -1 với xác suất bằng nhau.

Trong bài toán ranking, mỗi mẫu là một cặp (x_i, x'_i) . Ta ký hiệu D_1 là phân phối biên của phần tử thứ nhất x , và D_2 là phân phối biên của phần tử thứ hai x' . Tương ứng, ta xét hai đại lượng:

$$\mathfrak{R}_m^{D_1}(\mathcal{H}), \quad \mathfrak{R}_m^{D_2}(\mathcal{H}).$$

Hai đại lượng này xuất hiện trong chặn tổng quát hoá vì hàm điểm được đánh giá trên cả hai phần tử của mỗi cặp.

3.5 Định lý chặn margin cho ranking

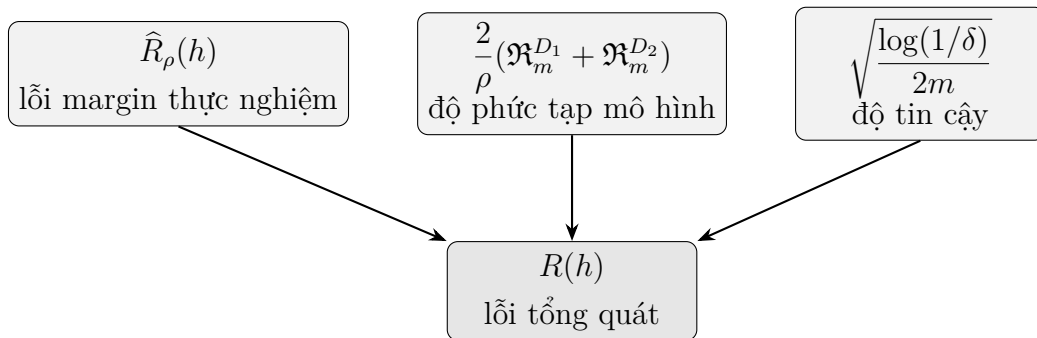
Ta phát biểu kết quả chặn tổng quát hoá chính cho score-based ranking.

Định lý 3.1. Cho $\mathcal{H}$ là một lớp các hàm thực trên $\mathcal{X}$. Cố định $\rho > 0$. Khi đó, với mọi $\delta > 0$, với xác suất ít nhất $1 - \delta$ theo việc chọn mẫu S kích thước m , bất đẳng thức sau đúng đồng thời với mọi $h \in \mathcal{H}$:

$$R(h) \leq \hat{R}_\rho(h) + \frac{2}{\rho} \left(\mathfrak{R}_m^{D_1}(\mathcal{H}) + \mathfrak{R}_m^{D_2}(\mathcal{H}) \right) + \sqrt{\frac{\log(1/\delta)}{2m}}. \quad (16)$$

Ngoài ra, ta cũng có dạng chặn theo empirical Rademacher complexity:

$$R(h) \leq \hat{R}_\rho(h) + \frac{2}{\rho} \left(\hat{\mathfrak{R}}_{S_1}(\mathcal{H}) + \hat{\mathfrak{R}}_{S_2}(\mathcal{H}) \right) + 3\sqrt{\frac{\log(2/\delta)}{2m}}. \quad (17)$$



Hình 4: Ba thành phần trong chặn tổng quát hoá: lỗi trên tập huấn luyện, độ phức tạp của lớp giả thuyết và sai số do lấy mẫu hữu hạn.

Chặn trên gồm ba thành phần. Thành phần thứ nhất là lỗi margin thực nghiệm $\hat{R}_\rho(h)$. Thành phần thứ hai phụ thuộc vào độ phức tạp của lớp giả thuyết $\mathcal{H}$. Thành phần thứ ba phụ thuộc vào kích thước mẫu m và mức tin cậy δ .

Định lý cho thấy để lỗi tổng quát $R(h)$ nhỏ, ta cần đồng thời giảm lỗi margin thực nghiệm và kiểm soát độ phức tạp của lớp giả thuyết. Tham số ρ tạo ra một sự đánh đổi: nếu ρ lớn, yêu cầu margin nghiêm ngặt hơn nên lỗi margin thực nghiệm có thể tăng; nếu ρ nhỏ, hệ số $1/\rho$ trong thành phần độ phức tạp lại tăng.

3.6 Chứng minh Định lý 3.1

Ta chỉ trình bày ý tưởng chính của chứng minh. Đầu tiên, định nghĩa lớp hàm margin theo cặp:

$$\mathcal{H}_\Delta = \left\{ ((x, x'), y) \mapsto y(h(x') - h(x)) \mid h \in \mathcal{H} \right\}.$$

Sau đó, xét lớp hàm sau khi ghép với margin loss:

$$\Phi_\rho \circ \mathcal{H}_\Delta = \left\{ ((x, x'), y) \mapsto \Phi_\rho(y(h(x') - h(x))) \mid h \in \mathcal{H} \right\}.$$

Vì lớp này nhận giá trị trong $[0, 1]$, ta có thể áp dụng chặn tổng quát hoá bằng Rademacher complexity:

$$R(h) \leq \hat{R}_\rho(h) + 2\mathfrak{R}_m(\Phi_\rho \circ \mathcal{H}_\Delta) + \sqrt{\frac{\log(1/\delta)}{2m}}.$$

Do Φ_ρ là hàm $1/\rho$ -Lipschitz, theo tính chất contraction của Rademacher complexity:

$$\mathfrak{R}_m(\Phi_\rho \circ \mathcal{H}_\Delta) \leq \frac{1}{\rho} \mathfrak{R}_m(\mathcal{H}_\Delta).$$

Mặt khác, vì mỗi hàm trong $\mathcal{H}_\Delta$ có dạng hiệu $h(x') - h(x)$, nên độ phức tạp của lớp này được chặn bởi tổng độ phức tạp trên hai phân phối biên:

$$\mathfrak{R}_m(\mathcal{H}_\Delta) \leq \mathfrak{R}_m^{D_1}(\mathcal{H}) + \mathfrak{R}_m^{D_2}(\mathcal{H}).$$

Thay hai chặn trên vào nhau, ta thu được bất đẳng thức trong Định lý 3.1.

3.7 [MỞ RỘNG] Giải thích bước Lipschitz của Φ_ρ

Từ định nghĩa của Φ_ρ , ta thấy hàm này là hằng số trên $(-\infty, 0]$, giảm tuyến tính trên $(0, \rho)$, và lại là hằng số trên $[\rho, +\infty)$. Trên đoạn giữa, độ dốc của hàm là

$$-\frac{1}{\rho}.$$

Do đó, với mọi $u, v \in \mathbb{R}$, ta có:

$$|\Phi_\rho(u) - \Phi_\rho(v)| \leq \frac{1}{\rho}|u - v|.$$

Vì vậy, Φ_ρ là hàm $1/\rho$ -Lipschitz. Đây là lý do trong bước contraction, Rademacher complexity của lớp sau khi ghép với Φ_ρ được nhân với hệ số $1/\rho$. Trực quan từ Hình 3, độ dốc lớn nhất về trị tuyệt đối của đồ thị chính là $1/\rho$.

3.8 Hệ quả cho lớp hàm kernel

Một hệ quả quan trọng của Định lý 3.1 là chặn tổng quát hoá cho lớp hàm kernel. Giả sử $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ là một kernel xác định dương, $\Phi : \mathcal{X} \rightarrow \mathcal{F}$ là ánh xạ đặc trưng tương ứng, và xét lớp giả thuyết:

$$\mathcal{H} = \{x \mapsto w \cdot \Phi(x) \mid \|w\|_{\mathcal{F}} \leq \Lambda\}.$$

Đặt

$$r = \sup_{x \in \mathcal{X}} K(x, x).$$

Khi đó, với xác suất ít nhất $1 - \delta$, với mọi $h \in \mathcal{H}$, ta có:

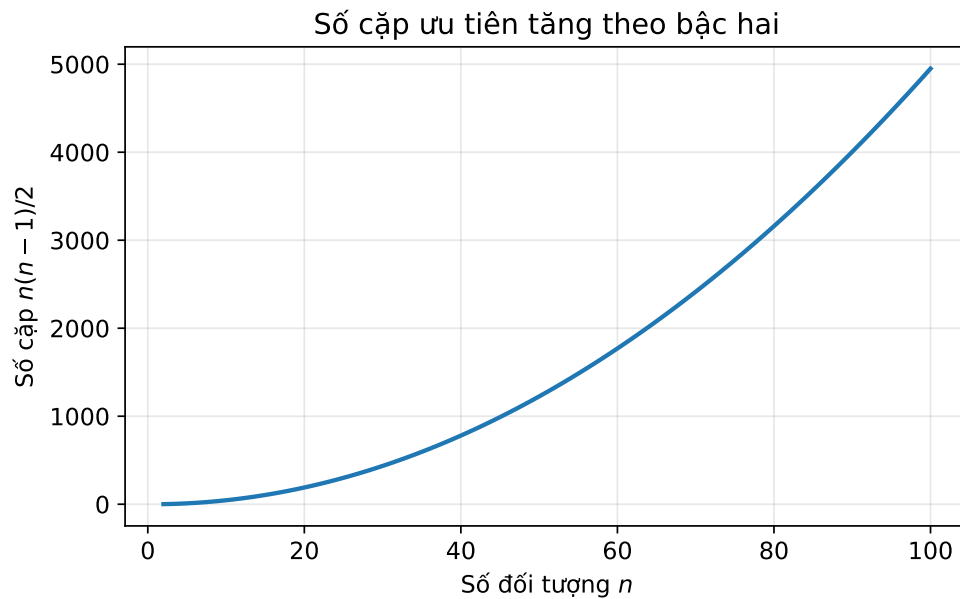
$$R(h) \leq \hat{R}_\rho(h) + 4\sqrt{\frac{r^2\Lambda^2}{\rho^2m}} + \sqrt{\frac{\log(1/\delta)}{2m}}. \quad (18)$$

Chặn này đáng chú ý vì nó không phụ thuộc trực tiếp vào số chiều của không gian đặc trưng $\mathcal{F}$. Thay vào đó, nó phụ thuộc vào margin ρ , kích thước mẫu m , cận r của kernel và chuẩn của vector trọng số thông qua Λ .

3.9 [MỞ RỘNG] Phân tích độ phức tạp của score-based ranking

Ranking theo từng cặp có thể tốn kém vì số lượng cặp tăng nhanh theo số đối tượng. Nếu có n đối tượng và xét tất cả các cặp không lặp, số lượng cặp là:

$$|\mathcal{P}| = \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2). \quad (19)$$



Hình 5: Số cặp ưu tiên tăng theo bậc hai khi số đối tượng n tăng. Với $n = 100$, nếu xét toàn bộ các cặp không lặp thì có $\binom{100}{2} = 4950$ cặp.

Hình 5 minh họa vì sao pairwise ranking có thể nhanh chóng trở nên tốn kém khi n lớn. Chỉ cần tăng số đối tượng, số cặp cần xét tăng nhanh hơn nhiều so với n . Đây là lý do các hệ thống ranking thực tế thường không huấn luyện trên toàn bộ mọi cặp có thể, mà dùng lấy mẫu cặp, mini-batch hoặc các kỹ thuật tối ưu chuyên biệt.

Với mô hình tuyến tính $h_w(x) = w^\top x$, chi phí tính điểm cho n đối tượng là $O(nd)$, và chi phí sắp xếp để tạo ranking là $O(n \log n)$. Do đó, chi phí suy luận là:

$$O(nd + n \log n).$$

Nếu huấn luyện trực tiếp trên toàn bộ các cặp, mỗi cặp cần tính hiệu điểm

$$h_w(x_j) - h_w(x_i) = w^\top (x_j - x_i),$$

có chi phí $O(d)$. Vì vậy, chi phí cho một epoch huấn luyện là:

$$O(n^2 d).$$

Bước xử lý	Chi phí	Ý nghĩa
Tính điểm cho n đối tượng	$O(nd)$	Tính $h_w(x) = w^\top x$
Sắp xếp để tạo ranking	$O(n \log n)$	Xếp đối tượng theo điểm giảm dần
Tạo toàn bộ cặp	$O(n^2)$	Số cặp tăng theo $\binom{n}{2}$
Một epoch pairwise training	$O(n^2 d)$	Tính loss/gradient trên mọi cặp

Bảng 1: Tóm tắt độ phức tạp của score-based pairwise ranking với mô hình tuyến tính.

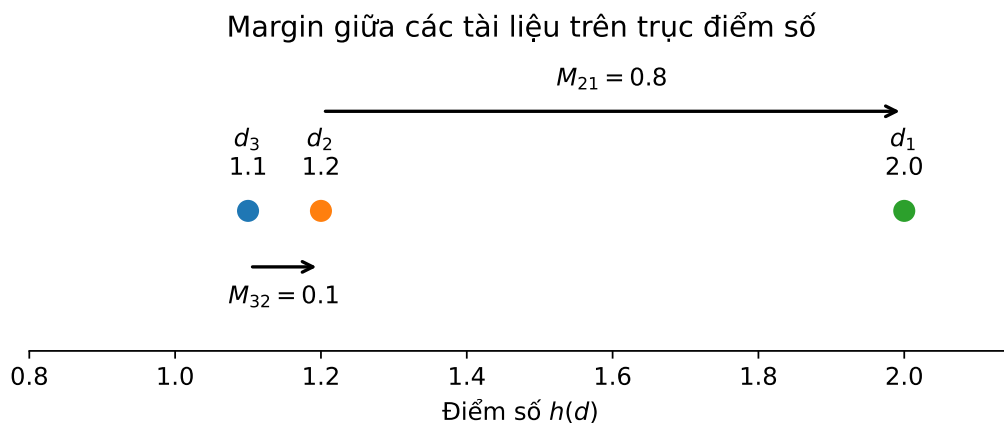
3.10 Ví dụ minh họa về margin

Xét ba tài liệu d_1, d_2, d_3 với thứ tự đúng:

$$d_1 \succ d_2 \succ d_3.$$

Giả sử mô hình dự đoán điểm:

$$h(d_1) = 2.0, \quad h(d_2) = 1.2, \quad h(d_3) = 1.1.$$



Hình 6: Minh họa margin giữa các tài liệu trên trục điểm số. Cặp (d_2, d_1) có margin lớn hơn, trong khi cặp (d_3, d_2) có margin nhỏ nên kém chắc chắn hơn dù vẫn được xếp đúng.

Với cặp (d_2, d_1) , margin là:

$$M_{21}(h) = h(d_1) - h(d_2) = 0.8.$$

Với cặp (d_3, d_2) , margin là:

$$M_{32}(h) = h(d_2) - h(d_3) = 0.1.$$

Cả hai cặp đều được xếp đúng vì margin dương. Tuy nhiên, Hình 6 cho thấy hai trường

hợp này không có cùng mức độ chắc chắn. Cặp (d_2, d_1) có khoảng cách điểm lớn, còn cặp (d_3, d_2) chỉ cách nhau 0.1, nên dễ bị đảo thứ tự nếu điểm dự đoán có nhiễu nhỏ.

Nếu chọn $\rho = 0.5$, cặp (d_3, d_2) vẫn bị phạt bởi margin loss vì margin chỉ bằng 0.1, nhỏ hơn ρ :

$$\Phi_{0.5}(0.1) = 1 - \frac{0.1}{0.5} = 0.8.$$

Ví dụ này cho thấy margin loss không chỉ quan tâm đến việc xếp đúng hay sai, mà còn quan tâm đến mức độ chắc chắn của thứ tự được dự đoán.

3.11 Tóm tắt

Phần này đã trình bày thiết lập score-based ranking, trong đó mô hình học một hàm điểm $h : \mathcal{X} \rightarrow \mathbb{R}$ và dùng hàm này để sinh thứ tự xếp hạng. Ta đã định nghĩa lỗi tổng quát, lỗi thực nghiệm, margin theo cặp và lỗi margin thực nghiệm. Kết quả chính là chặn tổng quát hoá dựa trên Rademacher complexity, cho thấy lỗi ranking có thể được kiểm soát thông qua lỗi margin thực nghiệm và độ phức tạp của lớp giả thuyết.

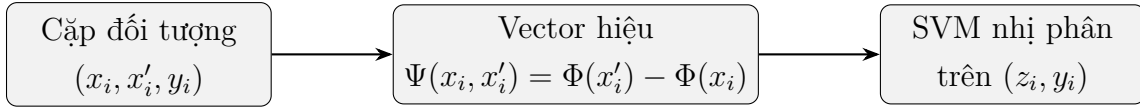
Các hình minh hoạ trong phần này giúp làm rõ ba ý chính. Thứ nhất, Φ_ρ là một surrogate loss mềm hơn lỗi chỉ thị và phạt cả các cặp đúng nhưng margin nhỏ. Thứ hai, margin có thể được hiểu trực quan là khoảng cách điểm giữa hai đối tượng trên trục score. Thứ ba, số lượng cặp trong pairwise ranking tăng theo $O(n^2)$, khiến việc huấn luyện trực tiếp trên toàn bộ cặp trở nên tốn kém với dữ liệu lớn.

Các kết quả trên cũng giải thích vì sao các thuật toán ranking dựa trên margin thường tìm một hàm điểm vừa có lỗi margin nhỏ, vừa có độ phức tạp được kiểm soát. Đây là cơ sở tự nhiên dẫn đến các phương pháp như Ranking SVM ở phần tiếp theo.

4 Ranking với Support Vector Machines

Phần trước đã trình bày chặn tổng quát hoá dựa trên margin cho score-based ranking. Chặn này gợi ý một nguyên lý thiết kế thuật toán tự nhiên: tìm một hàm điểm có lỗi margin thực nghiệm nhỏ, đồng thời kiểm soát độ phức tạp của lớp giả thuyết. Trong phần này, ta trình bày thuật toán ranking dựa trên Support Vector Machines, thường được gọi là *SVM-ranking* hoặc *Ranking SVM*.

Ý tưởng chính của SVM-ranking là biến bài toán ranking theo từng cặp thành một bài toán phân loại nhị phân trên các vector hiệu. Thay vì học trực tiếp từ từng điểm x , thuật toán học từ các cặp (x_i, x'_i) bằng cách xét hiệu đặc trưng giữa hai đối tượng. Nhờ phép biến đổi này, các công cụ quen thuộc của SVM như margin, slack variable, regularization và kernel trick có thể được áp dụng trực tiếp cho bài toán ranking.



Hình 7: Ý tưởng chính của SVM-ranking: biến mỗi cặp ưu tiên thành một mẫu phân loại trên vector hiệu.

4.1 Từ chặn margin đến bài toán tối ưu

Trong score-based ranking, ta xét lớp giả thuyết kernel:

$$\mathcal{H} = \{x \mapsto w \cdot \Phi(x) \mid \|w\|_{\mathcal{F}} \leq \Lambda\},$$

trong đó $\Phi : \mathcal{X} \rightarrow \mathcal{F}$ là ánh xạ đặc trưng tương ứng với kernel K , còn $w \in \mathcal{F}$ là vector tham số.

Với một cặp huấn luyện (x_i, x'_i, y_i) , margin ranking là:

$$y_i (h_w(x'_i) - h_w(x_i)) = y_i w \cdot (\Phi(x'_i) - \Phi(x_i)).$$

Do đó, điều kiện một cặp được xếp đúng với margin ít nhất 1 là:

$$y_i w \cdot (\Phi(x'_i) - \Phi(x_i)) \geq 1. \quad (20)$$

Trong thực tế, dữ liệu ranking thường không thể được xếp đúng hoàn toàn với margin 1. Vì vậy, ta đưa vào các biến slack $\xi_i \geq 0$, cho phép một số cặp vi phạm điều kiện margin:

$$y_i w \cdot (\Phi(x'_i) - \Phi(x_i)) \geq 1 - \xi_i, \quad i = 1, \dots, m. \quad (21)$$

Biến ξ_i đo mức độ vi phạm margin của cặp thứ i . Nếu $\xi_i = 0$, cặp đó đạt margin ít nhất 1; nếu $0 < \xi_i < 1$, cặp vẫn có thể đúng chiều nhưng không đạt đủ margin; nếu $\xi_i \geq 1$, cặp có thể bị xếp sai hoặc nằm quá gần biên quyết định.

4.2 Bài toán tối ưu primal của SVM-ranking

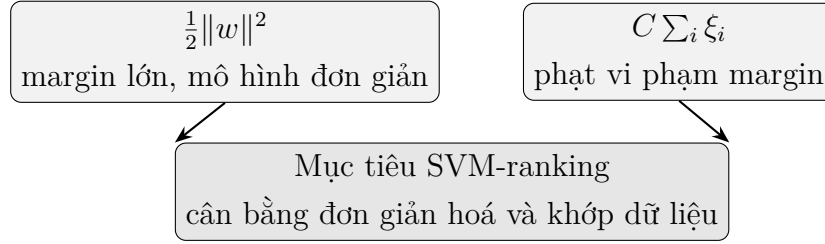
Từ các điều kiện trên, bài toán tối ưu primal của SVM-ranking được viết dưới dạng:

$$\min_{w, \xi} \quad \frac{1}{2} \|w\|_{\mathcal{F}}^2 + C \sum_{i=1}^m \xi_i, \quad (22)$$

với các ràng buộc:

$$y_i w \cdot (\Phi(x'_i) - \Phi(x_i)) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, m.$$

Trong hàm mục tiêu (22), thành phần $\frac{1}{2}\|w\|_{\mathcal{F}}^2$ kiểm soát độ phức tạp của mô hình, còn $C \sum_i \xi_i$ đo tổng mức độ vi phạm margin trên tập huấn luyện. Tham số $C > 0$ điều chỉnh sự đánh đổi giữa margin lớn và lỗi huấn luyện nhỏ.



Hình 8: Hai thành phần trong hàm mục tiêu của SVM-ranking: chuẩn của w kiểm soát độ phức tạp, còn các biến slack đo mức vi phạm margin.

Nếu C lớn, thuật toán ưu tiên giảm lỗi huấn luyện. Nếu C nhỏ, thuật toán ưu tiên nghiệm có chuẩn nhỏ hơn, tức là mô hình đơn giản hơn, nhưng có thể chấp nhận nhiều vi phạm margin hơn.

4.3 Liên hệ với pairwise hinge loss

Bài toán tối ưu trên có thể được viết dưới dạng tối thiểu hoá regularized pairwise hinge loss. Với một cặp (x_i, x'_i, y_i) , pairwise hinge loss là:

$$\ell_i(w) = \max(0, 1 - y_i w \cdot (\Phi(x'_i) - \Phi(x_i))). \quad (23)$$

Khi đó, bài toán SVM-ranking tương đương với:

$$\min_w \frac{1}{2}\|w\|_{\mathcal{F}}^2 + C \sum_{i=1}^m \max(0, 1 - y_i w \cdot (\Phi(x'_i) - \Phi(x_i))). \quad (24)$$

Công thức (24) cho thấy SVM-ranking không tối ưu trực tiếp lỗi pairwise misranking dạng chỉ thị, mà tối ưu một hàm mất mát thay thế là pairwise hinge loss. Hàm mất mát này phạt các cặp bị xếp sai và cả các cặp đúng nhưng chưa đạt margin 1.

4.4 Ranking như một bài toán SVM trên vector hiệu

Điểm quan trọng của SVM-ranking là bài toán ranking có thể được xem như một bài toán SVM thông thường trên không gian các vector hiệu. Định nghĩa ánh xạ:

$$\Psi(x, x') = \Phi(x') - \Phi(x). \quad (25)$$

Khi đó, điều kiện margin trở thành:

$$y_i w \cdot \Psi(x_i, x'_i) \geq 1 - \xi_i.$$

Đây chính là dạng ràng buộc quen thuộc của soft-margin SVM trong classification, với mỗi mẫu huấn luyện mới là:

$$(\Psi(x_i, x'_i), y_i).$$

Như vậy, SVM-ranking không phải là một thuật toán hoàn toàn tách biệt với SVM. Nó là một trường hợp đặc biệt của SVM, trong đó đặc trưng của một mẫu là hiệu đặc trưng giữa hai đối tượng cần so sánh.

Thành phần	SVM phân loại	SVM-ranking
Mẫu huấn luyện	(x_i, y_i)	(x_i, x'_i, y_i)
Đặc trưng đầu vào	x_i	$\Psi(x_i, x'_i) = \Phi(x'_i) - \Phi(x_i)$
Điều kiện margin	$y_i w^\top x_i \geq 1 - \xi_i$	$y_i w^\top \Psi(x_i, x'_i) \geq 1 - \xi_i$
Mục tiêu	phân loại đúng từng điểm	xếp đúng từng cặp ưu tiên

Bảng 2: SVM-ranking có cùng cấu trúc với SVM phân loại, nhưng dữ liệu đầu vào là các vector hiệu sinh từ từng cặp đối tượng.

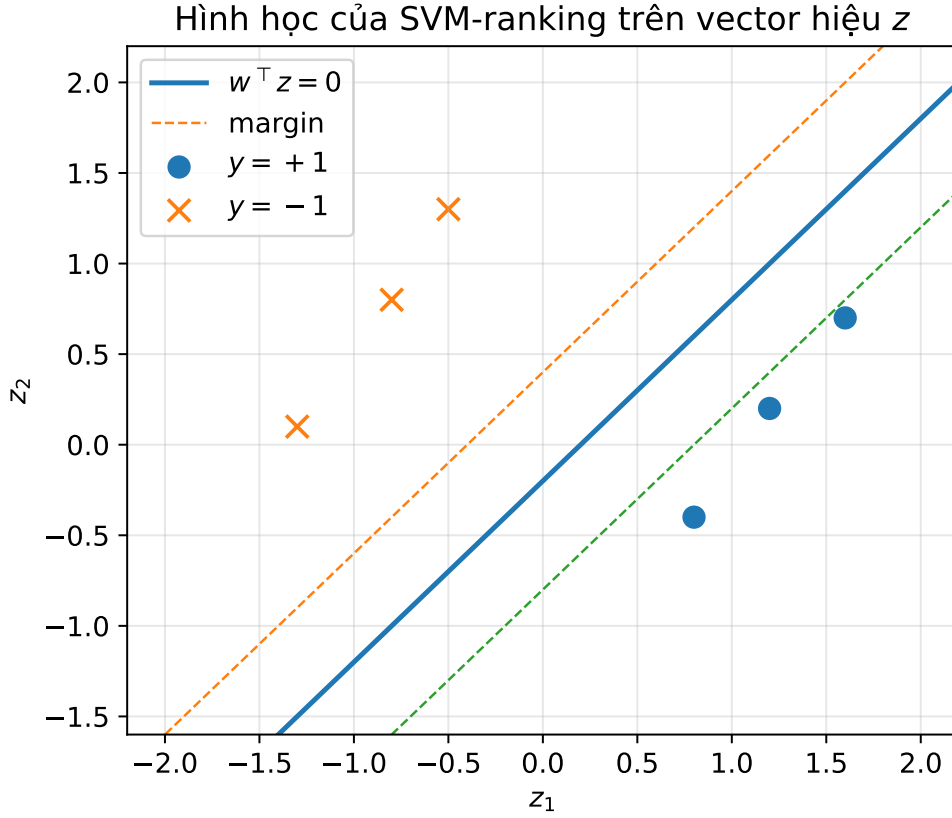
4.5 Hình học của SVM-ranking

Sau phép biến đổi $\Psi(x, x') = \Phi(x') - \Phi(x)$, mỗi cặp đối tượng trở thành một điểm trong không gian vector hiệu. Trong không gian này, SVM-ranking tìm một siêu phẳng phân tách các vector hiệu có nhãn $+1$ và -1 , tương tự SVM phân loại nhị phân.

Siêu phẳng $w^\top z = 0$, với $z = \Psi(x, x')$, quyết định xem mô hình xếp x' cao hơn x hay ngược lại. Nếu

$$w^\top \Psi(x, x') > 0,$$

thì mô hình có xu hướng xếp x' cao hơn x . Ngược lại, nếu giá trị này âm, mô hình xếp x cao hơn x' .



Hình 9: Hình học của SVM-ranking trong không gian vector hiệu $z = \Psi(x, x')$. Đường $w^T z = 0$ đóng vai trò siêu phẳng phân tách, còn hai đường biên margin tương ứng với $w^T z = 1$ và $w^T z = -1$. Các điểm có nhãn $y = +1$ và $y = -1$ biểu diễn các cặp ưu tiên sau khi được biến đổi thành vector hiệu.

Hình 9 minh họa trực quan cách SVM-ranking hoạt động. Sau khi chuyển cặp đối tượng thành vector hiệu z , bài toán ranking trở thành bài toán tìm siêu phẳng phân tách hai nhóm vector hiệu. Các điểm nằm đúng phía và cách xa siêu phẳng ít nhất một khoảng margin sẽ không bị phạt; các điểm nằm trong vùng margin hoặc sai phía sẽ tạo ra slack và đóng góp vào pairwise hinge loss.

Góc nhìn hình học này cho thấy SVM-ranking thực chất là việc học một siêu phẳng phân tách trong không gian các quan hệ so sánh.

4.6 Dạng dual và kernel cho ranking

Vì SVM-ranking có thể được xem là SVM trên vector hiệu, ta có thể sử dụng kernel trick. Kernel trên các cặp được xác định thông qua tích vô hướng:

$$K'_{ij} = \Psi(x_i, x'_i) \cdot \Psi(x_j, x'_j).$$

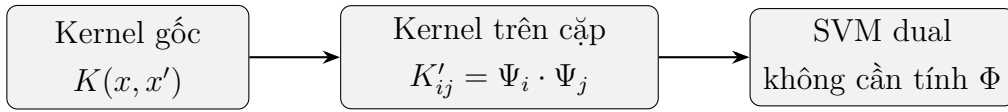
Thay định nghĩa $\Psi(x, x') = \Phi(x') - \Phi(x)$, ta có:

$$\begin{aligned} K'_{ij} &= (\Phi(x'_i) - \Phi(x_i)) \cdot (\Phi(x'_j) - \Phi(x_j)) \\ &= K(x'_i, x'_j) - K(x'_i, x_j) - K(x_i, x'_j) + K(x_i, x_j). \end{aligned} \quad (26)$$

Tương đương, công thức trên có thể viết là:

$$K'_{ij} = K(x_i, x_j) + K(x'_i, x'_j) - K(x'_i, x_j) - K(x_i, x'_j).$$

Công thức (26) cho thấy ta không cần tính trực tiếp ánh xạ Φ . Chỉ cần có kernel K giữa các đối tượng ban đầu, ta có thể xây dựng kernel K' giữa các cặp đối tượng.



Hình 10: Kernel trick trong SVM-ranking: từ kernel giữa các đối tượng ban đầu, ta xây dựng kernel giữa các cặp thông qua vector hiệu.

4.7 Chứng minh liên hệ giữa SVM-ranking và SVM chuẩn

Ta trình bày ngắn gọn liên hệ giữa SVM-ranking và SVM chuẩn. Với mỗi cặp (x_i, x'_i) , định nghĩa mẫu mới:

$$z_i = \Psi(x_i, x'_i) = \Phi(x'_i) - \Phi(x_i).$$

Khi đó, ràng buộc của SVM-ranking:

$$y_i w \cdot (\Phi(x'_i) - \Phi(x_i)) \geq 1 - \xi_i$$

trở thành:

$$y_i w \cdot z_i \geq 1 - \xi_i.$$

Đây chính là ràng buộc của soft-margin SVM cho bài toán phân loại nhị phân trên tập mẫu (z_i, y_i) . Hàm mục tiêu cũng giữ nguyên dạng:

$$\min_{w, \xi} \quad \frac{1}{2} \|w\|_{\mathcal{F}}^2 + C \sum_{i=1}^m \xi_i.$$

Do đó, bài toán SVM-ranking tương đương với soft-margin SVM chuẩn sau khi biến đổi mỗi cặp đối tượng thành một vector hiệu. Điều này giải thích vì sao các tính chất của SVM, như kernel trick và margin maximization, có thể được áp dụng trực tiếp cho ranking.

4.8 Ví dụ minh họa

Xét ba tài liệu d_1, d_2, d_3 với thứ tự đúng:

$$d_1 \succ d_2 \succ d_3.$$

Giả sử mỗi tài liệu được biểu diễn bởi vector hai chiều:

$$x_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Với cặp (d_2, d_1) , vì d_1 được ưu tiên hơn d_2 , nhãn là $y_{21} = +1$. Vector hiệu tương ứng là:

$$x_1 - x_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Nếu mô hình có trọng số

$$w = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

thì margin của cặp này là:

$$y_{21}w^\top(x_1 - x_2) = 1.$$

Cặp này đạt đúng margin 1. Ví dụ này cho thấy SVM-ranking học một vector w sao cho hiệu đặc trưng giữa đối tượng được ưu tiên và đối tượng kém ưu tiên có tích vô hướng dương và đủ lớn với w .

4.9 [MỞ RỘNG] Phân tích độ phức tạp của SVM-ranking

SVM-ranking làm việc trên các cặp đối tượng, vì vậy chi phí tính toán phụ thuộc mạnh vào số lượng cặp huấn luyện. Nếu ban đầu có n đối tượng và tạo tất cả các cặp có thể, số lượng cặp là:

$$|\mathcal{P}| = \frac{n(n-1)}{2} = O(n^2). \quad (27)$$

Nếu mỗi đối tượng có d đặc trưng, việc tính vector hiệu $x'_i - x_i$ cho một cặp có chi phí $O(d)$. Do đó, việc tạo toàn bộ dữ liệu pairwise có chi phí:

$$O(n^2d).$$

Nếu sử dụng kernel, ta cần xây dựng ma trận kernel trên các cặp. Với m cặp huấn luyện, ma trận kernel K' có kích thước $m \times m$, nên chi phí lưu trữ là:

$$O(m^2).$$

Nếu $m = O(n^2)$, chi phí bộ nhớ trong trường hợp dùng kernel có thể lên tới $O(n^4)$. Vì vậy, SVM-ranking có thể trở nên rất tốn kém nếu áp dụng trực tiếp cho dữ liệu lớn với toàn bộ các cặp. Trong thực tế, người ta thường dùng lấy mẫu cặp, tối ưu tuyến tính hoặc các thuật toán chuyên biệt cho ranking quy mô lớn.

Thao tác	Chi phí	Ghi chú
Tạo toàn bộ cặp	$O(n^2)$	Có $\binom{n}{2}$ cặp
Tạo vector hiệu	$O(n^2d)$	Mỗi cặp cần hiệu hai vectơ
Huấn luyện tuyến tính trên cặp	phụ thuộc solver, thường theo m và d	$m = O(n^2)$ nếu dùng mọi cặp
Lưu kernel trên cặp	$O(m^2)$	Có thể thành $O(n^4)$ nếu $m = O(n^2)$

Bảng 3: Tóm tắt chi phí chính của SVM-ranking khi làm việc trên dữ liệu pairwise.

4.10 [MỞ RỘNG] So sánh với SVM phân loại nhị phân

SVM phân loại nhị phân học từ các mẫu riêng lẻ (x_i, y_i) , với điều kiện margin:

$$y_i w^\top x_i \geq 1 - \xi_i.$$

Trong khi đó, SVM-ranking học từ các cặp (x_i, x'_i, y_i) , với điều kiện:

$$y_i w^\top (x'_i - x_i) \geq 1 - \xi_i.$$

Hai công thức có cùng cấu trúc. Điểm khác biệt nằm ở biểu diễn đầu vào: SVM phân loại dùng trực tiếp x_i , còn SVM-ranking dùng vector hiệu $x'_i - x_i$. Vì vậy, có thể xem SVM-ranking là SVM phân loại nhị phân trên không gian các quan hệ so sánh.

Sự khác biệt này cũng phản ánh khác biệt về mục tiêu. SVM phân loại cố gắng dự đoán đúng nhãn của từng điểm, còn SVM-ranking cố gắng dự đoán đúng thứ tự tương đối giữa hai điểm.

4.11 Hạn chế của SVM-ranking

Mặc dù SVM-ranking có nền tảng lý thuyết rõ ràng và tận dụng được các công cụ của SVM, phương pháp này có một số hạn chế.

Thứ nhất, số lượng cặp huấn luyện có thể tăng bậc hai theo số đối tượng. Nếu một truy vấn có nhiều tài liệu, việc tạo toàn bộ các cặp tài liệu có thể gây chi phí lớn.

Thứ hai, trong score-based ranking, mô hình học một hàm điểm toàn cục. Hàm điểm này luôn sinh ra một thứ tự bắc cầu. Do đó, nếu quan hệ ưu tiên thật không bắc cầu, không có hàm điểm nào có thể biểu diễn hoàn hảo toàn bộ quan hệ ưu tiên.

Thứ ba, SVM-ranking tối ưu pairwise margin, trong khi nhiều ứng dụng thực tế

quan tâm nhiều hơn đến các vị trí đầu danh sách, chẳng hạn top- k kết quả tìm kiếm. Vì vậy, các độ đo như NDCG, MAP hoặc precision@ k có thể phù hợp hơn trong một số bối cảnh.

4.12 Tóm tắt

Phần này đã trình bày thuật toán ranking dựa trên SVM. Từ chặn tổng quát hoá theo margin, ta xây dựng bài toán tối ưu có dạng soft-margin SVM, trong đó mỗi mẫu là một cặp đối tượng và đặc trưng của mẫu là hiệu giữa hai vector đặc trưng. Ánh xạ

$$\Psi(x, x') = \Phi(x') - \Phi(x)$$

cho thấy SVM-ranking thực chất là một trường hợp đặc biệt của SVM chuẩn trên không gian pairwise.

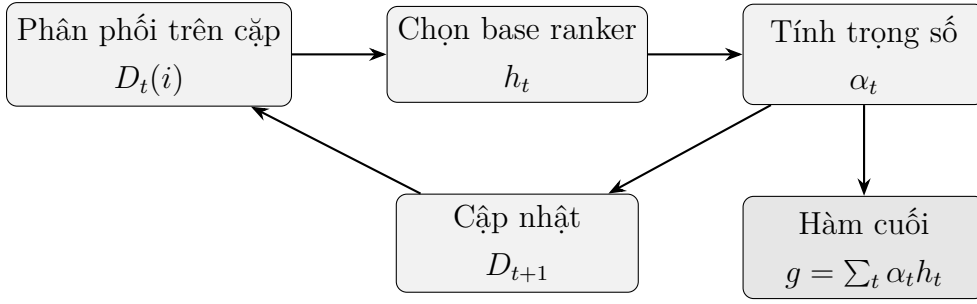
Hình 9 giúp trực quan hoá ý tưởng này: sau khi biến đổi mỗi cặp thành vector hiệu z , SVM-ranking tìm một siêu phẳng $w^\top z = 0$ để phân tách các cặp được gán nhãn $+1$ và -1 . Các đường margin cho biết vùng an toàn mà mô hình mong muốn các vector hiệu nằm ngoài.

Kết quả này tạo cầu nối giữa lý thuyết ranking và các thuật toán tối ưu margin. Trong phần tiếp theo, ta sẽ nghiên cứu một hướng tiếp cận khác trong score-based ranking: RankBoost.

5 RankBoost

Trong các phần trước, ta đã trình bày score-based ranking và thuật toán ranking dựa trên SVM. Một hướng tiếp cận khác trong score-based ranking là sử dụng boosting. Phần này trình bày RankBoost, một thuật toán boosting được thiết kế cho bài toán ranking theo từng cặp.

Tương tự AdaBoost trong phân loại nhị phân, RankBoost kết hợp nhiều mô hình yếu, gọi là *base rankers*, để tạo thành một hàm ranking mạnh hơn. Khác với AdaBoost, RankBoost duy trì phân phối trọng số trên các cặp đối tượng. Các cặp bị xếp sai ở những vòng trước sẽ được tăng trọng số, khiến thuật toán tập trung nhiều hơn vào các cặp khó ở những vòng sau.



Hình 11: Vòng lặp chính của RankBoost. Ở mỗi vòng, thuật toán chọn một base ranker tốt theo phân phối hiện tại, gán trọng số cho nó, rồi cập nhật phân phối để tập trung hơn vào các cặp khó.

5.1 Thiết lập của RankBoost

Giả sử tập huấn luyện gồm m cặp có nhãn:

$$S = ((x_1, x'_1, y_1), \dots, (x_m, x'_m, y_m)), \quad y_i \in \{-1, 0, +1\}.$$

Theo quy ước, $y_i = +1$ nghĩa là x'_i được ưu tiên hơn x_i , $y_i = -1$ nghĩa là x_i được ưu tiên hơn x'_i , và $y_i = 0$ nghĩa là hai đối tượng ngang nhau hoặc không có thông tin ưu tiên rõ ràng. Để đơn giản, ta giả sử các cặp có nhãn 0 đã được loại bỏ, tức là:

$$y_i \neq 0, \quad i = 1, \dots, m.$$

RankBoost sử dụng một tập các base rankers:

$$\mathcal{H} = \{h : \mathcal{X} \rightarrow \{0, 1\}\}.$$

Mỗi base ranker $h_t \in \mathcal{H}$ gán cho mỗi đối tượng một giá trị trong $\{0, 1\}$. Sau T vòng, RankBoost trả về hàm điểm:

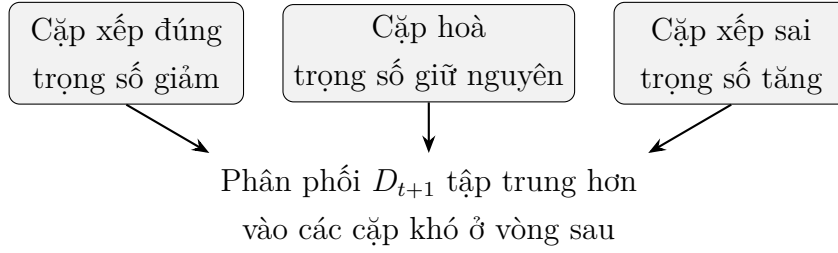
$$g(x) = \sum_{t=1}^T \alpha_t h_t(x), \quad (28)$$

trong đó $\alpha_t \geq 0$ là trọng số của base ranker h_t . Ranking cuối cùng được sinh ra bằng cách sắp xếp các đối tượng theo giá trị $g(x)$.

5.2 Ý tưởng lặp của RankBoost

RankBoost hoạt động theo từng vòng. Tại vòng t , thuật toán duy trì một phân phối trọng số D_t trên các cặp huấn luyện. Sau đó, thuật toán chọn một base ranker h_t tốt đối với phân phối hiện tại, tính trọng số α_t , rồi cập nhật phân phối thành D_{t+1} .

Trực giác của bước cập nhật là: các cặp được xếp đúng sẽ giảm trọng số tương đối, còn các cặp bị xếp sai sẽ tăng trọng số tương đối. Vì vậy, ở các vòng sau, thuật toán tập trung nhiều hơn vào các cặp khó.



Hình 12: Trực giác cập nhật trọng số trong RankBoost: cặp xếp sai được tăng trọng số để thuật toán chú ý hơn ở các vòng tiếp theo.

5.3 Phân phối trọng số trên các cặp

RankBoost duy trì một phân phối D_t trên các cặp huấn luyện:

$$D_t(i) \geq 0, \quad \sum_{i=1}^m D_t(i) = 1.$$

Ban đầu, mọi cặp được gán trọng số bằng nhau:

$$D_1(i) = \frac{1}{m}, \quad i = 1, \dots, m.$$

Tại mỗi vòng t , với một cặp (x_i, x'_i, y_i) , base ranker h_t xếp đúng nếu:

$$y_i(h_t(x'_i) - h_t(x_i)) > 0.$$

Nó xếp sai nếu biểu thức trên nhỏ hơn 0, và không phân biệt được cặp đó nếu biểu thức bằng 0.

5.4 Các đại lượng ϵ_t^+ , ϵ_t^- , ϵ_t^0

Tại vòng t , với phân phối D_t , ta định nghĩa:

$$\epsilon_t^+ = \sum_{i=1}^m D_t(i) \mathbf{1} [y_i(h_t(x'_i) - h_t(x_i)) = +1], \quad (29)$$

$$\epsilon_t^- = \sum_{i=1}^m D_t(i) \mathbf{1} [y_i(h_t(x'_i) - h_t(x_i)) = -1], \quad (30)$$

và:

$$\epsilon_t^0 = \sum_{i=1}^m D_t(i) \mathbf{1} [y_i(h_t(x'_i) - h_t(x_i)) = 0]. \quad (31)$$

Ba đại lượng này lần lượt là tổng trọng số của các cặp được xếp đúng, xếp sai và không được phân biệt bởi h_t . Ta có:

$$\epsilon_t^+ + \epsilon_t^- + \epsilon_t^0 = 1.$$

ϵ_t^+ : xếp đúng	ϵ_t^- : xếp sai	ϵ_t^0 : hoà
---------------------------	--------------------------	----------------------

$$\epsilon_t^+ + \epsilon_t^- + \epsilon_t^0 = 1$$

Hình 13: Phân phối trọng số D_t được chia thành ba nhóm: tổng trọng số của cặp xếp đúng, cặp xếp sai và cặp hoà.

RankBoost chọn base ranker sao cho $\epsilon_t^- - \epsilon_t^+$ nhỏ nhất, tức là ưu tiên base ranker có trọng số xếp đúng lớn và trọng số xếp sai nhỏ. Edge của base ranker được định nghĩa là:

$$\gamma_t = \frac{\epsilon_t^+ - \epsilon_t^-}{2}. \quad (32)$$

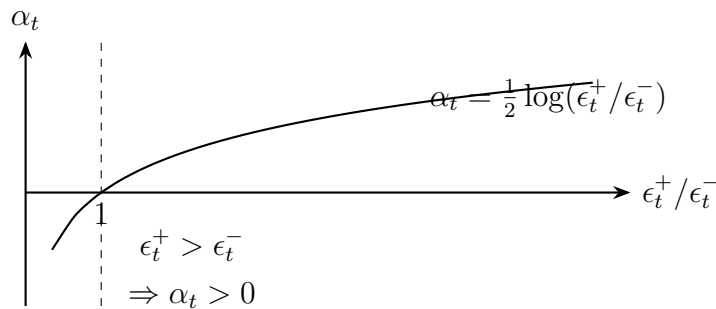
Nếu $\gamma_t > 0$, base ranker h_t xếp đúng nhiều hơn xếp sai theo phân phối hiện tại. Đây là điều kiện yếu giúp boosting cải thiện dần qua các vòng.

5.5 Trọng số của base ranker

Sau khi chọn được h_t , RankBoost gán cho nó trọng số:

$$\alpha_t = \frac{1}{2} \log \frac{\epsilon_t^+}{\epsilon_t^-}. \quad (33)$$

Nếu $\epsilon_t^+ > \epsilon_t^-$, thì $\alpha_t > 0$. Base ranker càng tốt, tức là ϵ_t^+ càng lớn so với ϵ_t^- , thì α_t càng lớn. Vì vậy, các base rankers mạnh hơn sẽ có ảnh hưởng lớn hơn trong hàm điểm cuối cùng g .



Hình 14: Trọng số α_t tăng khi base ranker xếp đúng nhiều hơn xếp sai theo phân phối hiện tại.

5.6 Cập nhật phân phối trọng số

Sau khi chọn h_t và α_t , RankBoost cập nhật phân phối trọng số trên các cặp:

$$D_{t+1}(i) = \frac{D_t(i) \exp \left(-\alpha_t y_i (h_t(x'_i) - h_t(x_i)) \right)}{Z_t}, \quad (34)$$

trong đó Z_t là hệ số chuẩn hoá:

$$Z_t = \sum_{i=1}^m D_t(i) \exp \left(-\alpha_t y_i (h_t(x'_i) - h_t(x_i)) \right). \quad (35)$$

Hệ số Z_t đảm bảo D_{t+1} vẫn là một phân phối. Nếu cặp thứ i được xếp đúng, trọng số của nó được nhân với $e^{-\alpha_t}$. Nếu cặp đó bị xếp sai, trọng số được nhân với e^{α_t} . Nếu cặp bị hoà, hệ số nhân là 1.



Hình 15: Ảnh hưởng của α lên hệ số nhân trọng số trong RankBoost. Khi α tăng, cặp xếp đúng được giảm trọng số theo $e^{-\alpha}$, cặp xếp sai được tăng trọng số theo e^{α} , còn cặp hoà giữ nguyên hệ số 1.

Hình 15 cho thấy rõ vai trò của α_t . Khi base ranker tốt hơn, α_t lớn hơn, làm cho các cặp xếp sai được phóng đại trọng số mạnh hơn ở vòng sau. Ngược lại, các cặp đã được xếp đúng bị giảm trọng số, vì chúng không còn là trọng tâm chính của thuật toán.

Vì $\alpha_t > 0$, các cặp bị xếp sai được tăng trọng số tương đối, còn các cặp được xếp đúng bị giảm trọng số tương đối. Đây là cơ chế giúp RankBoost tập trung vào các cặp khó hơn qua từng vòng.

5.7 Thuật toán RankBoost

Thuật toán RankBoost có thể tóm tắt như sau.

Bước 1. Khởi tạo phân phối đều:

$$D_1(i) = \frac{1}{m}, \quad i = 1, \dots, m.$$

Bước 2. Với mỗi vòng $t = 1, \dots, T$, chọn base ranker:

$$h_t \in \arg \min_{h \in \mathcal{H}} (\epsilon_t^-(h) - \epsilon_t^+(h)).$$

Bước 3. Tính trọng số:

$$\alpha_t = \frac{1}{2} \log \frac{\epsilon_t^+}{\epsilon_t^-}.$$

Bước 4. Cập nhật phân phối theo công thức (34).

Bước 5. Sau T vòng, trả về:

$$g(x) = \sum_{t=1}^T \alpha_t h_t(x).$$

Hàm g sau cùng được dùng để xếp hạng các đối tượng: đối tượng nào có $g(x)$ lớn hơn sẽ được xếp cao hơn.

5.8 Biểu diễn phân phối sau nhiều vòng

Để phân tích RankBoost, ta cần biểu diễn D_{t+1} theo tổ hợp các base rankers đã chọn. Gọi:

$$g_t(x) = \sum_{s=1}^t \alpha_s h_s(x).$$

Khi đó, bằng cách khai triển đệ quy công thức cập nhật, ta có:

$$D_{t+1}(i) = \frac{\exp(-y_i(g_t(x'_i) - g_t(x_i)))}{m \prod_{s=1}^t Z_s}. \quad (36)$$

Công thức này cho thấy trọng số của một cặp sau nhiều vòng phụ thuộc vào margin tích lũy của hàm g_t trên cặp đó. Nếu cặp có margin lớn và đúng chiều, trọng số giảm; nếu cặp bị xếp sai hoặc margin nhỏ, trọng số vẫn lớn.

5.9 Chặn lỗi thực nghiệm của RankBoost

Sau T vòng, RankBoost trả về:

$$g(x) = g_T(x) = \sum_{t=1}^T \alpha_t h_t(x).$$

Lỗi thực nghiệm của g là:

$$\hat{R}(g) = \frac{1}{m} \sum_{i=1}^m \mathbf{1} [y_i (g(x'_i) - g(x_i)) \leq 0]. \quad (37)$$

Ta có chặn lỗi thực nghiệm sau.

Định lý 5.1. Lỗi thực nghiệm của hàm g trả về bởi RankBoost thỏa:

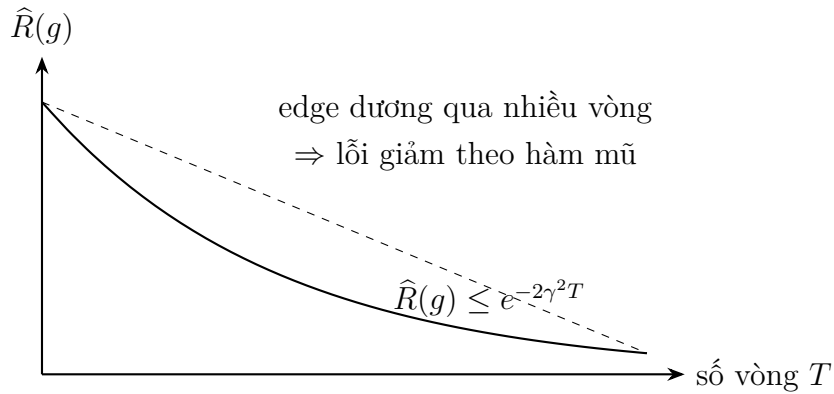
$$\hat{R}(g) \leq \exp \left(-2 \sum_{t=1}^T \left(\frac{\epsilon_t^+ - \epsilon_t^-}{2} \right)^2 \right). \quad (38)$$

Nếu tồn tại $\gamma > 0$ sao cho với mọi $t = 1, \dots, T$,

$$\gamma \leq \frac{\epsilon_t^+ - \epsilon_t^-}{2},$$

thì:

$$\hat{R}(g) \leq \exp(-2\gamma^2 T). \quad (39)$$



Hình 16: Nếu mỗi vòng tìm được base ranker có edge dương, chặn trên của lỗi thực nghiệm giảm theo hàm mũ theo số vòng boosting.

Định lý cho thấy nếu mỗi vòng boosting tìm được một base ranker có edge dương ít nhất γ , lỗi thực nghiệm của RankBoost giảm theo hàm mũ theo số vòng T . Đây là tính chất tương tự AdaBoost trong bài toán phân loại nhị phân.

5.10 Chứng minh Định lý 5.1

Ta chỉ trình bày ý tưởng chính của chứng minh. Sử dụng bất đẳng thức:

$$\mathbf{1}[u \leq 0] \leq \exp(-u), \quad \forall u \in \mathbb{R},$$

với

$$u_i = y_i(g(x'_i) - g(x_i)),$$

ta có:

$$\hat{R}(g) \leq \frac{1}{m} \sum_{i=1}^m \exp(-y_i(g(x'_i) - g(x_i))).$$

Từ công thức (36) với $t = T$, suy ra:

$$\hat{R}(g) \leq \prod_{t=1}^T Z_t.$$

Tiếp theo, chia các cặp thành ba nhóm được xếp đúng, xếp sai và hoà, ta có:

$$Z_t = \epsilon_t^+ e^{-\alpha_t} + \epsilon_t^- e^{\alpha_t} + \epsilon_t^0.$$

Với lựa chọn

$$\alpha_t = \frac{1}{2} \log \frac{\epsilon_t^+}{\epsilon_t^-},$$

ta thu được:

$$Z_t = 2\sqrt{\epsilon_t^+ \epsilon_t^-} + \epsilon_t^0.$$

Sử dụng các bước chặn chuẩn trong phân tích RankBoost, ta có:

$$Z_t \leq \exp\left(-2\left(\frac{\epsilon_t^+ - \epsilon_t^-}{2}\right)^2\right).$$

Thay chặn này vào $\hat{R}(g) \leq \prod_{t=1}^T Z_t$, ta thu được:

$$\hat{R}(g) \leq \exp\left(-2\sum_{t=1}^T \left(\frac{\epsilon_t^+ - \epsilon_t^-}{2}\right)^2\right).$$

Nếu mỗi edge đều không nhỏ hơn γ , thì tổng trong mũ không nhỏ hơn $T\gamma^2$, và do

đó:

$$\hat{R}(g) \leq \exp(-2\gamma^2 T).$$

5.11 [MỞ RỘNG] Vì sao α_t tối thiểu hoá Z_t

Hệ số α_t có thể được giải thích bằng cách xem Z_t như một hàm của α :

$$Z_t(\alpha) = \epsilon_t^+ e^{-\alpha} + \epsilon_t^- e^{\alpha} + \epsilon_t^0.$$

Lấy đạo hàm theo α :

$$\frac{dZ_t(\alpha)}{d\alpha} = -\epsilon_t^+ e^{-\alpha} + \epsilon_t^- e^{\alpha}.$$

Cho đạo hàm bằng 0, ta được:

$$e^{2\alpha} = \frac{\epsilon_t^+}{\epsilon_t^-}.$$

Do đó:

$$\alpha = \frac{1}{2} \log \frac{\epsilon_t^+}{\epsilon_t^-}.$$

Vì vậy, công thức chọn α_t trong RankBoost được chọn để tối thiểu hoá hệ số chuẩn hoá Z_t , mà tích các Z_t lại là một chặn trên của lỗi thực nghiệm.

5.12 RankBoost và coordinate descent

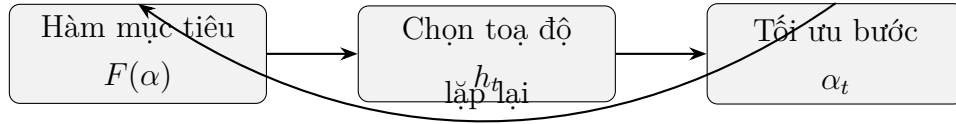
RankBoost cũng có thể được hiểu như việc áp dụng coordinate descent lên một hàm mục tiêu lỗi và khả vi. Xét hàm:

$$F(\alpha) = \sum_{i=1}^m \exp(-y_i [g(x'_i) - g(x_i)]), \quad (40)$$

trong đó:

$$g(x) = \sum_{t=1}^T \alpha_t h_t(x).$$

Hàm $F(\alpha)$ là một chặn lỗi của lỗi pairwise misranking dạng chỉ thị. Ở mỗi vòng, RankBoost chọn base ranker h_t tương ứng với một hướng toạ độ làm giảm nhanh nhất hàm F . Sau đó, hệ số α_t được chọn để tối thiểu hoá F theo hướng đó. Vì vậy, RankBoost có thể được xem là coordinate descent trong không gian các tổ hợp tuyến tính của base rankers.



Hình 17: Góc nhìn coordinate descent: RankBoost chọn một hướng h_t rồi tối ưu hệ số α_t theo hướng đó.

5.13 Ví dụ minh họa

Xét ba tài liệu d_1, d_2, d_3 với thứ tự đúng:

$$d_1 \succ d_2 \succ d_3.$$

Ta xét ba cặp:

$$(d_2, d_1, +1), \quad (d_3, d_1, +1), \quad (d_3, d_2, +1).$$

Ban đầu, mỗi cặp có trọng số $1/3$. Giả sử base ranker h_1 cho điểm:

$$h_1(d_1) = 1, \quad h_1(d_2) = 1, \quad h_1(d_3) = 0.$$

Khi đó, cặp (d_2, d_1) bị hoà vì $h_1(d_1) - h_1(d_2) = 0$, còn hai cặp (d_3, d_1) và (d_3, d_2) được xếp đúng. Vì vậy:

$$\epsilon_1^+ = \frac{2}{3}, \quad \epsilon_1^- = 0, \quad \epsilon_1^0 = \frac{1}{3}.$$

$$1 \quad (d_3, d_2, +1) - 0 = 1 \quad (d_3, d_1, +1) - 1 = 0 \quad (d_2, d_1, +1) \quad h_1(x') - h_1(x)$$

Hình 18: Ví dụ phân loại ba cặp thành nhóm đúng, sai hoặc hoà theo base ranker h_1 .

Ví dụ này cho thấy vai trò của ba nhóm cặp: xếp đúng, xếp sai và hoà điểm. Trong trường hợp $\epsilon_t^- = 0$, công thức $\alpha_t = \frac{1}{2} \log(\epsilon_t^+ / \epsilon_t^-)$ không xác định hữu hạn; thực tế có thể xử lý bằng làm tròn nhỏ hoặc dừng sớm nếu không còn cặp bị xếp sai.

5.14 [MỞ RỘNG] Phân tích độ phức tạp của RankBoost

Giả sử tập huấn luyện có m cặp và thuật toán chạy trong T vòng. Nếu bỏ qua chi phí tìm base ranker, việc tính các đại lượng ϵ_t^+ , ϵ_t^- , ϵ_t^0 cần duyệt qua m cặp, nên có chi phí $O(m)$. Việc cập nhật phân phối D_{t+1} cũng có chi phí $O(m)$.

Do đó, chi phí mỗi vòng là:

$$O(m),$$

và sau T vòng, tổng chi phí là:

$$O(Tm).$$

Nếu m được tạo từ tất cả các cặp của n đối tượng, thì:

$$m = \frac{n(n-1)}{2} = O(n^2).$$

Khi đó, chi phí tổng quát có thể trở thành $O(Tn^2)$, còn bộ nhớ để lưu phân phối trên các cặp là $O(n^2)$. Điều này cho thấy RankBoost có thể gặp khó khăn khi số lượng cặp rất lớn, và là động lực để xét các phiên bản hiệu quả hơn trong bipartite ranking.

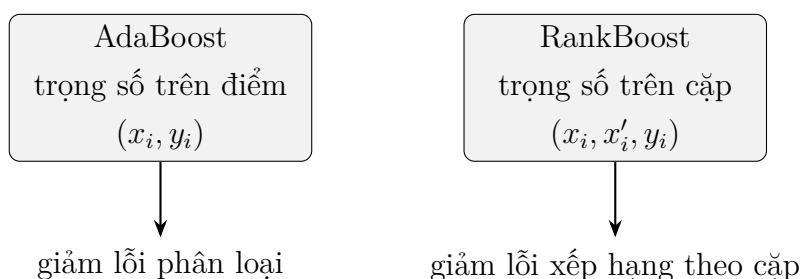
Thao tác	Chi phí	Ghi chú
Tính $\epsilon_t^+, \epsilon_t^-, \epsilon_t^0$	$O(m)$	Duyệt qua toàn bộ cặp
Cập nhật D_{t+1}	$O(m)$	Chuẩn hoá bằng Z_t
Một vòng boosting	$O(m)$	Chưa tính chi phí tìm h_t
T vòng boosting	$O(Tm)$	Nếu $m = O(n^2)$, chi phí là $O(Tn^2)$

Bảng 4: Tóm tắt độ phức tạp của RankBoost trong thiết lập pairwise tổng quát.

5.15 Liên hệ với AdaBoost

RankBoost có quan hệ chặt chẽ với AdaBoost. Cả hai thuật toán đều duy trì một phân phối trọng số, chọn một weak learner tại mỗi vòng, gán trọng số cho weak learner đó và cập nhật phân phối để tập trung vào các mẫu khó.

Điểm khác biệt chính là AdaBoost duy trì trọng số trên từng điểm dữ liệu (x_i, y_i) , còn RankBoost duy trì trọng số trên từng cặp (x_i, x'_i, y_i) . Vì vậy, AdaBoost cố gắng giảm lỗi phân loại từng điểm, còn RankBoost cố gắng giảm lỗi xếp hạng theo từng cặp.



Hình 19: AdaBoost và RankBoost có cùng tinh thần boosting, nhưng khác nhau ở đối tượng được gán trọng số.

5.16 Tóm tắt

RankBoost là một thuật toán boosting cho bài toán ranking theo từng cặp. Thuật toán duy trì phân phối trọng số trên các cặp huấn luyện, chọn base ranker có khả năng xếp đúng các cặp có trọng số lớn, sau đó cập nhật phân phối để tập trung vào các cặp bị xếp sai. Hàm ranking cuối cùng là tổ hợp tuyến tính của các base rankers.

Các hình trong phần này nhấn mạnh ba ý chính. Thứ nhất, RankBoost hoạt động theo vòng lặp chọn base ranker, tính α_t , rồi cập nhật phân phối. Thứ hai, hệ số α_t điều khiển mức tăng hoặc giảm trọng số của các cặp sau mỗi vòng. Thứ ba, nếu mỗi vòng có edge dương, chặn trên của lỗi thực nghiệm giảm theo hàm mũ theo số vòng boosting.

Kết quả lý thuyết quan trọng của RankBoost là lỗi thực nghiệm giảm theo hàm mũ nếu tại mỗi vòng tồn tại một base ranker có edge dương. Ngoài ra, RankBoost có thể được hiểu như coordinate descent trên một hàm mục tiêu lỗi dạng exponential loss. Phân tích này tạo nền tảng cho phần tiếp theo, nơi ta xét trường hợp đặc biệt nhưng rất quan trọng của ranking: bipartite ranking.

6 Bipartite ranking, ROC và AUC

Các phần trước đã trình bày score-based ranking trong thiết lập tổng quát, nơi dữ liệu huấn luyện gồm các cặp đối tượng có nhãn ưu tiên. Trong phần này, ta xét một trường hợp đặc biệt nhưng rất quan trọng của ranking, gọi là *bipartite ranking*. Đây là thiết lập trong đó các đối tượng được chia thành hai nhóm: positive và negative. Mục tiêu của mô hình là xếp các đối tượng positive cao hơn các đối tượng negative.

Bipartite ranking có quan hệ chặt chẽ với binary classification, nhưng mục tiêu của hai bài toán khác nhau. Trong binary classification, ta muốn dự đoán đúng nhãn của từng điểm. Trong bipartite ranking, ta muốn tạo ra một thứ tự sao cho các điểm positive đứng trước các điểm negative. Một độ đo phổ biến cho thiết lập này là AUC, tức diện tích dưới đường cong ROC.

6.1 Thiết lập bipartite ranking

Gọi $\mathcal{X}$ là không gian đầu vào. Trong bipartite ranking, tập dữ liệu được chia thành hai lớp rời nhau:

$$\mathcal{X} = \mathcal{X}_+ \cup \mathcal{X}_-, \quad \mathcal{X}_+ \cap \mathcal{X}_- = \emptyset.$$

Các phần tử trong $\mathcal{X}_+$ được gọi là positive points, còn các phần tử trong $\mathcal{X}_-$ được gọi là negative points. Mục tiêu là học một hàm điểm

$$h : \mathcal{X} \rightarrow \mathbb{R},$$

sao cho với mọi $x^+ \in \mathcal{X}_+$ và $x^- \in \mathcal{X}_-$, ta mong muốn:

$$h(x^+) > h(x^-). \quad (41)$$

Điều kiện trong công thức (41) nói rằng điểm positive nên được xếp cao hơn điểm negative. Đây là mục tiêu cốt lõi của bipartite ranking.

6.2 Lỗi tổng quát trong bipartite ranking

Gọi D_+ là phân phối trên các điểm positive và D_- là phân phối trên các điểm negative. Lỗi tổng quát của một hàm điểm h được định nghĩa là xác suất một điểm negative được xếp cao hơn một điểm positive:

$$R(h) = \Pr_{\substack{x^+ \sim D_+ \\ x^- \sim D_-}} [h(x^+) < h(x^-)]. \quad (42)$$

Công thức này đo trực tiếp lỗi ranking theo từng cặp positive-negative. Nếu $R(h)$ nhỏ, điều đó có nghĩa là phần lớn các điểm positive được mô hình xếp cao hơn các điểm negative.

Trong thực tế, có thể xảy ra trường hợp hoà điểm. Khi đó, một quy ước thường dùng là xem một cặp hoà điểm như nửa đúng, nửa sai:

$$R_{\text{tie}}(h) = \Pr [h(x^+) < h(x^-)] + \frac{1}{2} \Pr [h(x^+) = h(x^-)].$$

6.3 Lỗi thực nghiệm

Giả sử ta có tập positive:

$$S_+ = \{x_1^+, x_2^+, \dots, x_m^+\},$$

và tập negative:

$$S_- = \{x_1^-, x_2^-, \dots, x_n^-\}.$$

Lỗi thực nghiệm của h là tỉ lệ các cặp positive-negative bị xếp sai:

$$\hat{R}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1} [h(x_i^+) < h(x_j^-)]. \quad (43)$$

Nếu xử lý hoà điểm bằng hệ số 1/2, lỗi thực nghiệm có thể viết là:

$$\hat{R}_{\text{tie}}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \left(\mathbf{1} [h(x_i^+) < h(x_j^-)] + \frac{1}{2} \mathbf{1} [h(x_i^+) = h(x_j^-)] \right).$$

Công thức (43) cho thấy bipartite ranking là một bài toán pairwise: thay vì đánh

giá từng điểm riêng lẻ, ta đánh giá tất cả các cặp gồm một positive point và một negative point.

6.4 So sánh binary classification và bipartite ranking

Binary classification và bipartite ranking đều làm việc với dữ liệu có hai lớp, nhưng mục tiêu tối ưu khác nhau. Binary classification dự đoán nhãn cho từng điểm, còn bipartite ranking quan tâm đến thứ tự tương đối giữa positive và negative points.

Một classifier tốt chưa chắc tạo ra ranking tốt. Ví dụ, mô hình có thể phân loại đúng nhiều điểm, nhưng nếu một số negative points vẫn có điểm số cao hơn nhiều positive points, thì AUC có thể thấp dù accuracy cao. Ngược lại, một mô hình ranking tốt cần đảm bảo rằng điểm của positive points thường lớn hơn điểm của negative points.

Do đó, binary classification là bài toán pointwise, còn bipartite ranking là bài toán pairwise. Sự khác biệt này giải thích vì sao các độ đo như accuracy, precision, recall thường dùng cho classification, trong khi AUC và ranking loss phù hợp hơn với bipartite ranking.

6.5 ROC curve

ROC là viết tắt của *Receiver Operating Characteristic*. Với một hàm điểm h , ta có thể biến h thành một classifier nhị phân bằng cách chọn một ngưỡng θ :

$$c_{\theta}(x) = \begin{cases} +1, & \text{nếu } h(x) \geq \theta, \\ -1, & \text{nếu } h(x) < \theta. \end{cases}$$

Khi thay đổi ngưỡng θ , ta thu được nhiều classifier khác nhau. Với mỗi ngưỡng, ta tính hai đại lượng: true positive rate và false positive rate.

True positive rate, hay TPR, là tỉ lệ positive points được dự đoán là positive:

$$\text{TPR}(\theta) = \Pr_{x^+ \sim D_+} [h(x^+) \geq \theta]. \quad (44)$$

False positive rate, hay FPR, là tỉ lệ negative points bị dự đoán nhầm là positive:

$$\text{FPR}(\theta) = \Pr_{x^- \sim D_-} [h(x^-) \geq \theta]. \quad (45)$$

ROC curve là đường cong gồm các điểm

$$(\text{FPR}(\theta), \text{TPR}(\theta))$$

khi ngưỡng θ thay đổi trên toàn bộ miền giá trị điểm số. Một mô hình tốt sẽ đạt TPR cao trong khi giữ FPR thấp, nên đường ROC nằm gần góc trên bên trái. Nếu mô hình

tương đương dự đoán ngẫu nhiên, ROC curve thường nằm gần đường chéo.

6.6 AUC

AUC là diện tích dưới ROC curve. Về mặt trực giác, AUC đo khả năng mô hình xếp một điểm positive cao hơn một điểm negative được chọn ngẫu nhiên. Cụ thể:

$$\text{AUC}(h) = \Pr_{\substack{x^+ \sim D_+ \\ x^- \sim D_-}} [h(x^+) > h(x^-)] + \frac{1}{2} \Pr_{\substack{x^+ \sim D_+ \\ x^- \sim D_-}} [h(x^+) = h(x^-)]. \quad (46)$$

Nếu bỏ qua trường hợp hoà điểm, công thức trên trở thành:

$$\text{AUC}(h) = \Pr [h(x^+) > h(x^-)].$$

Vì vậy, AUC chính là xác suất một cặp positive-negative được xếp đúng. Đây là lý do AUC được xem là một độ đo tự nhiên cho bipartite ranking.

6.7 Quan hệ giữa AUC và ranking loss

Ta chứng minh quan hệ giữa AUC và ranking loss trong trường hợp không có hoà điểm. Từ định nghĩa lỗi tổng quát trong công thức (42), ta có:

$$R(h) = \Pr [h(x^+) < h(x^-)].$$

Mặt khác, khi không có hoà điểm:

$$\text{AUC}(h) = \Pr [h(x^+) > h(x^-)].$$

Với mỗi cặp (x^+, x^-) , do không có hoà điểm nên đúng một trong hai sự kiện sau xảy ra:

$$h(x^+) > h(x^-) \quad \text{hoặc} \quad h(x^+) < h(x^-).$$

Hai sự kiện này rời nhau và bao phủ toàn bộ không gian mẫu. Vì vậy, tổng xác suất của chúng bằng 1:

$$\Pr [h(x^+) > h(x^-)] + \Pr [h(x^+) < h(x^-)] = 1.$$

Thay hai định nghĩa ở trên vào, ta thu được:

$$\text{AUC}(h) = 1 - R(h). \quad (47)$$

Tương tự, trên mẫu hữu hạn, nếu không có hoà điểm thì:

$$\widehat{\text{AUC}}(h) = 1 - \widehat{R}(h).$$

Nếu có hoà điểm, ta dùng quy ước hoà điểm đóng góp $1/2$ vào AUC và $1/2$ vào lỗi. Khi đó quan hệ tương tự vẫn đúng nếu lỗi ranking được định nghĩa có tính phần hoà điểm như trong $\widehat{R}_{\text{tie}}(h)$.

6.8 Cách tính AUC trên mẫu hữu hạn

Với mẫu positive S_+ kích thước m và mẫu negative S_- kích thước n , AUC thực nghiệm được tính bởi:

$$\widehat{\text{AUC}}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \left(\mathbf{1} [h(x_i^+) > h(x_j^-)] + \frac{1}{2} \mathbf{1} [h(x_i^+) = h(x_j^-)] \right). \quad (48)$$

Công thức (48) có diễn giải đơn giản: duyệt qua tất cả các cặp positive-negative, đếm số cặp được xếp đúng, và chia cho tổng số cặp mn . Cặp hoà điểm được tính nửa điểm.

6.9 Ví dụ minh hoạ cách tính AUC

Xét hai điểm positive và ba điểm negative với điểm số:

$$h(x_1^+) = 0.9, \quad h(x_2^+) = 0.6,$$

và:

$$h(x_1^-) = 0.8, \quad h(x_2^-) = 0.4, \quad h(x_3^-) = 0.3.$$

Tổng số cặp positive-negative là:

$$mn = 2 \cdot 3 = 6.$$

Với x_1^+ , ta có $0.9 > 0.8$, $0.9 > 0.4$, và $0.9 > 0.3$, nên điểm này thắng cả ba negative points. Với x_2^+ , ta có $0.6 < 0.8$, nhưng $0.6 > 0.4$ và $0.6 > 0.3$, nên điểm này thắng hai trong ba negative points.

Do đó, tổng số cặp được xếp đúng là $3 + 2 = 5$. AUC thực nghiệm là:

$$\widehat{\text{AUC}}(h) = \frac{5}{6} \approx 0.833.$$

Lỗi bipartite ranking tương ứng là:

$$\widehat{R}(h) = 1 - \widehat{\text{AUC}}(h) = \frac{1}{6}.$$

Ví dụ này cho thấy AUC có thể được hiểu hoàn toàn theo nghĩa pairwise: mô hình càng xếp nhiều cặp positive-negative đúng thì AUC càng cao.

6.10 [MỞ RỘNG] Phân tích độ phức tạp tính AUC

Nếu tính AUC trực tiếp theo công thức (48), ta cần duyệt qua toàn bộ mn cặp positive-negative. Do đó, độ phức tạp thời gian là:

$$O(mn).$$

Khi m và n lớn, cách tính trực tiếp có thể tốn kém. Một cách hiệu quả hơn là sắp xếp toàn bộ $m + n$ điểm theo điểm số $h(x)$, sau đó dùng thứ hạng để tính AUC. Chi phí sắp xếp là:

$$O((m + n) \log(m + n)).$$

Sau khi sắp xếp, ta có thể duyệt danh sách một lần để đếm số cặp positive-negative được xếp đúng, với chi phí $O(m + n)$. Vì vậy, tổng chi phí của cách tính dựa trên sắp xếp là:

$$O((m + n) \log(m + n)).$$

Cách tính này hiệu quả hơn $O(mn)$ khi cả hai lớp đều có nhiều điểm.

6.11 Liên hệ với các thuật toán ranking

Bipartite ranking là một trường hợp đặc biệt của score-based ranking. Thay vì có nhãn ưu tiên tùy ý giữa các cặp, ta chỉ xét các cặp gồm một positive point và một negative point. Với mỗi cặp (x^-, x^+) , ta có thể gán nhãn $y = +1$, vì x^+ nên được xếp cao hơn x^- .

Do đó, các thuật toán pairwise như SVM-ranking hoặc RankBoost có thể được áp dụng cho bipartite ranking. Tuy nhiên, cấu trúc positive-negative giúp bài toán có dạng đơn giản hơn, đồng thời cho phép đánh giá trực tiếp bằng AUC.

6.12 Tóm tắt

Phần này đã trình bày bipartite ranking, một trường hợp quan trọng của ranking trong đó các đối tượng được chia thành hai lớp positive và negative. Mục tiêu của bài toán là học một hàm điểm sao cho positive points được xếp cao hơn negative points. Ta đã định nghĩa lỗi tổng quát, lỗi thực nghiệm, ROC curve và AUC.

Kết quả quan trọng là AUC có thể được hiểu như xác suất một điểm positive được xếp cao hơn một điểm negative được chọn ngẫu nhiên. Trong trường hợp không có hoà điểm, AUC bằng 1 trừ đi ranking loss. Phân tích này cho thấy AUC là một độ đo tự nhiên cho bipartite ranking và đóng vai trò cầu nối giữa lý thuyết ranking và các ứng dụng phân loại hai lớp.

7 Preference-based ranking

Các phần trước chủ yếu xét *score-based ranking*, trong đó mô hình học một hàm điểm $h : \mathcal{X} \rightarrow \mathbb{R}$, rồi sinh ranking bằng cách sắp xếp các đối tượng theo điểm số. Trong phần này, ta xét một thiết lập khác gọi là *preference-based ranking*. Thay vì học điểm số riêng cho từng đối tượng, mô hình học trực tiếp một hàm ưu tiên giữa từng cặp đối tượng.

Ý tưởng chính của preference-based ranking là: nếu ta có thể dự đoán đối tượng nào được ưu tiên hơn trong từng cặp, thì ta có thể dùng các dự đoán này để xây dựng một ranking cho toàn bộ tập đối tượng cần xếp hạng. Tuy nhiên, các quan hệ ưu tiên theo từng cặp có thể không bắc cầu, nên việc chuyển từ dự đoán cặp sang ranking toàn cục không phải lúc nào cũng đơn giản.

7.1 Thiết lập preference-based ranking

Gọi $\mathcal{X}$ là không gian đầu vào. Trong preference-based ranking, ta học một hàm:

$$h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]. \quad (49)$$

Giá trị $h(u, v)$ được hiểu là mức độ mô hình tin rằng u nên được xếp trước v . Nếu

$$h(u, v) > \frac{1}{2},$$

thì mô hình có xu hướng xếp u cao hơn v . Ngược lại, nếu

$$h(u, v) < \frac{1}{2},$$

thì mô hình có xu hướng xếp v cao hơn u .

Trong một số trường hợp, ta xét hàm ưu tiên nhị phân:

$$h : \mathcal{X} \times \mathcal{X} \rightarrow \{0, 1\},$$

trong đó $h(u, v) = 1$ nghĩa là $u \succ v$.

Khác với score-based ranking, hàm $h(u, v)$ không nhất thiết được sinh ra từ một hàm điểm $s : \mathcal{X} \rightarrow \mathbb{R}$. Vì vậy, preference-based ranking có thể biểu diễn các quan hệ ưu

tiên phức tạp hơn, bao gồm cả các quan hệ không bắc cầu.

7.2 Hai giai đoạn của preference-based ranking

Preference-based ranking thường gồm hai giai đoạn.

Giai đoạn 1. Từ dữ liệu huấn luyện gồm các cặp đối tượng có nhãn ưu tiên, ta học một hàm ưu tiên:

$$h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1].$$

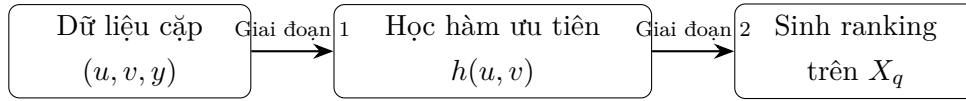
Giai đoạn 2. Với một tập truy vấn hữu hạn:

$$X_q = \{x_1, x_2, \dots, x_n\} \subseteq \mathcal{X},$$

ta dùng hàm h để sinh ra một ranking:

$$\pi = (\pi(1), \pi(2), \dots, \pi(n)).$$

Ở đây, $x_{\pi(1)}$ là đối tượng được xếp cao nhất, $x_{\pi(2)}$ là đối tượng xếp thứ hai, và cứ tiếp tục như vậy.



Học quan hệ từng cặp, sau đó dùng các quan hệ này để tạo ranking toàn cục.

Hình 20: Hai giai đoạn của preference-based ranking.

Trong score-based ranking, ranking được tạo trực tiếp bằng cách sắp xếp điểm số. Trong preference-based ranking, ta cần thêm một bước chuyển từ các dự đoán theo cặp sang một ranking toàn cục.

7.3 Ranking và loss giữa hai thứ tự

Với một tập $X_q = \{x_1, \dots, x_n\}$, một ranking π là một hoán vị của các chỉ số $\{1, \dots, n\}$. Nếu x_i đứng trước x_j trong ranking π , ta viết:

$$x_i \succ_{\pi} x_j.$$

Giả sử tồn tại một ranking mục tiêu π^* . Một cách tự nhiên để đo lỗi của ranking

π là đếm số cặp mà π sắp xếp ngược so với π^* . Pairwise ranking loss được định nghĩa là:

$$L(\pi, \pi^*) = \frac{1}{\binom{n}{2}} \sum_{1 \leq i < j \leq n} \mathbf{1} \left[\pi \text{ và } \pi^* \text{ sắp xếp cặp } (x_i, x_j) \text{ khác nhau} \right]. \quad (50)$$

Nếu dùng ký hiệu $\pi(i)$ là vị trí của x_i trong ranking π , thì hai ranking sắp xếp khác nhau trên cặp (x_i, x_j) khi:

$$(\pi(i) - \pi(j))(\pi^*(i) - \pi^*(j)) < 0.$$

Do đó, có thể viết:

$$L(\pi, \pi^*) = \frac{1}{\binom{n}{2}} \sum_{1 \leq i < j \leq n} \mathbf{1} \left[(\pi(i) - \pi(j))(\pi^*(i) - \pi^*(j)) < 0 \right].$$

Loss này chính là tỉ lệ cặp bị đảo thứ tự, tương tự Kendall tau distance sau khi chuẩn hoá.

7.4 Regret trong preference-based ranking

Trong preference-based ranking, một hàm ưu tiên h có thể không sinh ra ranking hoàn hảo. Do đó, ta quan tâm đến mức độ thiệt hại của ranking mà thuật toán sinh ra so với ranking tốt nhất có thể.

Gọi A là một thuật toán nhận vào tập X_q và hàm ưu tiên h , rồi trả về ranking:

$$\pi_A = A(X_q, h).$$

Gọi π^* là ranking tối ưu theo một tiêu chí loss nào đó. Regret của thuật toán A có thể được viết dưới dạng:

$$\text{Regret}(A) = L(\pi_A) - L(\pi^*). \quad (51)$$

Ở đây, $L(\pi)$ là loss của ranking π theo phân phối hoặc theo tập truy vấn đang xét. Regret càng nhỏ thì thuật toán sinh ranking càng gần với ranking tối ưu. Một thách thức quan trọng là thiết kế thuật toán có regret nhỏ, ngay cả khi hàm ưu tiên h có thể chứa lỗi hoặc không bắc cầu.

7.5 Thuật toán sort-by-degree

Một thuật toán đơn giản để chuyển hàm ưu tiên thành ranking là *sort-by-degree*. Ý tưởng là xem các đối tượng như đỉnh của một đồ thị có hướng. Nếu $h(u, v)$ cho rằng u được ưu tiên hơn v , ta thêm một cạnh từ u đến v . Sau đó, ta xếp các đối tượng theo số lần thắng,

tức là số cạnh đi ra.

Với một tập $X_q = \{x_1, \dots, x_n\}$, định nghĩa degree score của x_i :

$$d(x_i) = \sum_{\substack{j=1 \\ j \neq i}}^n \mathbf{1} \left[h(x_i, x_j) > \frac{1}{2} \right]. \quad (52)$$

Thuật toán sort-by-degree sắp xếp các đối tượng theo $d(x_i)$ giảm dần:

$$d(x_{\pi(1)}) \geq d(x_{\pi(2)}) \geq \dots \geq d(x_{\pi(n)}).$$

Đối tượng thắng nhiều cặp hơn sẽ có degree score lớn hơn và được xếp ở vị trí cao hơn.

7.6 Thuật toán sort-by-degree

Thuật toán sort-by-degree có thể được mô tả như sau.

Bước 1. Với mỗi $x_i \in X_q$, khởi tạo:

$$d(x_i) = 0.$$

Bước 2. Với mỗi cặp $i \neq j$, nếu $h(x_i, x_j) > 1/2$, cập nhật:

$$d(x_i) \leftarrow d(x_i) + 1.$$

Bước 3. Sắp xếp các đối tượng theo $d(x_i)$ giảm dần.

Nếu có hoà điểm degree, ta có thể phá hoà bằng thứ tự bất kỳ hoặc bằng một tiêu chí phụ như điểm trung bình của $h(x_i, x_j)$.

7.7 Hạn chế của sort-by-degree

Sort-by-degree đơn giản và dễ cài đặt, nhưng có một hạn chế quan trọng: khi quan hệ ưu tiên không bắc cầu hoặc chứa nhiều chu trình, degree score có thể không phản ánh chính xác ranking tốt nhất.

Ví dụ, xét ba đối tượng x_1, x_2, x_3 với quan hệ:

$$x_1 \succ x_2, \quad x_2 \succ x_3, \quad x_3 \succ x_1. \quad (53)$$

Khi đó, mỗi đối tượng thắng đúng một lần:

$$d(x_1) = d(x_2) = d(x_3) = 1.$$

Sort-by-degree không thể phân biệt ba đối tượng này chỉ dựa trên degree score. Điều này cho thấy khi quan hệ ưu tiên có chu trình, việc sinh ranking toàn cục trở nên khó hơn.

[MỞ RỘNG] Phản ví dụ về quan hệ không bắc cầu. Quan hệ trong công thức (53) là một phản ví dụ cho giả định bắc cầu. Ta chứng minh rằng không tồn tại hàm điểm thực $s : \mathcal{X} \rightarrow \mathbb{R}$ nào có thể biểu diễn hoàn hảo chu trình này.

Thật vậy, nếu tồn tại s , từ $x_1 \succ x_2$ ta phải có:

$$s(x_1) > s(x_2).$$

Từ $x_2 \succ x_3$, ta phải có:

$$s(x_2) > s(x_3).$$

Kết hợp hai bất đẳng thức trên, ta suy ra:

$$s(x_1) > s(x_2) > s(x_3),$$

nên:

$$s(x_1) > s(x_3).$$

Nhưng quan hệ $x_3 \succ x_1$ lại yêu cầu:

$$s(x_3) > s(x_1).$$

Điều này mâu thuẫn với $s(x_1) > s(x_3)$. Do đó, không tồn tại hàm điểm thực nào biểu diễn hoàn hảo chu trình ưu tiên này.

7.8 Randomized QuickSort cho preference-based ranking

Một hướng tiếp cận khác là dùng *randomized QuickSort*. Thuật toán này lấy cảm hứng từ QuickSort cổ điển, nhưng phép so sánh giữa hai phần tử được thực hiện bằng hàm ưu tiên h .

Với tập X_q , thuật toán chọn ngẫu nhiên một pivot $u \in X_q$. Sau đó, với mỗi phần tử $v \neq u$, thuật toán so sánh v với u . Nếu $h(v, u) > 1/2$, ta đưa v vào tập bên trái pivot, vì v nên xếp trước u . Nếu $h(u, v) > 1/2$, ta đưa v vào tập bên phải pivot, vì u nên xếp trước v .

Điểm quan trọng là thuật toán không yêu cầu quan hệ ưu tiên phải bắc cầu. Nó chỉ cần một hàm h để so sánh pivot với từng phần tử còn lại, rồi đệ quy trên hai tập con.

7.9 Thuật toán randomized QuickSort

Thuật toán có thể được mô tả đệ quy như sau.

Bước 1. Nếu $|X_q| \leq 1$, trả về chính X_q .

Bước 2. Chọn ngẫu nhiên một pivot:

$$u \sim \text{Uniform}(X_q).$$

Bước 3. Khởi tạo hai tập:

$$L = \emptyset, \quad R = \emptyset.$$

Bước 4. Với mỗi $v \in X_q \setminus \{u\}$, nếu $h(v, u) > 1/2$, đưa v vào L ; ngược lại đưa v vào R .

Bước 5. Đệ quy trên L và R , rồi trả về:

$$\text{QuickSort}(L) \parallel (u) \parallel \text{QuickSort}(R),$$

trong đó $\parallel$ ký hiệu phép nối danh sách.

Randomized QuickSort thường cần ít phép so sánh hơn sort-by-degree, đặc biệt khi số lượng đối tượng trong tập truy vấn lớn.

7.10 So sánh sort-by-degree và randomized QuickSort

Hai thuật toán trên đều dùng hàm ưu tiên h để sinh ranking, nhưng có đặc điểm khác nhau.

Sort-by-degree đếm số lần thắng của từng đối tượng, nên đơn giản và dễ cài đặt. Tuy nhiên, thuật toán này cần xét gần như toàn bộ các cặp, do đó số lần gọi hàm ưu tiên thường là $O(n^2)$.

Randomized QuickSort dùng chiến lược chia để trị bằng pivot. Thuật toán có tính ngẫu nhiên do pivot được chọn ngẫu nhiên, và số lần so sánh kỳ vọng là $O(n \log n)$. Vì vậy, randomized QuickSort thường phù hợp hơn khi n lớn, trong khi sort-by-degree phù hợp hơn khi n nhỏ hoặc khi cần một phương pháp rất đơn giản.

7.11 [MỞ RỘNG] Phân tích độ phức tạp

7.11.1 Độ phức tạp của sort-by-degree

Sort-by-degree cần xét các cặp trong tập X_q . Nếu có n đối tượng, số cặp không thứ tự là:

$$\binom{n}{2} = \frac{n(n-1)}{2}. \quad (54)$$

Do đó, số lần gọi hàm ưu tiên h là $O(n^2)$. Sau khi tính degree score, ta cần sắp xếp n đối tượng, có chi phí $O(n \log n)$. Vì vậy, tổng chi phí bị chi phối bởi bước xét cặp:

$$O(n^2 + n \log n) = O(n^2).$$

7.11.2 Độ phức tạp của randomized QuickSort

Randomized QuickSort có cùng cấu trúc với QuickSort cổ điển. Trong trường hợp trung bình, số lần so sánh là:

$$O(n \log n).$$

Trong trường hợp xấu nhất, nếu pivot liên tục tạo ra phân hoạch lệch, số lần so sánh có thể là:

$$O(n^2).$$

Tuy nhiên, do pivot được chọn ngẫu nhiên, độ phức tạp kỳ vọng thường là $O(n \log n)$. Đây là lợi thế lớn so với sort-by-degree khi n lớn.

7.12 Mở rộng sang weighted loss

Trong một số ứng dụng, không phải mọi cặp bị đảo thứ tự đều quan trọng như nhau. Ví dụ, lỗi ở các vị trí đầu danh sách thường nghiêm trọng hơn lỗi ở cuối danh sách. Khi đó, ta có thể dùng weighted loss:

$$L_w(\pi, \pi^*) = \sum_{1 \leq i < j \leq n} w_{ij} \mathbf{1} [\pi \text{ và } \pi^* \text{ sắp xếp cặp } (x_i, x_j) \text{ khác nhau}], \quad (55)$$

trong đó $w_{ij} \geq 0$ là trọng số của cặp (x_i, x_j) .

Nếu muốn nhấn mạnh các lỗi ở đầu danh sách, ta có thể chọn w_{ij} lớn hơn cho các cặp liên quan đến đối tượng có độ liên quan cao. Cách này giúp tiêu chí đánh giá gần hơn với các độ đo trong truy xuất thông tin như NDCG hoặc precision@ k .

7.13 Ví dụ minh họa

Xét bốn đối tượng a, b, c, d . Giả sử hàm ưu tiên dự đoán các quan hệ:

$$b \succ a, \quad b \succ c, \quad b \succ d,$$

và:

$$c \succ a, \quad c \succ d, \quad a \succ d.$$

Khi đó, số lần thắng của từng đối tượng là:

$$d(b) = 3, \quad d(c) = 2, \quad d(a) = 1, \quad d(d) = 0.$$

Sort-by-degree trả về ranking:

$$b \succ c \succ a \succ d.$$

Nếu dùng randomized QuickSort, ranking có thể phụ thuộc vào pivot được chọn ở từng bước. Tuy nhiên, nếu hàm ưu tiên nhất quán với các quan hệ trên, thuật toán cũng có xu hướng đưa b lên đầu và d xuống cuối.

7.14 Tóm tắt

Phần này đã trình bày preference-based ranking, trong đó mô hình học trực tiếp một hàm ưu tiên $h(u, v)$ trên từng cặp đối tượng. Khác với score-based ranking, hàm ưu tiên không nhất thiết phải được sinh ra từ một hàm điểm thực, nên có thể biểu diễn các quan hệ không bắc cầu. Tuy nhiên, điều này cũng làm cho việc sinh ranking toàn cục trở nên khó hơn.

Ta đã trình bày hai thuật toán để chuyển từ hàm ưu tiên sang ranking: sort-by-degree và randomized QuickSort. Sort-by-degree đơn giản nhưng có độ phức tạp $O(n^2)$. Randomized QuickSort phức tạp hơn nhưng có số lần so sánh kỳ vọng $O(n \log n)$. Các phân tích này cho thấy preference-based ranking là một thiết lập linh hoạt, nhưng cần thuật toán cẩn thận để xử lý các quan hệ ưu tiên có thể không nhất quán.

8 Thảo luận về các tiêu chí đánh giá ranking

Các phần trước đã trình bày hai cách tiếp cận chính cho bài toán ranking: score-based ranking và preference-based ranking. Trong score-based ranking, mô hình học một hàm điểm $h : \mathcal{X} \rightarrow \mathbb{R}$, sau đó sắp xếp các phần tử theo giá trị $h(x)$. Trong preference-based ranking, mô hình trực tiếp dự đoán quan hệ ưu tiên giữa từng cặp phần tử. Dù cách biểu diễn mô hình khác nhau, câu hỏi quan trọng nhất vẫn là: một thứ tự xếp hạng được xem là tốt khi nào?

Khác với bài toán phân loại, ranking không chỉ quan tâm từng điểm dữ liệu được dự đoán đúng hay sai, mà quan tâm đến quan hệ thứ tự giữa các phần tử. Trong nhiều ứng dụng như công cụ tìm kiếm, hệ gợi ý phim, phát hiện gian lận hoặc truy xuất thông tin, người dùng thường chỉ quan tâm đến một số phần tử đứng đầu. Vì vậy, việc chọn tiêu chí đánh giá phù hợp có ảnh hưởng trực tiếp đến cách thiết kế thuật toán học, hàm mất mát và cách diễn giải kết quả thực nghiệm.

Phần này thảo luận các tiêu chí đánh giá ranking thường gặp, bao gồm pairwise misranking, margin-based loss, Kendall tau distance, AUC, NDCG, MAP và MRR. Cuối phần, ta so sánh ưu nhược điểm của các tiêu chí này và giải thích vì sao phần thực nghiệm của báo cáo sử dụng pairwise accuracy, ranking loss và Kendall tau distance.

8.1 Pairwise misranking và ranking loss

Trong thiết lập score-based ranking, dữ liệu huấn luyện thường bao gồm các cặp phần tử (x_i, x'_i) cùng với nhãn ưu tiên $y_i \in \{-1, +1\}$. Với một hàm điểm h , cặp (x_i, x'_i) được xếp đúng nếu dấu của hiệu điểm $h(x'_i) - h(x_i)$ phù hợp với nhãn y_i . Ngược lại, nếu mô hình đặt phần tử kém ưu tiên lên trên phần tử được ưu tiên, ta có một lỗi xếp hạng theo cặp.

Empirical pairwise misranking được định nghĩa bởi:

$$\hat{R}(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1} [y_i(h(x'_i) - h(x_i)) \leq 0]. \quad (56)$$

Đại lượng này đo tỉ lệ các cặp ưu tiên bị mô hình sắp xếp sai. Trong báo cáo này, ta gọi đại lượng tương ứng là *ranking loss*. Nếu định nghĩa pairwise accuracy là tỉ lệ các cặp được xếp đúng, ta có:

$$\text{RankingLoss}(h) = 1 - \text{PairwiseAccuracy}(h). \quad (57)$$

Pairwise accuracy được viết dưới dạng:

$$\text{PairwiseAccuracy}(h) = \frac{1}{m} \sum_{i=1}^m \mathbf{1} [y_i(h(x'_i) - h(x_i)) > 0]. \quad (58)$$

Ưu điểm của pairwise misranking là nó phù hợp trực tiếp với bản chất của learning-to-rank: học quan hệ thứ tự giữa từng cặp phần tử. Đây cũng là tiêu chí tự nhiên cho Ranking SVM, RankBoost và các mô hình pairwise learning-to-rank. Tuy nhiên, tiêu chí này xem mọi cặp là quan trọng như nhau. Trong các bài toán truy xuất thông tin, lỗi ở những vị trí đầu danh sách thường nghiêm trọng hơn lỗi ở cuối danh sách. Vì vậy, pairwise loss không phải lúc nào cũng phản ánh đúng chất lượng mà người dùng cảm nhận được.

8.2 Margin-based ranking criteria

Pairwise misranking chỉ kiểm tra một cặp được xếp đúng hay sai, nhưng không đo mức độ tự tin của mô hình. Trong Ranking SVM và các phân tích dựa trên margin, ta không chỉ yêu cầu:

$$y_i(h(x'_i) - h(x_i)) > 0, \quad (59)$$

mà còn yêu cầu hiệu điểm đủ lớn:

$$y_i(h(x'_i) - h(x_i)) \geq \rho, \quad (60)$$

với $\rho > 0$ là margin.

Empirical margin loss có thể được viết dưới dạng:

$$\hat{R}_\rho(h) = \frac{1}{m} \sum_{i=1}^m \Phi_\rho(y_i(h(x'_i) - h(x_i))), \quad (61)$$

trong đó Φ_ρ là hàm margin loss. Với hinge loss, một dạng thường dùng là:

$$\ell_i(h) = \max(0, 1 - y_i(h(x'_i) - h(x_i))). \quad (62)$$

Cách đánh giá dựa trên margin có ý nghĩa quan trọng về mặt lý thuyết. Một mô hình không chỉ cần xếp đúng các cặp, mà còn cần tạo ra khoảng cách điểm đủ lớn giữa phần tử được ưu tiên và phần tử kém ưu tiên hơn. Margin lớn thường giúp mô hình ổn định hơn trước nhiễu và có khả năng tổng quát hoá tốt hơn.

Tuy nhiên, margin-based loss thường là một surrogate loss, tức là hàm mất mát thay thế cho lỗi ranking rời rạc. Nó thuận tiện cho tối ưu hoá vì có dạng lồi hoặc gần lồi trong nhiều trường hợp, nhưng giá trị của nó không phải lúc nào cũng trùng khớp với metric cuối cùng mà ứng dụng quan tâm, chẳng hạn như NDCG@10 hay MRR.

8.3 Kendall tau distance

Kendall tau distance là một tiêu chí tự nhiên khi cần so sánh hai thứ tự xếp hạng hoàn chỉnh. Giả sử σ là ranking dự đoán và σ^* là ranking thật trên n phần tử. Kendall tau distance chuẩn hoá được định nghĩa là tỉ lệ số cặp bị đảo thứ tự:

$$\Delta_{\text{KT}}(\sigma, \sigma^*) = \frac{2}{n(n-1)} \sum_{u \neq v} \mathbf{1}[\sigma(u) < \sigma(v)] \mathbf{1}[\sigma^*(v) < \sigma^*(u)]. \quad (63)$$

Giá trị của Kendall tau distance nằm trong $[0, 1]$. Giá trị bằng 0 nghĩa là ranking dự đoán trùng hoàn toàn với ranking thật. Giá trị càng lớn nghĩa là càng nhiều cặp bị đảo thứ tự.

Kendall tau distance có quan hệ chặt chẽ với pairwise ranking loss vì cả hai đều đếm số cặp bị xếp sai. Điểm khác biệt là Kendall tau thường được dùng khi ta có hai hoán vị hoàn chỉnh, còn pairwise loss thường được định nghĩa trực tiếp trên tập các cặp huấn luyện. Trong phần thực nghiệm, vì dữ liệu tổng hợp có điểm xếp hạng thật cho tất cả các mẫu, Kendall tau distance là một metric phù hợp để đo mức độ gần nhau giữa thứ tự thật và thứ tự dự đoán.

8.4 AUC trong bipartite ranking

Bipartite ranking là trường hợp đặc biệt trong đó các phần tử được chia thành hai lớp: lớp dương $\mathcal{X}^+$ và lớp âm $\mathcal{X}^-$. Mục tiêu là học hàm điểm h sao cho các điểm dương được

xếp cao hơn các điểm âm. Lỗi bipartite misranking được định nghĩa bởi:

$$R(h) = \Pr_{x^- \sim D^-, x^+ \sim D^+} [h(x^+) < h(x^-)] . \quad (64)$$

Trên tập kiểm tra gồm m điểm dương và n điểm âm, empirical bipartite misranking là:

$$\hat{R}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1} [h(x_i^+) < h(x_j^-)] . \quad (65)$$

AUC, viết tắt của Area Under the ROC Curve, có thể được hiểu là xác suất một điểm dương được xếp cao hơn một điểm âm:

$$\text{AUC}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1} [h(x_i^+) \geq h(x_j^-)] . \quad (66)$$

Do đó:

$$\text{AUC}(h) = 1 - \hat{R}(h) . \quad (67)$$

AUC là một tiêu chí rất phù hợp cho bipartite ranking vì nó không phụ thuộc vào một ngưỡng phân loại cố định. Thay vì hỏi một điểm được phân loại là dương hay âm, AUC hỏi liệu điểm dương có được xếp trên điểm âm hay không. Điều này làm cho AUC đặc biệt hữu ích trong các bài toán như phát hiện gian lận, chẩn đoán y khoa, hoặc lọc tài liệu liên quan/không liên quan.

Tuy nhiên, AUC cũng có hạn chế: nó xem mọi cặp dương–âm là quan trọng như nhau. Trong công cụ tìm kiếm, người dùng thường chỉ xem vài kết quả đầu tiên, nên một mô hình có AUC cao nhưng xếp sai ở các vị trí đầu vẫn có thể không tốt trong thực tế.

8.5 Precision@k, MAP và MRR

Trong truy xuất thông tin, chất lượng của các phần tử đứng đầu danh sách thường quan trọng hơn toàn bộ danh sách. Do đó, các tiêu chí như precision@k, MAP và MRR được sử dụng rộng rãi.

Precision@k đo tỉ lệ phần tử liên quan trong k kết quả đầu tiên:

$$\text{Precision@}k = \frac{\text{\#phần tử liên quan trong top } k}{k} . \quad (68)$$

Metric này đơn giản và dễ hiểu. Ví dụ, precision@10 cho biết trong 10 kết quả đầu tiên có bao nhiêu kết quả thật sự liên quan. Tuy nhiên, precision@k không phân biệt vị trí cụ thể bên trong top- k . Một tài liệu liên quan đứng ở vị trí 1 và một tài liệu liên quan đứng ở vị trí 10 được tính như nhau.

Average Precision (AP) khắc phục một phần hạn chế này bằng cách tính trung bình precision tại các vị trí mà phần tử liên quan xuất hiện. Với nhiều truy vấn, ta lấy

trung bình AP trên tất cả truy vấn để thu được Mean Average Precision (MAP). MAP phù hợp với các bài toán có nhãn liên quan dạng nhị phân, chẳng hạn một tài liệu là liên quan hoặc không liên quan đến truy vấn.

MRR, viết tắt của Mean Reciprocal Rank, tập trung vào vị trí của kết quả liên quan đầu tiên. Với một truy vấn, Reciprocal Rank được định nghĩa là:

$$RR = \frac{1}{r}, \quad (69)$$

trong đó r là vị trí của phần tử liên quan đầu tiên. MRR là trung bình của RR trên nhiều truy vấn.

MRR đặc biệt phù hợp trong các ứng dụng mà người dùng chỉ cần tìm một kết quả đúng đầu tiên, ví dụ hỏi đáp, tìm kiếm câu trả lời, hoặc gợi ý một lựa chọn tốt nhất. Tuy nhiên, MRR bỏ qua các phần tử liên quan xuất hiện sau kết quả đúng đầu tiên.

8.6 DCG và NDCG

Trong nhiều bài toán ranking, mức độ liên quan không chỉ là nhị phân. Một tài liệu có thể rất liên quan, liên quan vừa phải, ít liên quan hoặc không liên quan. Khi đó, các tiêu chí như DCG và NDCG được sử dụng.

Discounted Cumulative Gain (DCG) đo tổng độ liên quan của các phần tử trong danh sách, đồng thời giảm trọng số cho các vị trí thấp hơn. Một dạng phổ biến của DCG@k là:

$$DCG@k = \sum_{i=1}^k \frac{2^{r_i} - 1}{\log_2(i + 1)}, \quad (70)$$

trong đó r_i là độ liên quan của phần tử ở vị trí thứ i .

Hệ số $\log_2(i + 1)$ đóng vai trò discount, phản ánh rằng phần tử liên quan ở vị trí cao có giá trị hơn phần tử liên quan ở vị trí thấp. Vì DCG phụ thuộc vào số lượng và độ lớn của nhãn liên quan, người ta thường chuẩn hoá nó bằng giá trị DCG lý tưởng. Normalized Discounted Cumulative Gain được định nghĩa bởi:

$$NDCG@k = \frac{DCG@k}{IDCG@k}, \quad (71)$$

trong đó IDCG@k là DCG@k của thứ tự lý tưởng.

NDCG là một trong những metric phổ biến nhất trong learning-to-rank hiện đại vì nó xử lý được nhãn liên quan nhiều mức và nhấn mạnh các vị trí đầu danh sách. Đây là lý do nhiều thuật toán thực tế như LambdaRank, LambdaMART và các biến thể gần đây thường cố gắng tối ưu trực tiếp hoặc gián tiếp NDCG.

8.7 So sánh các tiêu chí đánh giá

Mỗi tiêu chí đánh giá phản ánh một khía cạnh khác nhau của bài toán ranking. Không có một metric duy nhất phù hợp cho mọi ứng dụng. Bảng 5 tóm tắt đặc điểm chính của các tiêu chí đã thảo luận.

Bảng 5: So sánh một số tiêu chí đánh giá ranking

Tiêu chí	Ý nghĩa	Ưu điểm	Hạn chế
Pairwise accuracy	Tỉ lệ cặp được xếp đúng	Phù hợp với pairwise ranking, dễ hiểu, dễ cài đặt	Không nhấn mạnh top- k
Ranking loss	Tỉ lệ cặp bị xếp sai	Liên hệ trực tiếp với pairwise misranking	Xem mọi cặp quan trọng như nhau
Margin loss	Đo lỗi có xét khoảng cách điểm	Thuận tiện cho tối ưu hoá và phân tích lý thuyết	Là surrogate loss, không phải metric cuối cùng
Kendall tau distance	Tỉ lệ cặp bị đảo trong hai ranking	Phù hợp khi so sánh toàn bộ thứ tự	Không tập trung riêng vào top- k
AUC	Xác suất điểm dương xếp trên điểm âm	Phù hợp với bipartite ranking, không cần ngưỡng phân loại	Không nhấn mạnh vị trí đầu danh sách
Precision@ k	Tỉ lệ phần tử liên quan trong top- k	Đơn giản, tập trung vào kết quả đầu	Không xét thứ tự bên trong top- k
MAP	Trung bình precision tại các vị trí liên quan	Phù hợp với nhãn nhị phân và nhiều truy vấn	Ít phù hợp với nhãn liên quan nhiều mức
MRR	Nghịch đảo vị trí kết quả liên quan đầu tiên	Phù hợp với hỏi đáp hoặc tìm kiếm một kết quả đúng	Bỏ qua các kết quả liên quan phía sau
NDCG@ k	Độ liên quan có discount theo vị trí	Phù hợp với search/recommendation, hỗ trợ nhãn nhiều mức	Cần relevance score và IDCG để chuẩn hoá

Từ bảng trên, có thể thấy các tiêu chí pairwise như pairwise accuracy, ranking loss và Kendall tau distance phù hợp với phân tích lý thuyết trong các phần trước. Trong khi

đó, các tiêu chí như NDCG, MAP và MRR phù hợp hơn với hệ thống truy xuất thông tin thực tế, nơi vị trí của các kết quả đầu tiên có ý nghĩa lớn đối với trải nghiệm người dùng.

8.8 Liên hệ với thực nghiệm trong báo cáo

Trong phần thực nghiệm, nhóm lựa chọn cài đặt mô hình pairwise ranking tuyến tính và huấn luyện bằng pairwise hinge loss. Vì vậy, các metric được chọn cần phản ánh trực tiếp chất lượng học quan hệ ưu tiên giữa các cặp mẫu.

Cụ thể, nhóm sử dụng ba tiêu chí chính:

- **Pairwise accuracy:** đo tỉ lệ các cặp ưu tiên được mô hình sắp xếp đúng.
- **Ranking loss:** đo tỉ lệ các cặp bị xếp sai, bằng $1 - \text{pairwise accuracy}$.
- **Kendall tau distance:** đo mức độ khác biệt giữa ranking thật và ranking dự đoán trên toàn bộ tập mẫu.

Ngoài ra, nhóm sử dụng thêm top- k overlap để kiểm tra xem mô hình có khôi phục đúng các phần tử đứng đầu hay không. Metric này không phải là tiêu chí lý thuyết chính trong chương, nhưng giúp minh họa trực quan hơn vai trò của ranking ở các vị trí đầu danh sách.

Việc không sử dụng NDCG, MAP hoặc MRR trong thực nghiệm chính là có chủ đích. Dữ liệu tổng hợp trong báo cáo được sinh từ một hàm điểm tuyến tính liên tục, không phải từ các truy vấn với danh sách tài liệu và nhãn relevance nhiều mức. Do đó, pairwise accuracy, ranking loss và Kendall tau distance phù hợp hơn với mục tiêu kiểm chứng lý thuyết về pairwise learning-to-rank. Nếu mở rộng thực nghiệm sang dữ liệu truy xuất thông tin thực tế, chẳng hạn dữ liệu query-document, thì NDCG@ k , MAP và MRR sẽ là các metric cần được bổ sung.

Tóm lại, tiêu chí đánh giá ranking cần được chọn theo đúng mục tiêu ứng dụng. Với bài toán học quan hệ ưu tiên theo cặp, pairwise loss và Kendall tau distance là lựa chọn tự nhiên. Với bipartite ranking, AUC là tiêu chí phù hợp. Với search engine và recommender system, các metric tập trung vào top- k như NDCG@ k , MAP và MRR thường phản ánh tốt hơn chất lượng thực tế của hệ thống.

9 Thực nghiệm

Phần này trình bày các thực nghiệm nhằm minh họa các khái niệm lý thuyết về ranking đã trình bày ở các phần trước. Nhóm sử dụng dữ liệu tổng hợp thay vì dữ liệu thực tế phức tạp để có thể kiểm soát hàm sinh dữ liệu, mức nhiễu và thứ tự đúng của các mẫu.

Mục tiêu của phần thực nghiệm gồm:

1. Minh hoạ quá trình huấn luyện thông qua loss và các metric ranking.
2. Cài đặt từ đầu mô hình pairwise learning-to-rank bằng Python và NumPy.
3. Kiểm chứng ảnh hưởng của nhiễu và kích thước mẫu đến chất lượng ranking.
4. Trình bày các đoạn code chính bằng môi trường `lstlisting`.

9.1 Thiết lập thực nghiệm

Ta xét bài toán ranking với n mẫu, mỗi mẫu có vector đặc trưng $x_i \in \mathbb{R}^d$. Dữ liệu tổng hợp được sinh theo mô hình tuyến tính:

$$y_i = x_i^\top w^* + \epsilon_i, \quad (72)$$

trong đó w^* là vector trọng số thật và ϵ_i là nhiễu Gaussian. Điểm y_i được dùng làm điểm xếp hạng thật của mẫu x_i .

Từ các điểm thật này, ta sinh tập các cặp ưu tiên. Với mỗi cặp (i, j) , nhãn ưu tiên được định nghĩa bởi:

$$y_{ij} = \begin{cases} 1, & \text{nếu } y_i > y_j, \\ -1, & \text{ngược lại.} \end{cases} \quad (73)$$

Mục tiêu của mô hình là học một hàm điểm $s(x)$ sao cho nếu $y_i > y_j$ thì $s(x_i) > s(x_j)$.

Trong thực nghiệm chính, nhóm sử dụng $n = 100$, $d = 2$, độ lệch chuẩn nhiễu $\sigma = 0.1$, số epoch là 200, learning rate 0.05, regularization 10^{-3} , và random seed bằng 42. Với 100 mẫu, số cặp ưu tiên sinh ra là 4950.

9.2 Cài đặt thuật toán pairwise ranking từ đầu

Mô hình được sử dụng là mô hình scoring tuyến tính:

$$s_w(x) = w^\top x. \quad (74)$$

Với một cặp ưu tiên (i, j) , mô hình cần tạo ra hiệu điểm phù hợp:

$$y_{ij}(s_w(x_i) - s_w(x_j)) > 0.$$

Để học mô hình, nhóm sử dụng pairwise hinge loss:

$$\ell_{ij}(w) = \max(0, 1 - y_{ij}w^\top(x_i - x_j)). \quad (75)$$

Hàm mất mát trên toàn bộ tập cặp ưu tiên là:

$$L(w) = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \max(0, 1 - y_{ij} w^\top (x_i - x_j)) + \frac{\lambda}{2} \|w\|_2^2, \quad (76)$$

trong đó $\mathcal{P}$ là tập các cặp ưu tiên và λ là hệ số regularization.

Đoạn code sau minh hoạ cách sinh các cặp ưu tiên từ điểm xếp hạng thật.

Listing 1: Sinh các cặp ưu tiên từ điểm xếp hạng thật

```

1 def generate_preference_pairs(scores):
2     scores = np.asarray(scores)
3     n_samples = scores.shape[0]
4     pairs = []
5     labels = []
6
7     for i in range(n_samples):
8         for j in range(i + 1, n_samples):
9             pairs.append((i, j))
10            labels.append(1 if scores[i] > scores[j] else -1)
11
12    return np.asarray(pairs, dtype=int), np.asarray(labels, dtype=int)

```

Mô hình ranking tuyến tính được cài đặt bằng vector trọng số w . Hàm `predict` trả về điểm dự đoán, còn hàm `update` thực hiện bước gradient descent.

Listing 2: Mô hình scoring tuyến tính cho pairwise ranking

```

1 class LinearRanker:
2     def __init__(self, n_features, seed=42):
3         rng = np.random.default_rng(seed)
4         self.w = rng.normal(loc=0.0, scale=0.01, size=n_features)
5
6     def predict(self, X):
7         return np.asarray(X) @ self.w
8
9     def update(self, grad, lr):
10        self.w = self.w - lr * grad

```

Với các cặp vi phạm margin, tức là:

$$y_{ij} w^\top (x_i - x_j) < 1,$$

gradient được cập nhật theo hướng làm tăng khoảng cách điểm giữa mẫu được ưu tiên

và mẫu kém ưu tiên hơn.

Listing 3: Pairwise hinge loss và gradient

```

1 def pairwise_hinge_loss_and_grad(model, X, pairs, labels, reg
  =0.0):
2     X = np.asarray(X)
3     pairs = np.asarray(pairs, dtype=int)
4     labels = np.asarray(labels)
5
6     diffs = X[pairs[:, 0]] - X[pairs[:, 1]]
7     margins = labels * (diffs @ model.w)
8     violations = margins < 1.0
9
10    hinge_losses = np.maximum(0.0, 1.0 - margins)
11    loss = float(np.mean(hinge_losses))
12
13    grad = np.zeros_like(model.w)
14    if np.any(violations):
15        grad = np.mean((-labels[violations, None]) *
16                        diffs[violations], axis=0)
17
18    loss += 0.5 * reg * float(model.w @ model.w)
19    grad += reg * model.w
20
21    return loss, grad

```

Quá trình huấn luyện được thực hiện bằng full-batch gradient descent. Ở mỗi epoch, thuật toán tính loss, gradient, cập nhật trọng số và lưu lại các metric để vẽ đồ thị.

Listing 4: Vòng lặp huấn luyện bằng gradient descent

```

1 def train_ranker(model, X, pairs, labels,
2                 lr=0.05, epochs=200, reg=1e-3):
3     history = {
4         "loss": [],
5         "pairwise_accuracy": [],
6         "ranking_loss": []
7     }
8
9     for epoch in range(1, epochs + 1):
10        loss, grad = pairwise_hinge_loss_and_grad(
11            model, X, pairs, labels, reg=reg

```

```

12         )
13
14     model.update(grad, lr=lr)
15
16     pred_scores = model.predict(X)
17     history["loss"].append(loss)
18     history["pairwise_accuracy"].append(
19         pairwise_accuracy(pred_scores, pairs, labels)
20     )
21     history["ranking_loss"].append(
22         ranking_loss(pred_scores, pairs, labels)
23     )
24
25     return history

```

9.3 Các độ đo đánh giá

Nhóm sử dụng bốn độ đo chính để đánh giá chất lượng ranking.

9.3.1 Pairwise accuracy

Pairwise accuracy đo tỉ lệ các cặp ưu tiên được mô hình sắp xếp đúng:

$$\text{PairwiseAccuracy} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \mathbf{1}[\text{sign}(s(x_i) - s(x_j)) = y_{ij}]. \quad (77)$$

9.3.2 Ranking loss

Ranking loss là tỉ lệ các cặp bị xếp sai:

$$\text{RankingLoss} = 1 - \text{PairwiseAccuracy}. \quad (78)$$

9.3.3 Kendall tau distance

Kendall tau distance đo mức độ bất đồng giữa ranking thật và ranking dự đoán:

$$\text{KendallTauDistance} = \frac{\text{\#cặp bất đồng}}{\binom{n}{2}}. \quad (79)$$

9.3.4 Top-k overlap

Top- k overlap đo mức độ trùng nhau giữa tập k phần tử đứng đầu theo ranking thật và ranking dự đoán:

$$\text{TopKOverlap} = \frac{|\text{TopK}_{\text{true}} \cap \text{TopK}_{\text{pred}}|}{k}. \quad (80)$$

Trong thực nghiệm, nhóm sử dụng $k = 10$.

9.4 Kết quả thực nghiệm chính

Bảng 6 trình bày kết quả của mô hình trên thiết lập chính với 100 mẫu, 2 đặc trưng và độ lệch chuẩn nhiễu bằng 0.1.

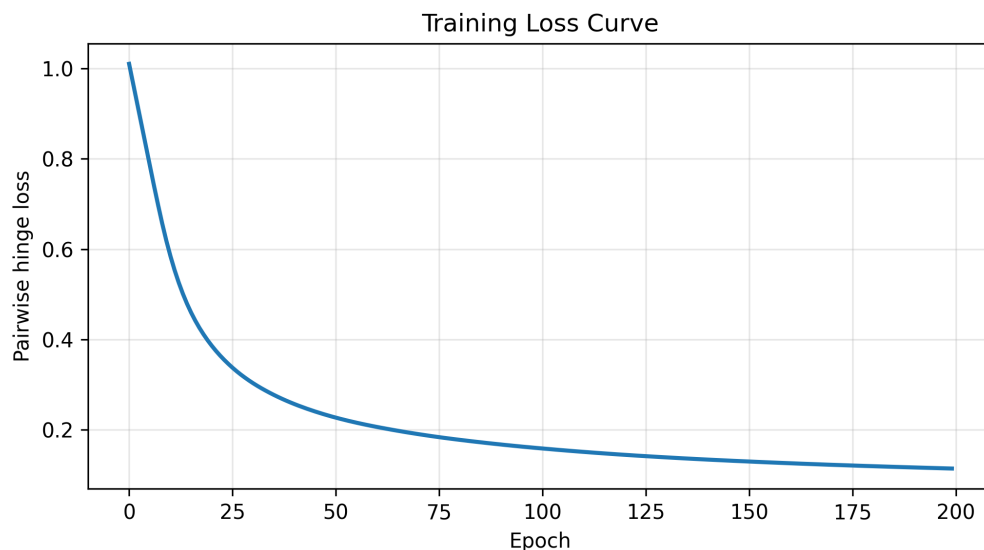
Bảng 6: Kết quả thực nghiệm chính của mô hình pairwise ranking

Đại lượng	Giá trị
Số mẫu	100
Số đặc trưng	2
Độ lệch chuẩn nhiễu	0.1
Số cặp ưu tiên	4950
Số epoch	200
Learning rate	0.05
Regularization	0.001
Pairwise accuracy	0.985051
Ranking loss	0.014949
Kendall tau distance	0.014949
Top-10 overlap	1.000000
Final training loss	0.114590

Kết quả cho thấy mô hình tuyến tính học được ranking tốt trên dữ liệu tổng hợp. Pairwise accuracy đạt khoảng 0.985, ranking loss và Kendall tau distance đều xấp xỉ 0.015. Top-10 overlap bằng 1.0, nghĩa là mô hình khôi phục đúng toàn bộ 10 phần tử đứng đầu.

9.5 Minh hoạ quá trình hội tụ

Hình 21 biểu diễn sự thay đổi của pairwise hinge loss trong quá trình huấn luyện.

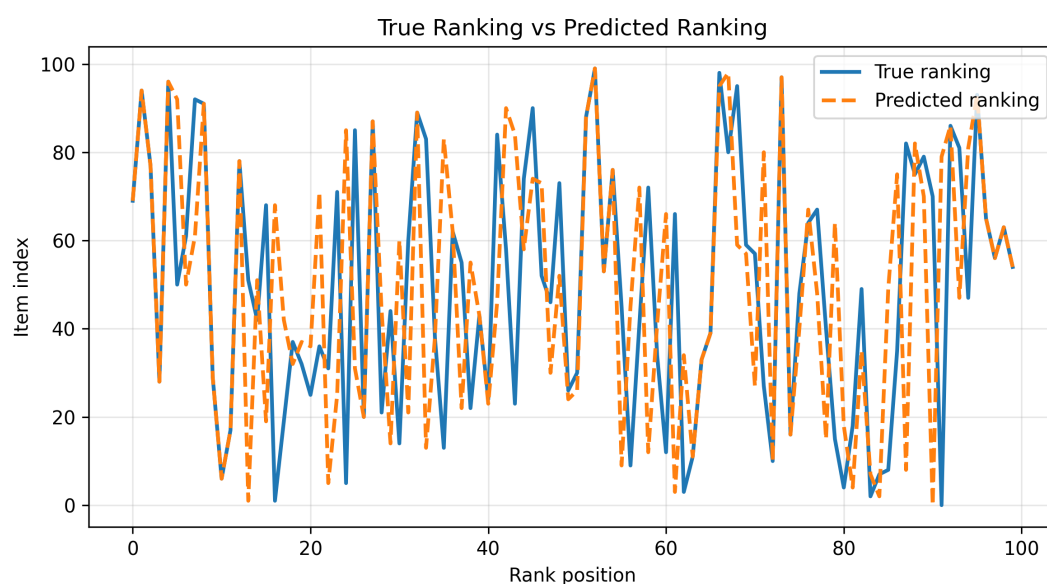


Hình 21: Đường cong pairwise hinge loss trong quá trình huấn luyện. Loss giảm nhanh ở các epoch đầu và tiếp tục giảm chậm dần về sau, cho thấy quá trình tối ưu đang hội tụ.

Loss giảm mạnh ở giai đoạn đầu và giảm chậm dần về sau. Điều này phù hợp với gradient descent: ban đầu mô hình còn sai nhiều nên gradient lớn; khi số cặp vi phạm margin giảm, tốc độ cải thiện cũng nhỏ hơn.

9.6 So sánh ranking thật và ranking dự đoán

Hình 22 so sánh thứ tự ranking thật và thứ tự ranking dự đoán sau khi huấn luyện.



Hình 22: So sánh thứ tự ranking thật và ranking dự đoán. Hai đường càng gần nhau thì mô hình càng bảo toàn tốt quan hệ thứ tự giữa các mẫu.

Ranking dự đoán có xu hướng bám sát ranking thật. Tuy nhiên, do trực tung biểu diễn chỉ số mẫu theo từng vị trí xếp hạng, đồ thị có dạng dao động mạnh. Vì vậy, kết luận định lượng nên dựa chủ yếu vào pairwise accuracy, ranking loss và Kendall tau distance.

9.7 Ảnh hưởng của nhiễu đến chất lượng ranking

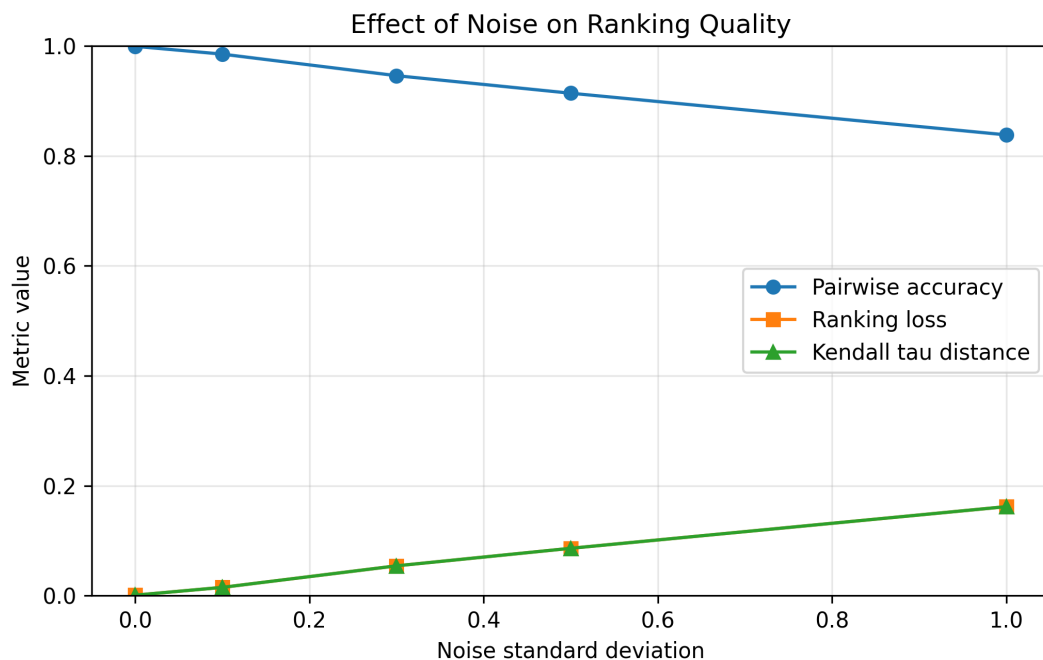
Để kiểm chứng ảnh hưởng của nhiễu, nhóm giữ cố định $n = 100$ và thay đổi độ lệch chuẩn nhiễu:

$$\sigma \in \{0.0, 0.1, 0.3, 0.5, 1.0\}.$$

Kết quả được trình bày trong Bảng 7 và Hình 23.

Bảng 7: Ảnh hưởng của nhiễu đến chất lượng ranking

Noise	Pairwise acc.	Ranking loss	Kendall tau	Top-10 overlap
0.0	0.999192	0.000808	0.000808	1.0
0.1	0.985051	0.014949	0.014949	1.0
0.3	0.945859	0.054141	0.054141	1.0
0.5	0.913939	0.086061	0.086061	0.8
1.0	0.838182	0.161818	0.161818	0.7



Hình 23: Ảnh hưởng của độ lệch chuẩn nhiễu đến chất lượng ranking. Khi nhiễu tăng, pairwise accuracy giảm, trong khi ranking loss và Kendall tau distance tăng.

Khi nhiễu tăng, chất lượng ranking giảm rõ rệt. Với $\sigma = 0.0$, dữ liệu gần như hoàn

toàn tuyến tính nên pairwise accuracy gần bằng 1. Khi $\sigma = 1.0$, pairwise accuracy giảm xuống khoảng 0.838, ranking loss tăng lên khoảng 0.162, và top-10 overlap giảm còn 0.7. Điều này cho thấy nhiễu làm quan hệ ưu tiên kém ổn định hơn, khiến mô hình khó khôi phục ranking đúng.

9.8 Ảnh hưởng của kích thước mẫu

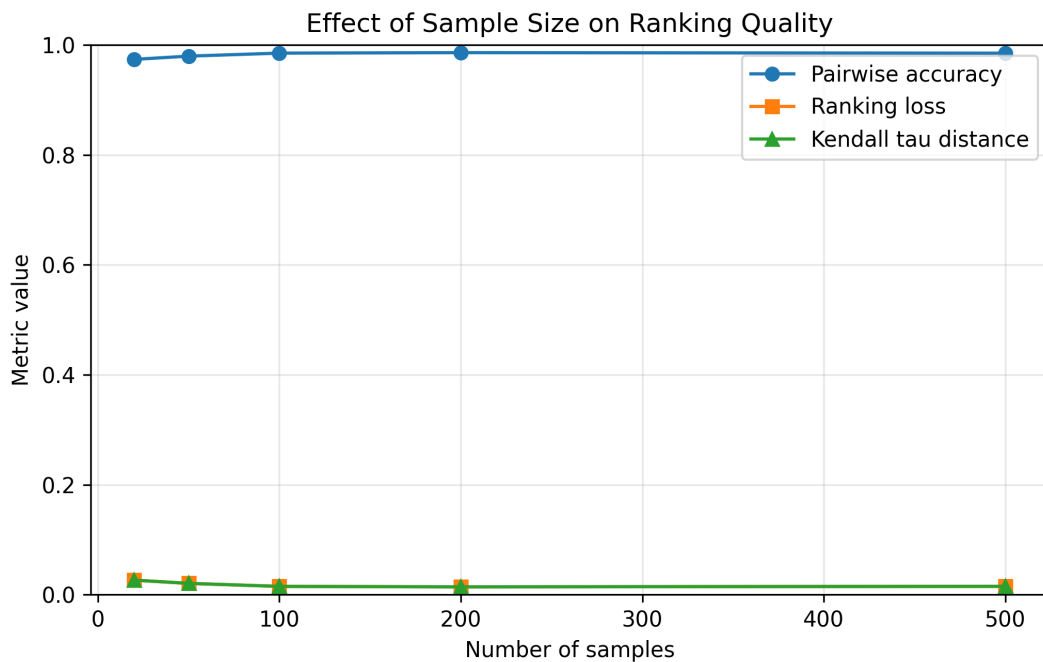
Trong thí nghiệm tiếp theo, nhóm cố định $\sigma = 0.1$ và thay đổi số lượng mẫu:

$$n \in \{20, 50, 100, 200, 500\}.$$

Kết quả được trình bày trong Bảng 8 và Hình 24.

Bảng 8: Ảnh hưởng của kích thước mẫu đến chất lượng ranking

Số mẫu	Pairwise acc.	Ranking loss	Kendall tau	Top-10 overlap
20	0.973684	0.026316	0.026316	0.9
50	0.979592	0.020408	0.020408	0.9
100	0.985051	0.014949	0.014949	1.0
200	0.985930	0.014070	0.014070	0.9
500	0.984970	0.015030	0.015030	1.0



Hình 24: Ảnh hưởng của kích thước mẫu đến chất lượng ranking. Khi số mẫu tăng, pairwise accuracy nhìn chung tăng và ranking loss giảm, sau đó ổn định ở mức tốt.

Khi số mẫu tăng từ 20 lên 100, pairwise accuracy tăng và ranking loss giảm. Khi số mẫu tiếp tục tăng lên 200 và 500, chất lượng ranking duy trì ở mức cao. Do dữ liệu được sinh từ mô hình tuyến tính đơn giản và nhiễu nhỏ, mô hình đã đạt kết quả tốt ngay cả khi số mẫu chưa quá lớn.

9.9 Khả năng tái tạo thực nghiệm

Các thực nghiệm được thiết kế để có thể tái tạo. Nhóm sử dụng random seed cố định trong quá trình sinh dữ liệu, khởi tạo mô hình và chạy thực nghiệm. Các tham số như số mẫu, số đặc trưng, số epoch, learning rate và regularization đều được ghi rõ.

Các kết quả được lưu thành các file CSV:

- `results/tables/main_results.csv`: kết quả thực nghiệm chính.
- `results/tables/noise_results.csv`: kết quả thí nghiệm thay đổi mức nhiễu.
- `results/tables/sample_size_results.csv`: kết quả thí nghiệm thay đổi kích thước mẫu.

Các hình minh họa được lưu trong thư mục `figures/`. Các thư viện chính gồm `numpy`, `matplotlib`, `csv` và `pathlib`. Phiên bản cụ thể của các thư viện được liệt kê trong file `requirements.txt`.

9.10 Nhận xét chung

Các thực nghiệm cho thấy mô hình pairwise ranking tuyến tính có thể học tốt quan hệ ưu tiên khi dữ liệu được sinh từ một hàm tuyến tính. Điều này thể hiện qua pairwise accuracy cao, ranking loss thấp và top-10 overlap tốt.

Đường cong loss cho thấy quá trình huấn luyện có xu hướng hội tụ. Khi nhiễu tăng, số lượng cặp bị đảo thứ tự tăng lên, làm giảm pairwise accuracy và tăng ranking loss. Khi kích thước mẫu tăng, mô hình quan sát được nhiều cặp ưu tiên hơn nên học ổn định hơn, dù mức cải thiện sau 100 mẫu không còn quá lớn.

Nhìn chung, kết quả thực nghiệm phù hợp với lý thuyết về pairwise learning-to-rank: mô hình học một hàm scoring sao cho bảo toàn nhiều nhất có thể các quan hệ ưu tiên giữa các cặp mẫu.

10 Tổng hợp và thảo luận các bài báo liên quan

Sau khi trình bày các khái niệm nền tảng của learning-to-rank, phần này tổng hợp một số bài báo liên quan nhằm làm rõ cách các lý thuyết trong chương được sử dụng và mở rộng trong các nghiên cứu hiện đại. Các bài báo được chọn đại diện cho ba hướng phát triển

khác nhau: tối ưu metric ranking trong các mô hình boosting, diễn giải decision-focused learning như một bài toán ranking, và phân tích lý thuyết cho label ranking trong điều kiện có nhiễu.

Với mỗi bài báo, ta trình bày ba nội dung chính: bài toán đặt ra, đóng góp chính và mối liên hệ với các lý thuyết đã trình bày trong chương. Cuối phần là nhận xét tổng quan về xu hướng phát triển của lĩnh vực, các hạn chế còn tồn tại và một số câu hỏi mở.

10.1 Bài báo 1: Which Tricks are Important for Learning to Rank?

Bài báo *Which Tricks are Important for Learning to Rank?* nghiên cứu các thuật toán learning-to-rank dựa trên gradient-boosted decision trees, trong đó có LambdaMART, YetiRank, StochasticRank và biến thể mới YetiLoss. Bối cảnh của bài báo là các hệ thống learning-to-rank thực tế, đặc biệt trong truy xuất thông tin, nơi mô hình cần xếp hạng danh sách tài liệu theo mức độ liên quan với truy vấn.

10.1.1 Bài toán đặt ra

Bài toán chính của bài báo là tìm hiểu những yếu tố kỹ thuật nào thực sự quan trọng khi huấn luyện mô hình learning-to-rank. Cụ thể, bài báo đặt ra các câu hỏi:

- Có nên tối ưu trực tiếp một ranking loss đã được làm trơn hay nên tối ưu một surrogate loss lồi?
- Việc làm trơn ranking loss bằng nhiễu ngẫu nhiên có giúp cải thiện chất lượng mô hình không?
- Các cách xây dựng upper bound khác nhau ảnh hưởng như thế nào đến hiệu quả của thuật toán?
- Liệu có thể mở rộng YetiRank để tối ưu các metric cụ thể như MRR, MAP, ERR hay NDCG không?

Đây là bài toán quan trọng vì ranking loss thường phụ thuộc vào phép sắp xếp, mà phép sắp xếp là không khả vi. Do đó, các thuật toán học như gradient boosting không thể tối ưu trực tiếp ranking metric nếu không có một dạng surrogate hoặc một cách làm trơn phù hợp.

10.1.2 Đóng góp chính

Đóng góp đầu tiên của bài báo là phân tích các thuật toán GBDT-based learning-to-rank trong cùng một thiết lập thực nghiệm. Thay vì chỉ so sánh kết quả cuối cùng, bài báo

phân tích vai trò của từng thành phần như smoothing, convex upper bound và cách chọn các cặp tài liệu dùng trong gradient.

Đóng góp thứ hai là chỉ ra rằng YetiRank ngầm tối ưu một dạng biến thể của DCG với discount hàm mũ. Từ quan sát này, bài báo đề xuất YetiLoss, một biến thể của YetiRank cho phép thay đổi trọng số gradient theo ranking metric mong muốn. Nhờ đó, YetiLoss có thể tối ưu các metric như MRR, MAP hoặc ERR thay vì chỉ phù hợp với một dạng DCG cố định.

Đóng góp thứ ba là kết quả thực nghiệm trên nhiều bộ dữ liệu learning-to-rank chuẩn. Bài báo cho thấy YetiRank đạt kết quả rất tốt với NDCG, trong khi YetiLoss thường tốt hơn với MAP và MRR. Ngoài ra, bài báo cũng chỉ ra rằng smoothing có vai trò quan trọng: khi bỏ smoothing, chất lượng của YetiLoss giảm rõ rệt.

10.1.3 Liên hệ với lý thuyết trong chương

Bài báo này liên hệ trực tiếp với phần score-based ranking. Mô hình học một hàm điểm $f(x)$, sau đó sắp xếp các tài liệu theo điểm số để tạo ranking. Đây chính là thiết lập:

$$x \mapsto f(x),$$

trong đó ranking được sinh ra bằng cách sắp xếp các giá trị $f(x)$.

Bài báo cũng liên hệ với phần ranking loss và các tiêu chí đánh giá như NDCG, MAP, MRR và ERR. Trong chương, ta đã thấy các metric này không chỉ đo cặp đúng sai như pairwise loss, mà còn nhấn mạnh vị trí của tài liệu trong danh sách. Bài báo mở rộng ý tưởng này bằng cách nghiên cứu cách tối ưu trực tiếp hoặc gián tiếp các metric đó trong mô hình GBDT.

Ngoài ra, bài báo còn liên hệ với phần surrogate loss. Vì ranking loss không khả vi, các thuật toán như LambdaMART xây dựng upper bound khả vi để tối ưu. YetiLoss cũng dùng surrogate nhưng khác LambdaMART ở chỗ nó chỉ xét các hoán vị lân cận trong ranking sau khi thêm nhiễu. Điều này gần với trực giác rằng một bước cập nhật nhỏ của mô hình chỉ có thể làm thay đổi cục bộ thứ tự ranking.

10.1.4 Hạn chế và nhận xét

Một hạn chế của bài báo là phạm vi phân tích chủ yếu tập trung vào mô hình GBDT. Đây là lựa chọn hợp lý vì GBDT vẫn rất mạnh trên dữ liệu bảng, nhưng kết luận của bài báo chưa chắc chuyển trực tiếp sang các mô hình neural ranking. Ngoài ra, YetiLoss cho kết quả tốt với MAP và MRR, nhưng với NDCG thì YetiRank vẫn thường mạnh hơn. Điều này cho thấy không có một thuật toán duy nhất tối ưu cho mọi metric.

Một hướng mở tự nhiên là nghiên cứu liệu các kết luận về smoothing, local permutation và convex surrogate có còn đúng trong neural rankers, recommender systems

hoặc các bài toán ranking có dữ liệu đa phương thức hay không.

10.2 Bài báo 2: Decision-Focused Learning: Through the Lens of Learning to Rank

Bài báo *Decision-Focused Learning: Through the Lens of Learning to Rank* mở rộng learning-to-rank sang một bối cảnh khác: decision-focused learning, hay còn gọi là predict-and-optimize. Trong thiết lập này, mô hình học máy không chỉ dự đoán một đại lượng trung gian, mà dự đoán các tham số được đưa vào một bài toán tối ưu tổ hợp ở bước sau.

10.2.1 Bài toán đặt ra

Trong predict-and-optimize, ta thường có một bài toán tối ưu tổ hợp với hàm mục tiêu phụ thuộc vào một vector tham số chưa biết c . Mô hình học máy nhận đầu vào x và dự đoán $\hat{c}$. Sau đó, lời giải tối ưu được tính bằng cách giải bài toán tối ưu với $\hat{c}$. Mục tiêu cuối cùng không phải là dự đoán $\hat{c}$ thật gần c , mà là tạo ra quyết định có regret nhỏ.

Bài toán có thể viết ở dạng:

$$v^*(c) \in \arg \min_{v \in V} f(v, c), \quad (81)$$

trong đó V là tập nghiệm khả thi của bài toán tối ưu tổ hợp. Regret của dự đoán $\hat{c}$ được định nghĩa là:

$$\text{Regret}(\hat{c}, c) = f(v^*(\hat{c}), c) - f(v^*(c), c). \quad (82)$$

Khó khăn chính là ánh xạ $\hat{c} \mapsto v^*(\hat{c})$ thường không khả vi vì V là tập rời rạc. Do đó, việc lan truyền gradient qua bộ giải tối ưu tổ hợp là rất khó và tốn kém.

10.2.2 Đóng góp chính

Đóng góp quan trọng nhất của bài báo là diễn giải decision-focused learning như một bài toán learning-to-rank. Với mỗi instance, các nghiệm khả thi $v \in V$ được xem như các “items” cần xếp hạng. Hàm mục tiêu $f(v, c)$ tạo ra thứ tự đúng trên các nghiệm khả thi: nghiệm có giá trị mục tiêu nhỏ hơn nên được xếp cao hơn.

Từ góc nhìn này, thay vì cố gắng đạo hàm trực tiếp qua nghiệm tối ưu $v^*(\hat{c})$, bài báo đề xuất học sao cho hàm mục tiêu dự đoán $f(v, \hat{c})$ tạo ra cùng thứ tự với hàm mục tiêu thật $f(v, c)$ trên một tập con nghiệm khả thi $S \subseteq V$. Vì không thể liệt kê toàn bộ V , bài báo dùng một tập nghiệm con S và cập nhật dần trong quá trình huấn luyện.

Bài báo đề xuất ba nhóm loss theo learning-to-rank:

- *Pointwise loss*: so khớp giá trị mục tiêu dự đoán và giá trị mục tiêu thật trên từng nghiệm.

- *Pairwise loss*: yêu cầu nghiệm tốt hơn theo c cũng phải có điểm tốt hơn theo $\hat{c}$.
- *Listwise loss*: so sánh phân phối xác suất trên toàn bộ danh sách nghiệm trong tập S .

Đặc biệt, bài báo chỉ ra rằng loss noise-contrastive estimation trong nghiên cứu trước có thể được xem như một trường hợp đặc biệt của pairwise ranking theo dạng best-versus-rest.

10.2.3 Liên hệ với lý thuyết trong chương

Bài báo này liên hệ rõ với cả score-based ranking và pairwise ranking. Trong chương, ta học hàm điểm $h(x)$ để sắp xếp các đối tượng. Ở đây, đối tượng cần xếp hạng không phải là tài liệu hay sản phẩm, mà là các nghiệm khả thi v của một bài toán tối ưu. Hàm điểm tương ứng là:

$$v \mapsto -f(v, \hat{c}),$$

nếu bài toán là bài toán cực tiểu. Nghiệm có giá trị mục tiêu nhỏ hơn sẽ có điểm ranking cao hơn.

Pairwise loss trong bài báo cũng rất gần với Ranking SVM và pairwise hinge loss đã trình bày. Với một cặp nghiệm (v^p, v^q) , nếu v^p tốt hơn v^q theo c , ta muốn:

$$f(v^p, \hat{c}) < f(v^q, \hat{c}).$$

Dạng margin ranking loss của bài báo có cùng tinh thần với pairwise hinge loss: không chỉ yêu cầu đúng thứ tự, mà còn muốn có một khoảng cách đủ lớn giữa nghiệm tốt và nghiệm kém hơn.

Bài báo cũng liên hệ với phần preference-based ranking. Trong decision-focused learning, ta có thể xem mỗi cặp nghiệm khả thi như một cặp ưu tiên. Nếu nghiệm v^p cho giá trị mục tiêu tốt hơn v^q , thì $v^p \succ v^q$. Do đó, học để giảm regret có thể được xem như học để bảo toàn quan hệ ưu tiên giữa các nghiệm.

10.2.4 Hạn chế và nhận xét

Điểm mạnh của bài báo là kết nối hai lĩnh vực tưởng như khác nhau: decision-focused learning và learning-to-rank. Tuy nhiên, phương pháp vẫn phụ thuộc vào tập nghiệm con S . Nếu S không chứa các nghiệm quan trọng, loss ranking có thể không phản ánh đúng regret thật. Ngoài ra, việc mở rộng S vẫn cần gọi solver trong quá trình huấn luyện, dù số lần gọi solver có thể được kiểm soát.

Một hướng cải tiến là thiết kế chiến lược chọn nghiệm S thông minh hơn, chẳng hạn ưu tiên các nghiệm gần tối ưu, các nghiệm dễ gây nhầm lẫn hoặc các nghiệm nằm

gần biên quyết định của mô hình. Điều này tương tự hard negative mining trong ranking và retrieval.

10.3 Bài báo 3: Linear Label Ranking with Bounded Noise

Bài báo *Linear Label Ranking with Bounded Noise* nghiên cứu bài toán label ranking dưới góc nhìn lý thuyết học máy. Khác với các bài báo thực nghiệm về ranking trong truy xuất thông tin, bài báo này tập trung vào khả năng học một hàm xếp hạng tuyến tính khi dữ liệu nhãn ranking có nhiều bị chặn.

10.3.1 Bài toán đặt ra

Trong label ranking, mỗi đầu vào $x \in \mathbb{R}^d$ không chỉ có một nhãn đơn, mà có một hoán vị trên k nhãn. Một lớp mô hình tự nhiên là Linear Sorting Function, trong đó mỗi nhãn i có một vector trọng số W_i , và ranking được tạo bằng cách sắp xếp các điểm:

$$W_1x, W_2x, \dots, W_kx.$$

Hàm mục tiêu có dạng:

$$\sigma_W(x) = \text{argsort}(Wx). \quad (83)$$

Bài toán đặt ra là: nếu các ranking quan sát được bị nhiễu nhưng xác suất đảo sai mỗi cặp bị chặn bởi một hằng số $\eta < 1/2$, liệu ta có thể học được một hàm xếp hạng tuyến tính gần với hàm mục tiêu hay không? Độ gần được đo bằng Kendall tau distance, tức là tỉ lệ cặp nhãn bị đảo thứ tự.

10.3.2 Đóng góp chính

Đóng góp chính của bài báo là đưa ra thuật toán học cho Linear Sorting Functions trong điều kiện bounded noise. Ý tưởng cơ bản là chuyển bài toán ranking nhiều nhãn thành nhiều bài toán phân loại nhị phân theo cặp nhãn.

Cụ thể, với mỗi cặp nhãn (i, j) , ta xét xem trong ranking quan sát được nhãn nào đứng trước. Điều này tạo ra một bài toán học halfspace với nhãn nhị phân. Vì nhiễu trên ranking bị chặn, mỗi bài toán nhị phân này có thể được xem như học halfspace dưới Massart noise. Sau khi học các bộ phân loại theo từng cặp, thuật toán ghép các quan hệ cặp thành một ranking toàn cục.

Khi các quan hệ cặp tạo ra chu trình, bài báo dùng bài toán minimum feedback arc set để loại bỏ các mâu thuẫn và tạo ra một ranking hợp lệ. Kết quả lý thuyết cho thấy có thể đạt sai số nhỏ theo Kendall tau distance với số mẫu và thời gian chạy đa thức theo các tham số chính.

Bài báo cũng trình bày phiên bản proper learning, trong đó thuật toán không chỉ tạo ra một bộ phân loại cặp riêng lẻ, mà còn tìm lại một Linear Sorting Function thật sự. Điều này được thực hiện thông qua một chương trình lỗi trên ma trận W .

10.3.3 Liên hệ với lý thuyết trong chương

Bài báo này liên hệ trực tiếp với phần preference-based ranking và Kendall tau distance. Trong preference-based ranking, ta học quan hệ ưu tiên theo từng cặp rồi cần chuyển các quan hệ đó thành ranking toàn cục. Bài báo này làm đúng điều đó trong bối cảnh label ranking: mỗi cặp nhãn (i, j) được học bằng một halfspace, sau đó các dự đoán cặp được ghép thành một thứ tự đầy đủ.

Bài báo cũng liên hệ với phần sort-by-degree và randomized QuickSort theo nghĩa rộng: cả ba đều xử lý bài toán chuyển quan hệ cặp thành ranking toàn cục. Tuy nhiên, bài báo này dùng công cụ lý thuyết mạnh hơn, cụ thể là đồ thị tournament và minimum feedback arc set, để xử lý trường hợp quan hệ cặp không nhất quán.

Ngoài ra, việc dùng Kendall tau distance làm thước đo sai số phù hợp với phần tiêu chí đánh giá ranking đã trình bày. Kendall tau đo tỉ lệ cặp bị đảo, nên rất tự nhiên khi phân tích các thuật toán học dựa trên cặp nhãn.

10.3.4 Hạn chế và nhận xét

Hạn chế chính của hướng tiếp cận này là tính lý thuyết cao và giả định tương đối mạnh. Phân tích thường dựa trên mô hình nhiễu bị chặn và phân phối đầu vào có cấu trúc tốt, chẳng hạn Gaussian hoặc log-concave. Trong dữ liệu thực tế, nhiễu có thể không đồng đều, phụ thuộc vào từng vùng dữ liệu hoặc từng cặp nhãn.

Ngoài ra, số lượng cặp nhãn là $O(k^2)$, nên khi số nhãn k lớn, việc học tất cả các bộ phân loại cặp có thể tốn kém. Đây cũng là hạn chế chung của nhiều phương pháp pairwise ranking.

Một hướng mở là kết hợp lý thuyết bounded noise với các mô hình thực nghiệm hiện đại, chẳng hạn neural ranking hoặc gradient boosting, để vừa giữ được bảo đảm lý thuyết vừa có khả năng áp dụng trên dữ liệu lớn.

10.4 Nhận xét tổng quan về xu hướng phát triển

Từ ba bài báo trên, có thể thấy learning-to-rank đang phát triển theo ba xu hướng chính.

10.4.1 Từ pairwise ranking sang tối ưu metric trực tiếp

Các phương pháp cổ điển như Ranking SVM hay RankBoost thường tối ưu pairwise loss. Tuy nhiên, trong các ứng dụng thực tế, metric cuối cùng thường là NDCG, MAP, MRR hoặc ERR. Bài báo về YetiLoss cho thấy xu hướng hiện đại là thiết kế thuật toán sao

cho gần hơn với metric thực sự được quan tâm. Thay vì chỉ đếm số cặp đúng sai, các thuật toán mới cố gắng đưa thông tin về vị trí trong danh sách và độ quan trọng của từng metric vào gradient.

10.4.2 Sử dụng smoothing và surrogate loss để xử lý tính không khả vi

Một vấn đề trung tâm của ranking là phép sắp xếp không khả vi. Các bài báo hiện đại xử lý vấn đề này theo hai hướng: xây dựng surrogate loss khả vi hoặc làm trơn ranking loss bằng nhiều ngẫu nhiên. LambdaMART dùng upper bound khả vi, YetiRank và YetiLoss dùng smoothing trên thứ tự ranking, còn StochasticRank tối ưu loss được làm trơn trực tiếp. Điều này cho thấy smoothing và surrogate loss là hai công cụ quan trọng để kết nối ranking metric rời rạc với các thuật toán tối ưu gradient.

10.4.3 Mở rộng learning-to-rank ra ngoài truy xuất thông tin

Bài báo về decision-focused learning cho thấy learning-to-rank không chỉ dùng cho tìm kiếm tài liệu hay hệ gợi ý. Khi các nghiệm khả thi của một bài toán tối ưu tổ hợp được xem như các items cần xếp hạng, ta có thể dùng ranking loss để huấn luyện mô hình dự đoán tham số. Đây là một hướng mở rộng quan trọng, vì nó biến learning-to-rank thành một công cụ chung cho các bài toán ra quyết định.

10.4.4 Tăng cường phân tích lý thuyết cho ranking có nhiễu

Bài báo về Linear Label Ranking with Bounded Noise cho thấy một hướng khác: xây dựng bảo đảm học được trong môi trường có nhiễu. Thay vì chỉ đánh giá thực nghiệm, bài báo phân tích số mẫu, độ phức tạp thuật toán và sai số theo Kendall tau distance. Điều này bổ sung nền tảng lý thuyết cho các phương pháp pairwise và preference-based ranking.

10.5 Hạn chế chung của các phương pháp

Dù các hướng tiếp cận trên có nhiều đóng góp, vẫn còn một số hạn chế chung.

Thứ nhất, nhiều phương pháp ranking vẫn phụ thuộc mạnh vào surrogate loss. Surrogate loss giúp tối ưu dễ hơn, nhưng không phải lúc nào cũng khớp hoàn toàn với metric cuối cùng. Ví dụ, tối ưu pairwise loss có thể không tối ưu tốt NDCG@10 nếu các lỗi ở đầu danh sách quan trọng hơn nhiều so với lỗi ở cuối danh sách.

Thứ hai, các phương pháp pairwise thường có chi phí lớn vì số cặp tăng theo $O(n^2)$ hoặc $O(k^2)$. Điều này xuất hiện trong Ranking SVM, RankBoost, preference-based ranking và cả label ranking theo cặp nhãn. Khi số đối tượng hoặc số nhãn lớn, cần có chiến lược lấy mẫu cặp hoặc chọn cặp quan trọng.

Thứ ba, nhiều kết quả lý thuyết cần giả định khá mạnh, chẳng hạn nhiều bị chặn, phân phối đầu vào chuẩn hoặc log-concave. Những giả định này giúp phân tích rõ ràng, nhưng có thể không phản ánh đầy đủ dữ liệu thực tế.

Thứ tư, trong các hệ thống thực tế, ranking thường chịu ảnh hưởng của nhiều yếu tố khác như fairness, robustness, distribution shift, cold-start và feedback loop. Các yếu tố này chưa được xử lý đầy đủ trong các mô hình lý thuyết cơ bản.

10.6 Câu hỏi mở và ý tưởng cải tiến

Một số câu hỏi mở tự nhiên từ các bài báo trên là:

- Làm thế nào để tối ưu trực tiếp các metric như NDCG, MAP hoặc MRR mà vẫn có bảo đảm hội tụ và tổng quát hoá?
- Có thể thiết kế surrogate loss vừa khả vi, vừa bám sát metric cuối cùng hơn pairwise hinge loss hay không?
- Trong preference-based ranking, làm thế nào để xử lý quan hệ ưu tiên không bắc cầu một cách hiệu quả khi số đối tượng rất lớn?
- Với decision-focused learning, nên chọn tập nghiệm con S như thế nào để vừa giảm chi phí gọi solver, vừa giữ được thông tin quan trọng cho regret?
- Các bảo đảm lý thuyết trong label ranking có thể mở rộng sang neural networks hoặc dữ liệu không Gaussian hay không?

Từ các câu hỏi trên, có thể đề xuất một số ý tưởng cải tiến. Với các thuật toán pairwise, có thể dùng hard pair mining, tức là chỉ chọn các cặp mà mô hình đang dễ nhầm hoặc các cặp ảnh hưởng lớn đến metric top- k . Với decision-focused learning, có thể cập nhật tập nghiệm S bằng cách kết hợp nghiệm tối ưu thật, nghiệm dự đoán hiện tại và các nghiệm gần tối ưu. Với các thuật toán như YetiLoss, có thể thử nghiệm các phân phối smoothing khác nhau hoặc học tham số smoothing thay vì cố định trước.

10.7 Kết luận

Ba bài báo được trình bày cho thấy learning-to-rank là một lĩnh vực có cả chiều sâu lý thuyết lẫn giá trị thực tiễn. Bài báo về YetiLoss tập trung vào tối ưu metric ranking trong các mô hình GBDT hiện đại. Bài báo về decision-focused learning cho thấy ranking có thể được dùng để huấn luyện mô hình ra quyết định trong tối ưu tổ hợp. Bài báo về Linear Label Ranking with Bounded Noise cung cấp góc nhìn lý thuyết về khả năng học ranking khi dữ liệu có nhiễu.

Nhìn chung, các bài báo đều nhấn mạnh cùng một ý tưởng cốt lõi: chất lượng của mô hình ranking không chỉ nằm ở việc dự đoán đúng từng điểm riêng lẻ, mà nằm ở việc

bảo toàn đúng quan hệ thứ tự giữa các đối tượng. Đây cũng chính là tinh thần xuyên suốt của chương về learning-to-rank.

11 Kết luận

Chương này đã trình bày tổng quan về bài toán *learning-to-rank*, từ các khái niệm nền tảng đến một số thuật toán, tiêu chí đánh giá, thực nghiệm minh họa và các hướng nghiên cứu liên quan. Khác với bài toán phân loại hoặc hồi quy thông thường, ranking không chỉ quan tâm đến dự đoán trên từng đối tượng riêng lẻ, mà tập trung vào quan hệ thứ tự giữa các đối tượng. Đây là điểm làm cho ranking trở thành một bài toán quan trọng trong nhiều ứng dụng như truy xuất thông tin, công cụ tìm kiếm, hệ gợi ý, phát hiện gian lận và ra quyết định tối ưu.

Trước hết, chương đã giới thiệu các thiết lập cơ bản của ranking, bao gồm *score-based ranking* và *preference-based ranking*. Trong score-based ranking, mô hình học một hàm điểm

$$h : \mathcal{X} \rightarrow \mathbb{R},$$

sau đó sinh ranking bằng cách sắp xếp các đối tượng theo điểm số. Trong preference-based ranking, mô hình học trực tiếp quan hệ ưu tiên giữa từng cặp đối tượng. Hai thiết lập này cho thấy ranking có thể được tiếp cận theo nhiều cách khác nhau, tùy thuộc vào dạng dữ liệu huấn luyện và mục tiêu ứng dụng.

Tiếp theo, chương đã trình bày các khái niệm lý thuyết quan trọng như lỗi xếp hạng theo từng cặp, margin, pairwise loss và chặn tổng quát hoá dựa trên Rademacher complexity. Các kết quả này cho thấy để một mô hình ranking có khả năng tổng quát hoá tốt, ta cần đồng thời kiểm soát lỗi thực nghiệm và độ phức tạp của lớp giả thuyết. Đây là cơ sở lý thuyết cho các phương pháp ranking dựa trên margin.

Dựa trên các ý tưởng đó, chương đã trình bày hai thuật toán tiêu biểu là *Ranking SVM* và *RankBoost*. Ranking SVM chuyển bài toán ranking theo từng cặp thành một bài toán phân loại nhị phân trên các vector hiệu, từ đó tận dụng được nguyên lý margin của SVM. RankBoost sử dụng tư tưởng boosting, duy trì trọng số trên các cặp đối tượng và tập trung dần vào các cặp khó. Hai thuật toán này minh họa rõ cách các bài toán ranking có thể được giải quyết bằng cách chuyển về các bài toán học máy quen thuộc hơn.

Chương cũng đã xét trường hợp đặc biệt *bipartite ranking*, trong đó mục tiêu là xếp các điểm positive cao hơn các điểm negative. Trong thiết lập này, AUC là một độ đo tự nhiên vì nó có thể được hiểu là xác suất một điểm positive được xếp cao hơn một điểm negative. Khi bỏ qua trường hợp hoà điểm, ta có quan hệ quan trọng:

$$\text{AUC}(h) = 1 - R(h),$$

trong đó $R(h)$ là ranking loss trong bipartite ranking. Quan hệ này cho thấy AUC không chỉ là một metric thực nghiệm, mà còn có diễn giải trực tiếp dưới góc nhìn pairwise ranking.

Bên cạnh đó, chương đã thảo luận nhiều tiêu chí đánh giá ranking như pairwise accuracy, ranking loss, Kendall tau distance, AUC, Precision@ k , MAP, MRR và NDCG. Mỗi tiêu chí phản ánh một khía cạnh khác nhau của chất lượng ranking. Các tiêu chí pairwise phù hợp với phân tích lý thuyết và các mô hình học theo cặp, trong khi các tiêu chí như NDCG, MAP và MRR phù hợp hơn với các hệ thống truy xuất thông tin thực tế, nơi các vị trí đầu danh sách có vai trò đặc biệt quan trọng.

Phần thực nghiệm đã minh họa một mô hình pairwise ranking tuyến tính được cài đặt từ đầu bằng Python và NumPy. Kết quả cho thấy mô hình có thể học tốt quan hệ ưu tiên khi dữ liệu được sinh từ một hàm tuyến tính và mức nhiễu không quá lớn. Thí nghiệm cũng cho thấy khi nhiễu tăng, chất lượng ranking giảm rõ rệt; trong khi khi kích thước mẫu tăng, mô hình có xu hướng học ổn định hơn. Các kết quả này phù hợp với lý thuyết: ranking tốt đòi hỏi mô hình bảo toàn được càng nhiều quan hệ ưu tiên theo cặp càng tốt.

Cuối cùng, chương đã tổng hợp một số bài báo liên quan để cho thấy learning-to-rank vẫn là một hướng nghiên cứu đang phát triển mạnh. Các nghiên cứu hiện đại không chỉ dừng lại ở pairwise loss cổ điển, mà còn hướng đến tối ưu trực tiếp các metric như NDCG, MAP, MRR, mở rộng ranking sang decision-focused learning, và xây dựng bảo đảm lý thuyết cho label ranking trong điều kiện có nhiễu. Điều này cho thấy learning-to-rank vừa có nền tảng lý thuyết phong phú, vừa có nhiều ứng dụng thực tiễn.

Tóm lại, learning-to-rank là một bài toán trung tâm trong học máy hiện đại, đặc biệt khi mục tiêu không chỉ là dự đoán đúng từng điểm dữ liệu mà là tạo ra một thứ tự có ý nghĩa. Các mô hình và lý thuyết trong chương cho thấy bản chất của ranking nằm ở việc học và bảo toàn quan hệ ưu tiên giữa các đối tượng. Đây cũng là nền tảng để phát triển các hệ thống ranking hiệu quả hơn trong thực tế, đặc biệt trong những ứng dụng mà chất lượng của các vị trí đầu danh sách có ảnh hưởng trực tiếp đến trải nghiệm người dùng và giá trị của hệ thống.

A Phụ lục: Cấu trúc mã nguồn và hướng dẫn tái tạo thực nghiệm

Phụ lục này trình bày cấu trúc thư mục mã nguồn, vai trò của từng thành phần và hướng dẫn chạy lại các thực nghiệm trong báo cáo. Toàn bộ phần thực nghiệm được tổ chức trong thư mục `code/`.

A.1 Cấu trúc thư mục

Cấu trúc thư mục chính của project như sau:

```
1 code/  
2 |-- experiments/  
3 |-- results/  
4 |-- src/  
5 |-- tests/  
6 |-- README.md  
7 |-- requirements.txt  
8 |-- run_experiments.py  
9 |-- run_main.py
```

Trong đó:

- `experiments/`: chứa các script hoặc cấu hình phục vụ cho các thí nghiệm, chẳng hạn thí nghiệm chính, thí nghiệm thay đổi mức nhiễu và thí nghiệm thay đổi kích thước mẫu.
- `results/`: lưu kết quả đầu ra của quá trình thực nghiệm, bao gồm các bảng kết quả, file CSV và hình minh họa được sử dụng trong báo cáo.
- `src/`: chứa mã nguồn chính của chương trình, chẳng hạn các hàm sinh dữ liệu, mô hình ranking tuyến tính, hàm mất mát, hàm huấn luyện và các độ đo đánh giá.
- `tests/`: chứa các kiểm thử đơn giản nhằm đảm bảo các thành phần quan trọng như sinh cặp ưu tiên, tính loss và tính metric hoạt động đúng.
- `README.md`: mô tả ngắn gọn project, cách cài đặt môi trường và cách chạy chương trình.
- `requirements.txt`: liệt kê các thư viện Python cần thiết để chạy lại thực nghiệm.
- `run_main.py`: script chạy thực nghiệm chính với thiết lập mặc định.
- `run_experiments.py`: script chạy toàn bộ các thực nghiệm, bao gồm thí nghiệm chính, thí nghiệm theo mức nhiễu và thí nghiệm theo kích thước mẫu.

A.2 Môi trường thực nghiệm

Các thực nghiệm được cài đặt bằng Python. Các thư viện chính được sử dụng gồm:

- `numpy`: dùng cho các phép toán vector, ma trận, sinh dữ liệu và cập nhật trọng số.
- `matplotlib`: dùng để vẽ các hình minh hoạ như đường cong loss, so sánh ranking và các biểu đồ thí nghiệm.
- `csv`: dùng để lưu kết quả thực nghiệm dưới dạng bảng.
- `pathlib`: dùng để quản lý đường dẫn thư mục và file.

Để cài đặt các thư viện cần thiết, chạy lệnh:

```
1 pip install -r requirements.txt
```

A.3 Cách chạy thực nghiệm chính

Để chạy thực nghiệm chính với cấu hình mặc định, sử dụng lệnh:

```
1 python run_main.py
```

Script này thực hiện các bước sau:

Bước 1. Sinh dữ liệu tổng hợp theo mô hình tuyến tính.

Bước 2. Sinh các cặp ưu tiên từ điểm xếp hạng thật.

Bước 3. Khởi tạo mô hình scoring tuyến tính.

Bước 4. Huấn luyện mô hình bằng pairwise hinge loss.

Bước 5. Tính các metric đánh giá như pairwise accuracy, ranking loss, Kendall tau distance và top- k overlap.

Bước 6. Lưu kết quả và hình minh hoạ vào thư mục `results/`.

A.4 Cách chạy toàn bộ thực nghiệm

Để chạy toàn bộ các thí nghiệm được trình bày trong báo cáo, sử dụng lệnh:

```
1 python run_experiments.py
```

Script này bao gồm ba nhóm thực nghiệm:

- **Thực nghiệm chính:** huấn luyện mô hình với thiết lập mặc định.

- **Thí nghiệm theo mức nhiễu:** thay đổi độ lệch chuẩn của nhiễu Gaussian để kiểm tra ảnh hưởng của nhiễu đến chất lượng ranking.
- **Thí nghiệm theo kích thước mẫu:** thay đổi số lượng mẫu huấn luyện để kiểm tra ảnh hưởng của kích thước dữ liệu.

Các kết quả sau khi chạy được lưu trong thư mục **results/**. Nếu các hình trong báo cáo được đặt trong thư mục **figures/**, có thể sao chép hoặc xuất các hình từ **results/** sang **figures/** để sử dụng trong file $\text{\LaTeX}$.

A.5 Các file kết quả

Sau khi chạy thực nghiệm, chương trình tạo ra các kết quả phục vụ cho phần báo cáo. Các file thường bao gồm:

- Bảng kết quả thực nghiệm chính.
- Bảng kết quả khi thay đổi mức nhiễu.
- Bảng kết quả khi thay đổi kích thước mẫu.
- Hình đường cong pairwise hinge loss.
- Hình so sánh ranking thật và ranking dự đoán.
- Hình ảnh hưởng của nhiễu đến chất lượng ranking.
- Hình ảnh hưởng của kích thước mẫu đến chất lượng ranking.

Các bảng kết quả được dùng để tạo các bảng trong Section 9, còn các hình được dùng cho các minh hoạ như đường cong hội tụ và phân tích ảnh hưởng của tham số.

A.6 Khả năng tái tạo

Để đảm bảo kết quả có thể tái tạo, chương trình sử dụng random seed cố định trong quá trình sinh dữ liệu và khởi tạo mô hình. Các tham số chính của thực nghiệm, chẳng hạn số mẫu, số đặc trưng, learning rate, số epoch và hệ số regularization, được đặt rõ trong script thực nghiệm.

Khi chạy lại trên cùng môi trường và cùng random seed, các kết quả thu được sẽ gần như trùng khớp với kết quả đã trình bày trong báo cáo. Sai khác nhỏ có thể xuất hiện nếu phiên bản thư viện hoặc môi trường chạy khác nhau.

A.7 Ghi chú về kiểm thử

Thư mục `tests/` được dùng để lưu các kiểm thử cơ bản cho mã nguồn. Các kiểm thử này giúp đảm bảo rằng:

- hàm sinh cặp ưu tiên trả về đúng số lượng cặp;
- nhãn cặp ưu tiên phù hợp với điểm xếp hạng thật;
- pairwise hinge loss không âm;
- ranking loss và pairwise accuracy có quan hệ đúng;
- các metric trả về giá trị trong miền hợp lệ.

Nếu có cấu hình kiểm thử bằng `pytest`, có thể chạy:

```
1 pytest tests/
```

Nếu project không dùng `pytest`, phần kiểm thử có thể được chạy trực tiếp bằng các script Python trong thư mục `tests/`.

Tài liệu

- [1] M. Mohri, A. Rostamizadeh **and** A. Talwalkar, *Foundations of Machine Learning*, 2 **edition**. MIT Press, 2018.
- [2] T.-Y. Liu, *Learning to Rank for Information Retrieval*. Springer, 2011.
- [3] T. Joachims, ?Optimizing Search Engines Using Clickthrough Data,? **in** *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* ACM, 2002, **pages** 133–142.
- [4] Y. Freund, R. Iyer, R. E. Schapire **and** Y. Singer, ?An Efficient Boosting Algorithm for Combining Preferences,? *Journal of Machine Learning Research*, **journal** 4, **pages** 933–969, 2003.
- [5] C. J. C. Burges **and others**, ?Learning to Rank Using Gradient Descent,? **in** *Proceedings of the 22nd International Conference on Machine Learning* ACM, 2005, **pages** 89–96.
- [6] C. J. C. Burges, ?From RankNet to LambdaRank to LambdaMART: An Overview,? **in** *Microsoft Research Technical Report* 2010.
- [7] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai **and** H. Li, ?Learning to Rank: From Pairwise Approach to Listwise Approach,? **in** *Proceedings of the 24th International Conference on Machine Learning* ACM, 2007, **pages** 129–136.
- [8] D. Fotakis, A. Kalavasis, V. Kontonis **and** C. Tzamos, ?Linear Label Ranking with Bounded Noise,? **in** *Advances in Neural Information Processing Systems* **volume** 35, 2022.
- [9] J. Mandi, V. Bucarey, M. Mulamba **and** T. Guns, ?Decision-Focused Learning: Through the Lens of Learning to Rank,? **in** *Proceedings of the 39th International Conference on Machine Learning* **journal** Proceedings of Machine Learning Research, **volume** 162, PMLR, 2022, **pages** 14 935–14 947.
- [10] I. Lyzhin, A. Ustimenko, A. Gulin **and** L. Prokhorenkova, ?Which Tricks are Important for Learning to Rank?? **in** *Proceedings of the 40th International Conference on Machine Learning* **journal** Proceedings of Machine Learning Research, **volume** 202, PMLR, 2023, **pages** 23 264–23 278.